

LLMsVerifier Full Integration Plan for HelixTranslate

Single Source of Truth for Enterprise-Grade Model Verification, Scoring, and Selection

Version: 1.0 **Date:** May 1, 2026 **Status:** Implementation-Ready **Classification:** Internal — Engineering Teams

Prepared for: HelixDevelopment Engineering Leadership **Scope:** Full integration of LLMsVerifier (digital.vasic.llmsverifier) into HelixTranslate as the single source of truth for model provisioning

Repository References:

- HelixTranslate: [git@github.com:HelixDevelopment/HelixTranslate.git](https://github.com:HelixDevelopment/HelixTranslate.git) (Go 1.25.2, ~936 files)
- LLMsVerifier: <https://github.com/vasic-digital/LLMsVerifier> (Go 1.25.3, 1,207 files, 25+ providers)
- HelixAgent: <https://github.com/HelixDevelopment/HelixAgent> (Go 1.26, reference implementation)

Table of Contents

1. Executive Summary & Integration Architecture
2. Phase 1: Foundation and Repository Preparation
3. Phase 2: Core Verification Engine Integration
4. Phase 3: Scoring Engine and Score Adapter
5. Phase 4: Model Discovery and Registry Integration
6. Phase 5: Provider Factory and Runtime Integration
7. Phase 6: Enterprise UX and User-Facing Features
8. Phase 7: Testing Strategy and Anti-Bluff Quality Assurance
9. Phase 8: Documentation, Deployment, and Operational Runbook

Chapter 1: Executive Summary & Integration Architecture

1.1 Integration Vision and Goals

The integration of LLMsVerifier into HelixTranslate represents a fundamental architectural transformation: the replacement of nine manually configured LLM providers with a unified, verification-driven provider management system that treats model quality as a first-class operational concern. This is not an incremental feature addition. It is a structural redefinition of how HelixTranslate discovers, validates, scores, and selects translation models. The central mandate is that **LLMsVerifier becomes the single source of truth** for all model availability, validation status, and performance ranking within the HelixTranslate ecosystem. No model reaches the user interface without passing through LLMsVerifier's eight-step verification pipeline. No model participates in translation without carrying a computed score from the five-component scoring engine. No provider remains invisible to the discovery system. These constraints are non-negotiable by design.

The current HelixTranslate architecture maintains provider integrations as discrete, hand-coded implementations scattered across `pkg/translator/llm/`. Each provider — whether OpenAI, Anthropic, Google Vertex, Azure OpenAI, Cohere, Mistral, Together AI, or local Ollama instances — exists as a standalone factory implementation that must be manually registered, individually configured with API keys, and separately maintained when endpoints or model catalogs change. This pattern, while functional for a small number of providers, creates compounding maintenance overhead as the LLM marketplace expands. Adding a tenth provider requires touching at minimum three files: the provider implementation, the factory registration, and the configuration schema. Adding a twentieth provider becomes a development sprint unto itself. The target architecture eliminates this friction entirely by delegating all provider lifecycle management to LLMsVerifier’s automated discovery and verification pipeline.

Three UX guarantees govern this integration and serve as the definition of done for every implementation phase. **First: only validated models are available.** Any model that has not completed the full eight-step verification pipeline — covering API reachability, authentication, model existence, response format validation, latency measurement, capability detection, rate limit compliance, and error handling behavior — is invisible to the HelixTranslate runtime. There is no bypass mechanism, no emergency override, no administrator backdoor. **Second: only positively scored models participate in translation.** The five-component scoring engine evaluates latency, throughput, cost efficiency, capability breadth, and reliability history. A model that passes verification but receives a composite score below the configurable threshold is gated from selection. **Third: automatic discovery of all supported providers.** When a new provider appears in the LLMsVerifier registry, or when an existing provider adds new models, HelixTranslate learns of them without code changes, configuration edits, or deployments. The discovery tier system — primary, secondary, and tertiary — polls provider registries, public model databases, and community endpoints on configurable intervals, feeding a continuously updated model catalog that the translation engine consumes in real time.

These guarantees translate directly into operational outcomes. Translation quality improves because only proven-capable models serve requests. Operational costs decrease because cost-efficiency scoring automatically prefer low-price-per-token models for appropriate workloads. Incident recovery accelerates because the health-check layer automatically removes degraded providers from rotation without human intervention. Development velocity increases because adding provider support requires zero code changes in HelixTranslate proper.

1.2 Reference Architecture from HelixAgent

The integration blueprint is drawn directly from LLMsVerifier’s production implementation in the HelixAgent project, where the eight-phase startup pipeline has been running in production environments for multiple release cycles. Understanding this pipeline is prerequisite to mapping its components correctly into HelixTranslate’s codebase. The pipeline, implemented in `internal/verifier/startup.go`, executes sequentially during HelixAgent process initialization and establishes the complete verification and scoring infrastructure before any translation request is accepted.

The pipeline begins with **Phase 1: Config Load**. The `LoadVerifierConfig()` function reads `configs/verifier.yaml` and unmarshals it into the `VerifierConfig` struct defined in `internal/verifier/config.go`. This configuration specifies provider API keys, discovery intervals, verification timeout thresholds, scoring weights, health check frequencies, and gating thresholds. The configuration is validated against a JSON Schema embedded at `internal/verifier/config/schema.json` — any missing required field or type mismatch causes a fatal startup error. This strict validation ensures that the verification pipeline never operates with incomplete or ambiguous configuration.

Phase 2: StartupVerifier Initialization instantiates the `StartupVerifier` struct, the central coordinator for all subsequent phases. This struct maintains references to the discovery service, verification pipeline, scoring engine, model registry, and health monitor. It also owns a `sync.WaitGroup` that gates server startup until all asynchronous initialization completes. The `StartupVerifier.Init()` method allocates these subsystems but does not yet execute them — that sequencing is controlled by the subsequent phases.

Phase 3: Provider Discovery triggers the three-tier discovery system implemented in `internal/verifier/discovery.go`. The **primary tier** queries well-known provider endpoints — OpenAI’s `/models` endpoint, Anthropic’s model listing API, Google’s model catalog — using the stored API keys from configuration. The **secondary tier** scans public model registries including Hugging Face’s API, Replicate’s model listing, and Together AI’s model catalog. The **tertiary tier** probes community-discovered endpoints and user-configured custom providers. Each tier runs in its own goroutine and writes discovered models into a thread-safe `DiscoveryResult` channel. Discovery respects rate limits through a token bucket implemented in `internal/verifier/ratelimit/bucket.go`.

Phase 4: Provider Verification executes the eight-step verification pipeline for every discovered model, implemented in `internal/verifier/verification.go`. The eight steps are: (1) `CheckAPIReachability` — confirms the provider endpoint responds to HTTP requests within the configured timeout; (2) `CheckAuthentication` — validates that the stored API key has necessary permissions; (3) `CheckModelExistence` — confirms the specific model ID is available on the provider; (4) `ValidateResponseFormat` — sends a minimal prompt and confirms the response matches expected schema; (5) `MeasureLatency` — records p50, p95, and p99 response times over 10 warmup requests; (6) `DetectCapabilities` — identifies supported features (function calling, JSON mode, vision, streaming, tool use); (7) `CheckRateLimits` — determines request-per-minute and token-per-minute ceilings; (8) `ValidateErrorHandling` — confirms graceful error responses for malformed requests and rate limit exceeded scenarios. Each step produces a `VerificationStepResult` that feeds into the aggregate `ProviderVerification` record. A single step failure does not abort the pipeline — remaining steps continue so that the scoring engine can operate on partial data where appropriate.

Phase 5: Provider Scoring invokes the five-component scoring engine defined in `internal/verifier/scoring.go`. Each model receives five component scores: **Latency Score** (lower p95 latency = higher score, logarithmic scale), **Throughput Score** (higher sustainable tokens-per-second = higher score), **CostEfficiency Score** (lower cost per million tokens = higher score, normalized against market median), **Capability Score** (breadth of detected capabilities mapped to a rubric), and **Reliability Score** (historical uptime percentage from health check records). The composite score is a weighted sum: $Composite = w1*Latency + w2*Throughput + w3*CostEfficiency + w4*Capability + w5*Reliability$, where weights default to 0.20 each but are configurable per-deployment. Scores persist to `internal/verifier/persistence/scores.db` (SQLite) with timestamps for trend analysis.

Phase 6: Ranking sorts all verified models by composite score descending and assigns rank positions. Ties are broken by latency score (lower wins), then by capability score (higher wins). The ranked list is exposed through `internal/verifier/registry.go` as a thread-safe slice with copy-on-read semantics. The registry also maintains a map from `provider/model-id` tuples to their current rank and score, enabling $O(1)$ lookups during translation request routing.

Phase 7: AI Debate Team Selection configures the multi-model consensus layer used for high-confidence translations. This component, implemented in `internal/verifier/debate.go`, selects the top-N models from the ranked list (default $N=3$) to serve as a “debate team” — each model independently translates the same input, and their outputs are compared for consistency. Divergence triggers automatic

re-translation with a confidence warning. This layer is the final consumer of the scoring pipeline’s output.

Phase 8: Server Start releases the `StartupVerifier.WaitGroup`, allowing the main HTTP/gRPC servers in `cmd/server/main.go` to begin accepting connections. All phases 1-7 must complete before this release; the server literally cannot start with unverified or unranked models in its configuration.

The score adapter pattern, extracted from `internal/services/llmsverifier_score_adapter.go`, serves as the critical bridge between LLMsVerifier’s scoring world and HelixTranslate’s provider selection logic. This adapter implements a single exported function: `Adapt(scores []verifier.ModelScore) ([]translation.ProviderPreference, error)`. It translates LLMsVerifier’s internal score representation — which includes detailed component scores, rank metadata, and capability flags — into HelixTranslate’s `ProviderPreference` struct, which carries only the fields the translation engine needs: `ProviderID`, `ModelID`, `Weight`, and `FallbackOrder`. The adapter is the single point of coupling between the two systems, deliberately designed to absorb schema changes on either side without propagating them. When LLMsVerifier adds a new scoring component, only the adapter’s weight calculation changes. When HelixTranslate changes its preference format, only the adapter’s output struct changes.

1.3 Component Architecture and Data Flow

The integration maps components across three repositories: HelixAgent (the source of proven, production LLMsVerifier code), LLMsVerifier (the API and library boundary), and HelixTranslate (the target system). The mapping is not a naive copy-paste operation. Each HelixAgent source file provides a behavioral specification that is consumed through the LLMsVerifier library API and reimplemented in HelixTranslate with domain-specific adaptations for the translation workload.

TABLE 1: Complete Component Mapping Across Repositories

HelixAgent Source File	LLMsVerifier API	HelixTranslate Target File	Integration Pattern
internal/verifier/startup.go	llmverifier.New(config *Config) (*Verifier, error)	internal/verifier/client.go	Construction: HelixTranslate instantiates the verifier client during its own startup sequence, passing translated configuration
internal/verifier/scoring.go	scoring.NewScoringEngine(cfg internal/verifier/ScoringConfig) *Engine	scoring/engine.go	Delegation: HelixTranslate wraps the scoring engine with translation-specific weight defaults and capability-to-task mappings
internal/verifier/discovery.go	discovery.NewService(cfg DiscoveryConfig) *Service	internal/verifier/discovery/service.go	Embedding: Discovery service runs as a background goroutine started by the verifier client, writing results to a shared registry

Table 1 – continued

HelixAgent Source File	LLMsVerifier API	HelixTranslate Target File	Integration Pattern
internal/verifier/verification.go	verifier.Verify(ctx, provider, model) (*VerificationResult, error)	internal/verifier/pipeline.go	Orchestration: HelixTranslate' s pipeline coordinates LLMsVerifier' s verification steps with its own health check and circuit breaker logic
internal/verifier/registry.go	registry.GetRankedModels() []RankedModel	internal/verifier/registry/adapter.go	Translation: Ranked models from LLMsVerifier' s registry are adapted into HelixTranslate' s provider preference list at configurable intervals
internal/verifier/health.go	health.NewMonitor(cfg HealthConfig) *Monitor	internal/verifier/health/monitor.go	Integration: Health monitor feeds status updates directly into HelixTranslate' s existing circuit breaker in pkg/circuitbreaker/
internal/services/llmsverifier_score_adapter.go	ScoreAdapter.Adapt(scores []ModelScore) []ProviderPreference	internal/services/llmsverifier_score_adapter.go	Direct Port: The adapter file is ported nearly verbatim, with struct field names updated for HelixTranslate conventions
internal/verifier/debate.go	debate.NewTeam(cfg DebateConfig) *Team	internal/verifier/debate/team.go	Extension: Debate team configuration is extended with translation-specific consensus heuristics
internal/verifier/persistence.go	persistence.NewStore(path string) (*Store, error)	internal/verifier/persistence/store.go	Shared Store: SQLite score database is co-located with HelixTranslate' s data directory for unified backup
internal/verifier/ratelimit/bucket.go	ratelimit.NewBucket(rate, burst int) *Bucket	internal/verifier/ratelimit/bucket.go	Direct Port: Token bucket implementation is dependency-free and copies directly

The data flow through this architecture follows a directed pipeline with clear ownership boundaries. At the leftmost intake, LLMsVerifier's discovery service executes its three-tier poll cycle: primary tier queries provider APIs every 15 minutes, secondary tier scans public registries every 60 minutes, tertiary

tier checks community endpoints every 240 minutes. Each poll produces a set of `DiscoveredModel` records containing provider name, model ID, endpoint URL, detected capabilities, and cost metadata. These records flow into the verification pipeline, which executes the eight-step validation sequence for each new or changed model. Successfully verified models receive a `VerificationResult` containing per-step pass/fail status, latency measurements, capability flags, and rate limit parameters.

Verified models then enter the scoring engine, which computes the five component scores and the weighted composite. The scoring engine reads historical performance data from the SQLite persistence store and writes updated scores back after each computation cycle. The scored and ranked model list is then consumed by the score adapter, which translates it into the `ProviderPreference` format that HelixTranslate’s existing provider factory understands. This adapter runs on a 5-minute refresh interval by default, ensuring that HelixTranslate’s view of available providers tracks LLMsVerifier’s current rankings without imposing synchronous coupling.

On the HelixTranslate side, the adapted provider preferences feed into a new verification-aware provider factory located at `pkg/translator/llm/verified_factory.go`. This factory implements the same `ProviderFactory` interface as the existing nine manual providers — `Create(config ProviderConfig) (Provider, error)` — but its implementation queries the LLMsVerifier adapter rather than instantiating hard-coded provider structs. The existing nine providers are not deleted; they are reimplemented as thin wrappers that delegate to the verified factory when LLMsVerifier is enabled, with a feature flag (`--use-llms-verifier`) controlling the routing. This preserves backward compatibility during the transition period and enables rollback without code changes.

TABLE 2: Before/After Architecture Comparison

Dimension	Before (Current State)	After (Target State)
Provider Count	9 manually configured providers (OpenAI, Anthropic, Vertex, Azure OpenAI, Cohere, Mistral, Together AI, Ollama, Custom)	30+ automatically discovered providers via 3-tier discovery, with the set growing as new providers enter the market
Validation Depth	No validation; any configured provider is assumed reachable and functional	8-step verification pipeline: API reachability, authentication, model existence, response format, latency measurement, capability detection, rate limit compliance, error handling validation
Scoring System	None; provider selection is either random, round-robin, or hard-coded priority	5-component weighted scoring: latency, throughput, cost efficiency, capability breadth, reliability history, producing a composite rank
Configuration Model	Per-provider API keys, endpoints, and model IDs in <code>configs/translators.yaml</code> ; manual edits required for any change	Single <code>configs/verifier.yaml</code> with provider credentials and scoring weights; all model catalogs auto-discovered and kept current

Table 2 – continued

Dimension	Before (Current State)	After (Target State)
Failure Handling	Static fallback to secondary provider with manual configuration	Automatic circuit breaker integration; health monitor removes degraded models; fallback chains adapt based on real-time scores
New Provider Onboarding	Development task: implement provider interface, add factory registration, update config schema, add tests, deploy	Operational task: add API key to <code>verifier.yaml</code> ; discovery and verification happen automatically within the next poll cycle
Model Quality Assurance	Reactive: user complaints drive investigation and provider removal	Proactive: scoring engine continuously evaluates; gating threshold prevents low-scoring models from serving requests
Multi-Model Consensus	Not available	AI Debate Team: top-3 ranked models translate independently; divergence detection triggers confidence warnings and automatic re-translation

1.4 Implementation Phase Overview

The integration is structured as eight sequential phases spanning sixteen weeks. Phase boundaries are defined by gating criteria, not calendar dates — a phase does not advance until its gating criteria are demonstrably met. This ensures that each layer of the integration is solid before the next layer is built upon it. The sixteen-week timeline assumes two backend engineers with prior Go experience, one DevOps engineer for deployment pipeline configuration, and part-time frontend capacity for the enterprise UX phase.

TABLE 3: Master Phase Timeline

Phase	Weeks	Key Deliverables	Dependencies	Gating Criteria
1. Foundation	1–2	<code>go.mod</code> updates with LLMsVerifier module requirement; <code>configs/verifier.yaml</code> schema definition; <code>internal/verifier/</code> project structure; CODEOWNERS and governance documentation; CI pipeline extension for verification tests	LLMsVerifier v1.x release tag available; Go 1.25.3 toolchain installed	Module compiles without errors; <code>go vet</code> passes; config schema validates against all known provider types; directory structure follows HelixTranslate conventions

Table 3 – continued

Phase	Weeks	Key Deliverables	Dependencies	Gating Criteria
2. Verification Engine	3–4	8-step verification pipeline implementation in <code>internal/verifier/pipeline.go</code> ; LLMsVerifier client wrapper in <code>internal/verifier/client.go</code> ; health check integration with existing circuit breaker; provider-specific test vectors for all 9 current providers	Phase 1 completion; API keys for at least 5 providers available in CI secrets	All 8 verification steps execute independently with deterministic pass/fail; each step produces structured output; pipeline completes within 30 seconds per model in CI; no data races detected by <code>go test -race</code>
3. Scoring Engine	5–6	5-component scoring engine in <code>internal/verifier/scoring/engine.go</code> ; score adapter port in <code>internal/services/llmsverifier_score_adapter.go</code> ; SQLite persistence layer in <code>internal/verifier/persistence/store.go</code> ; scoring weight configuration in <code>verifier.yaml</code> ; score calculation verification harness	Phase 2 completion; historical cost data available for pricing normalization	Score calculations produce deterministic output for identical inputs; weight adjustments propagate correctly; persistence round-trip verified; adapter output matches HelixTranslate <code>ProviderPreference</code> schema

Table 3 – continued

Phase	Weeks	Key Deliverables	Dependencies	Gating Criteria
4. Discovery & Registry	7–8	3-tier discovery service in <code>internal/verifier/discovery/service.go</code> ; in-memory model registry with copy-on-read in <code>internal/verifier/registry/adapter.go</code> ; gating threshold enforcement; background poll scheduler; rate limit bucket integration	Phase 3 completion; network access to provider APIs from staging environment	Models discovered from all three tiers within configured intervals; registry serves ranked lookups in <1ms; gating threshold correctly filters sub-threshold models; rate limiter prevents API quota exhaustion
5. Runtime Integration	9–10	Verification-aware provider factory in <code>pkg/translator/llm/verified_factory.go</code> ; selection engine with score-based weighting; fallback chain construction from ranked list; feature flag <code>--use-llms-verifier</code> ; backward compatibility layer for existing 9 providers	Phase 4 completion; HelixTranslate translation test suite passes on main branch	End-to-end translation requests complete successfully through LLMsVerifier-routed providers; fallback chains activate correctly on simulated failures; feature flag toggle switches between old and new paths without restart; p99 latency regression <5% compared to baseline

Table 3 – continued

Phase	Weeks	Key Deliverables	Dependencies	Gating Criteria
6. Enterprise UX	11–12	Model selection UI showing verification status, scores, and ranks; real-time provider health dashboard; batch translation interface with model preference pinning; admin panel for scoring weight adjustment; user-facing model quality indicators	Phase 5 completion; frontend component library updated; API contracts stable	UX designs reviewed and approved by product team; model status indicators update within 30 seconds of health changes; admin weight adjustments apply without restart; accessibility audit passes WCAG 2.1 AA
7. Testing & QA	13–14	Unit test suite achieving 100% coverage of <code>internal/verifier/</code> ; integration test suite covering all 8 verification steps against live provider sandboxes; chaos engineering challenges (provider failure injection, network partition simulation, rate limit exhaustion); performance benchmark suite establishing latency and throughput baselines	Phase 6 completion; staging environment mirrors production topology	All unit tests pass with <code>-race</code> flag; integration tests pass against 5+ live providers; all chaos challenges resolve within 60 seconds; benchmarks show no p99 latency regression >5% and no throughput regression >10%

Table 3 – continued

Phase	Weeks	Key Deliverables	Dependencies	Gating Criteria
8. Documentation & Deploy	15–16	Complete API reference for all <code>internal/verifier/</code> packages; deployment runbook with rollback procedures; migration guide for existing provider configurations; operational playbooks for incident response; architecture decision records (ADRs) for all design choices; final security review and sign-off	Phase 7 completion; production deployment pipeline configured	All documentation reviewed and signed off by tech lead and product owner; deployment to staging completes via automated pipeline in <10 minutes; rollback procedure tested and verified; security review identifies no high or critical findings

Resource Requirements. The integration requires API keys for a minimum of 25 providers to exercise the full discovery and verification pipeline. The core set includes: OpenAI (GPT-4o, GPT-4o-mini, GPT-3.5-turbo), Anthropic (Claude 3.5 Sonnet, Claude 3 Haiku, Claude 3 Opus), Google Vertex AI (Gemini 1.5 Pro, Gemini 1.5 Flash), Azure OpenAI (deployed via organization’s Azure subscription), Cohere (Command R, Command R Plus), Mistral (Large, Medium, Small), Together AI (Llama 3.1, Mixtral), AI21 Labs (Jamba), Perplexity, Groq, Fireworks AI, Replicate, Ollama (local endpoint), DeepSeek, Moonshot AI, OpenRouter, and Anyscale. For the testing phase, sandbox or low-quota keys are sufficient — the verification pipeline sends only minimal test prompts, and the discovery tier primarily reads model catalog endpoints rather than generating completions.

Go version alignment is a hard dependency. Both HelixTranslate and LLMsVerifier must compile with Go 1.25.3 to ensure compatible standard library behavior, particularly around the `slices` and `maps` generic functions, `log/slog` structured logging, and `runtime.Pinner` for CGO interactions if local model inference is enabled. The `go.mod` file changes reflect this alignment.

Code Block 1: Required `go.mod` Changes

```
// go.mod — additions for LLMsVerifier integration
require (
    // ... existing requirements ...

    // LLMsVerifier core library
    github.com/helixagent/llmsverifier v1.4.2

    // LLMsVerifier indirect dependencies (auto-resolved by go mod tidy)
    // github.com/helixagent/llmsverifier/scoring v1.4.2
    // github.com/helixagent/llmsverifier/discovery v1.4.2
```

```

    // github.com/helixagent/llmsverifier/verification v1.4.2
)

// Replace directives for local development and staging builds.
// Remove or comment out for production builds that consume published tags.
replace github.com/helixagent/llmsverifier => ../llmsverifier

// Verify Go toolchain alignment.
go 1.25.3

toolchain go1.25.3

```

The `replace` directive supports local development workflows where engineers have both repositories checked out side-by-side. In CI and production builds, the directive is removed and the published v1.4.2 tag is consumed from the module proxy. The `go 1.25.3` directive enforces the minimum Go version; builds attempted with older toolchains fail with a clear version mismatch error.

Code Block 2: `configs/verifier.yaml` Top-Level Schema

```

# configs/verifier.yaml — LLMsVerifier configuration for HelixTranslate
# Schema version: 1.4.2
# Documentation: https://docs.helixagent.dev/llmsverifier/config-reference

verifier:
  # Server configuration for the embedded verifier HTTP API
  server:
    host: "0.0.0.0"
    port: 8081                # Dedicated port; separate from HelixTranslate's main port
    read_timeout: 30s
    write_timeout: 60s

  # Discovery tier configuration
  discovery:
    primary_poll_interval: 15m  # Provider API catalog queries
    secondary_poll_interval: 60m # Public registry scans (HuggingFace, Replicate)
    tertiary_poll_interval: 240m # Community endpoint probes
    max_concurrent_requests: 10 # Per-tier parallelism limit
    request_timeout: 30s
    backoff:
      initial: 1s
      max: 5m
      multiplier: 2.0

  # Verification pipeline configuration
  verification:
    enabled: true
    timeout_per_step: 30s
    warmup_requests: 10      # For latency measurement step
    max_retries_per_step: 3
    steps:
      - api_reachability

```

```

- authentication
- model_existence
- response_format
- latency_measurement
- capability_detection
- rate_limit_compliance
- error_handling

# Scoring engine configuration
scoring:
  weights:
    latency: 0.20
    throughput: 0.20
    cost_efficiency: 0.20
    capability: 0.25          # Slightly higher for translation workloads
    reliability: 0.15
  gating_threshold: 0.60      # Minimum composite score for model inclusion
  recalculation_interval: 5m
  history_window: 168h        # 7 days of health history for reliability score

# Provider API credentials
# Keys are loaded from environment variables or secrets manager;
# values here are references, not literal keys.
providers:
  openai:
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
    default_models: ["gpt-4o", "gpt-4o-mini", "gpt-3.5-turbo"]
  anthropic:
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com"
    default_models: ["claude-3-5-sonnet-20241022", "claude-3-haiku-20240307"]
  google_vertex:
    api_key: "${GOOGLE_VERTEX_API_KEY}"
    project_id: "${GOOGLE_PROJECT_ID}"
    location: "us-central1"
    default_models: ["gemini-1.5-pro", "gemini-1.5-flash"]
  azure_openai:
    api_key: "${AZURE_OPENAI_API_KEY}"
    endpoint: "${AZURE_OPENAI_ENDPOINT}"
    api_version: "2024-10-21"
  # ... additional providers follow same pattern ...

# Health monitoring and circuit breaker integration
health:
  check_interval: 30s
  failure_threshold: 3        # Consecutive failures before marking unhealthy
  recovery_threshold: 2       # Consecutive successes before marking healthy
  circuit_breaker:
    timeout: 60s
    half_open_max_calls: 3

```

```
# Persistence configuration
persistence:
  store_path: "./data/verifier/scores.db"
  max_size_mb: 512
  backup_interval: 24h
```

The configuration schema is versioned (1.4.2) and validated against an embedded JSON Schema at startup. The `providers` section uses environment variable references (`${VAR_NAME}`) rather than literal keys, ensuring that credentials never appear in committed configuration files. The `scoring.weights.capability` parameter is set to `0.25` — slightly elevated from the default `0.20` — to reflect the importance of multilingual capability breadth in translation workloads. The `gating_threshold` of `0.60` means that any model scoring below 60% composite is excluded from the provider pool, providing a quality floor that eliminates underperforming models before they reach users.

The sixteen-week timeline concludes with a production deployment that preserves full backward compatibility through the feature flag mechanism. Existing HelixTranslate installations continue to function with their current nine manual providers even after the LLMsVerifier code is deployed. Activation is a runtime toggle (`--use-llms-verifier`), not a code change or a migration script. This risk-minimizing approach ensures that the integration can be rolled out gradually — first to a staging environment for two weeks, then to a canary production segment for one week, and finally to full production traffic only after all operational metrics are validated against baseline.

Chapter 2: Phase 1 — Foundation and Repository Preparation

Phase 1 establishes the structural and dependency foundation upon which all subsequent integration layers depend. This phase is strictly preparatory: no runtime logic is introduced, no behavioral changes are made to existing translation pipelines, and no provider interactions are altered. Instead, the objective is to create the module references, directory hierarchies, configuration schemas, interface contracts, and governance documentation that enable the incremental integration of LLMsVerifier capabilities across Phases 2–6. The phase boundary is defined by five concrete deliverables: (1) a validated `go.mod` with all transitive dependencies resolved, (2) a complete `internal/verifier/` package tree with stub implementations, (3) an extended configuration schema supporting 33+ environment variables, (4) aligned interface contracts between the existing `llm.LLMClient` abstraction and the LLMsVerifier provider model, and (5) a `CONSTITUTION.md` document codifying 33 mandatory governance rules. All changes in this phase are additive—no existing files are modified except for `go.mod` and `internal/config/config.go`, and those modifications are strictly append-only. The estimated effort for this phase is 16–24 engineering hours, with zero deployment risk because no executable codepaths are activated until the `LLMSVERIFIER_ENABLED` flag is set to `true` in Phase 3.

2.1 Go Module Dependency Management

The first engineering task is to declare LLMsVerifier as a module dependency within the HelixTranslate Go workspace. Because LLMsVerifier is developed as a sibling repository that must be co-located during the integration period, the dependency is established using a combination of a `require` directive and a local `replace` directive. This pattern allows the Go toolchain to resolve the module at build time without requiring a published version to exist on a remote module proxy. The `require` directive pins the semantic

version at `v1.0.0`, establishing the interface contract version that subsequent phases assume. The `replace` directive maps this virtual version to a local filesystem path, `./LLMsVerifier`, which is expected to be a Git submodule or a manually cloned sibling directory at the repository root.

The `go.mod` modifications are as follows. First, the `require` block is extended to include the `LLMsVerifier` module:

```
require (
    digital.vasic.llmsverifier v1.0.0
    // ... existing deps preserved in full
)
```

The `replace` directive is appended at the end of the `go.mod` file, after all `require` blocks:

```
replace digital.vasic.llmsverifier => ./LLMsVerifier
```

These two declarations must appear together; the `replace` directive without a corresponding `require` entry will cause the Go compiler to ignore the replacement, while a `require` without a `replace` would attempt remote resolution and fail because the module is not yet published to the default module proxy. After adding these lines, execute `go mod tidy` to verify that the Go toolchain can resolve all transitive dependencies introduced by `LLMsVerifier`.

The addition of `LLMsVerifier` introduces a significant transitive dependency footprint. Prior to integration, `HelixTranslate` maintained direct dependencies on nine manually-configured LLM provider SDKs: `github.com/sashabaranov/go-openai` (OpenAI), `github.com/anthropics/anthropic-sdk-go` (Anthropic), `github.com/google/generative-ai-go` (Google Gemini), `cloud.google.com/go/vertexai` (Vertex AI), `github.com/aws/aws-sdk-go-v2/service/bedrockruntime` (Amazon Bedrock), `github.com/Azure/azure-sdk-for-go/sdk/ai/azopenai` (Azure OpenAI), `github.com/deepseek-ai/deepseek-go` (DeepSeek), `github.com/mistralai/client-go` (Mistral AI), and `github.com/cohere-ai/cohere-go/v2` (Cohere). These nine providers were individually imported, initialized, and configured in `internal/translator/providers/`. `LLMsVerifier` subsumes provider management internally, but the integration strategy preserves these existing imports during the transition period—Phase 4 performs the actual provider consolidation. Therefore, the provider SDKs remain in `go.mod` throughout Phase 1.

Table 2.1: Dependency Audit — Modules Before and After Phase 1 Integration

#	Module Path	Pre-Phase 1	Post-Phase 1	Rationale
1	<code>digital.vasic.llmsverifier</code>	Absent	<code>v1.0.0</code> (local replace)	Core verification engine —required for all scoring, discovery, and selection logic
2	<code>github.com/sashabaranov/go-openai</code>	Present	Present	OpenAI provider SDK; retained during transition, removed in Phase 4
3	<code>github.com/anthropics/anthropic-sdk-go</code>	Present	Present	Anthropic provider SDK; retained during transition

Table 4 – continued

#	Module Path	Pre-Phase 1	Post-Phase 1	Rationale
4	github.com/google/ generative-ai-go	Present	Present	Google Gemini provider SDK; retained during transition
5	cloud.google.com/go/ vertexai	Present	Present	Vertex AI provider SDK; retained during transition
6	github.com/aws/ aws-sdk-go-v2/ service/ bedrockruntime	Present	Present	Amazon Bedrock provider SDK; retained during transition
7	github.com/Azure/ azure-sdk-for-go/sdk/ ai/azopenai	Present	Present	Azure OpenAI provider SDK; retained during transition
8	github.com/ deepseek-ai/ deepseek-go	Present	Present	DeepSeek provider SDK; retained during transition
9	github.com/mistralai/ client-go	Present	Present	Mistral AI provider SDK; retained during transition
10	github.com/cohere-ai/ cohere-go/v2	Present	Present	Cohere provider SDK; retained during transition
11	github.com/mattn/ go-sqlite3	Absent	v1.14.24	LLMsVerifier persistent storage driver; CGO_ENABLED=1 required
12	github.com/gin-gonic/ gin	Absent	v1.10.0	LLMsVerifier HTTP API framework; indirect via verifier module
13	github.com/dgrijalva/ jwt-go	Absent	v3.2.0+incompatible	LLMsVerifier API authentication middleware
14	golang.org/x/crypto/ argon2	Absent	v0.36.0	LLMsVerifier credential hashing for admin API endpoints
15	github.com/stretchcr/ testify	Absent	v1.10.0	Test assertions for verifier unit tests; indirect via verifier module

Table 4 – continued

#	Module Path	Pre-Phase 1	Post-Phase 1	Rationale
16	github.com/ testcontainers/ testcontainers-go	Absent	v0.35.0	Integration test infrastructure for SQLite-backed verifier scenarios
17	golang.org/x/sync/ errgroup	Absent	v0.12.0	Concurrent goroutine coordination in multi-provider discovery
18	google.golang.org/ grpc	v1.76.0	v1.77.0	Version conflict resolution: LLMsVerifier requires v1.77.0 for new health-check streaming API

Table 2.1 enumerates 18 module entries, of which 13 represent additions or version changes relative to the pre-integration baseline. The most operationally significant change is the `google.golang.org/grpc` version conflict: HelixTranslate previously pinned `grpc` at `v1.76.0` for compatibility with a legacy telemetry exporter, while LLMsVerifier requires `v1.77.0` for its streaming health-check API. The resolution is to upgrade HelixTranslate’s `grpc` dependency to `v1.77.0` and update the telemetry exporter initialization code (a single-line change in `internal/telemetry/exporter.go`) to use the new `grpc.DialContext` signature. This change is made in Phase 1 because it is a prerequisite for `go mod tidy` to resolve successfully.

Go Version Alignment. At the start of Phase 1, HelixTranslate targets Go 1.25.2 and HelixAgent targets Go 1.26. LLMsVerifier requires Go 1.25.3 as its minimum version because it uses the `iter.Seq` generic iterator pattern introduced in that patch release for its model discovery streaming API. The decision is to standardize all three modules on Go 1.25.3:

```
go 1.25.3
```

This is the minimum version that satisfies all three modules. HelixAgent’s `go 1.26` directive is downgraded to `1.25.3`; this is safe because HelixAgent does not use any Go 1.26-specific language features. The `toolchain` directive is set to `go1.25.3` to ensure consistent compiler behavior across development environments and CI pipelines. After changing the version directive, verify with `go version` and `go vet` `./...` across all three module roots.

The `github.com/mattn/go-sqlite3` dependency (row 11) requires special attention because it uses CGO to bind the SQLite C library. This means that all build environments—including CI runners, Docker build stages, and developer workstations—must have `CGO_ENABLED=1` set. The existing HelixTranslate build used `CGO_ENABLED=0` for static binary production. Phase 1 introduces a dual-build strategy: development and test builds use `CGO_ENABLED=1` to support SQLite-backed verifier storage, while production release builds continue to use `CGO_ENABLED=0` until Phase 5 completes the migration to a non-CGO persistence backend. This is documented in `Makefile` and `Dockerfile` comments during Phase 1.

2.2 Project Structure Creation

With module dependencies declared, the next deliverable is the directory hierarchy for the `internal/verifier/` package and its subpackages. All directories are created as empty shells with `.gitkeep` files, with the exception of `configs/verifier.yaml` which is populated with a template configuration. The structure is designed to mirror the five functional domains of LLMsVerifier—scoring, discovery, verification, selection, and configuration—while remaining idiomatic to Go package organization conventions.

Create the following directories using `mkdir -p`:

- `internal/verifier/scoring/` — Weighted scoring engine and component calculators
- `internal/verifier/discovery/` — Three-tier model discovery service and registry
- `internal/verifier/verification/` — Verification pipeline and adapter layer
- `internal/verifier/selection/` — Model selection engine with fallback chain
- `internal/verifier/challenges/` — Executable challenge scripts for subsystem validation
- `internal/verifier/config/` — Verifier-specific configuration types and validation
- `internal/services/` — Top-level service composition (shared with existing HelixTranslate services)

Each subdirectory contains a `doc.go` file with a package-level comment explaining its responsibility. The root `internal/verifier/` directory holds six files that serve as the public API surface for the package: `client.go` (HTTP client for LLMsVerifier API communication), `health.go` (health check integration with the existing `/healthz` endpoint), `pipeline.go` (the 8-step verification pipeline coordinator), `config.go` (configuration struct definitions), `bridge.go` (interface bridge between HelixTranslate's `llm.LLMClient` and LLMsVerifier's provider model), and `errors.go` (verifier-specific error types). The `challenges/` directory is unique in that it contains shell scripts rather than Go source files—these are executable validation scenarios that engineers run manually or in CI to verify subsystem behavior.

The `configs/verifier.yaml` template is created at the repository root to provide a reference configuration that operators can copy and customize:

```
verifier:
  enabled: true
  api_url: http://localhost:8080
  db_path: ./data/verifier.db
  cache_ttl: 1h
  providers:
    - id: openai
      api_key: ${OPENAI_API_KEY}
      models: []
    - id: anthropic
      api_key: ${ANTHROPIC_API_KEY}
      models: []
  scoring:
    weights:
      response_speed: 0.20
      cost_effectiveness: 0.30
      model_efficiency: 0.25
      capability: 0.20
      recency: 0.05
  discovery:
    mode: auto
```

```
refresh_interval: 1h
```

This YAML template encodes the default configuration values that match the tagged struct defaults in `internal/config/config.go` (Section 2.3). The `providers` section demonstrates the expected structure for provider entries: each provider has an opaque string `id`, an `api_key` that is resolved from environment variables using `${VAR}` interpolation, and a `models` array that starts empty because the discovery service populates it automatically at runtime. The `scoring.weights` subsection defines the five-component weight vector that the composite scoring engine uses; these values sum to `1.0` and can be adjusted per-environment to prioritize different operational objectives. The `discovery.mode: auto` setting enables the three-tier discovery pipeline (registry scan → health validation → capability assessment), while `refresh_interval: 1h` controls how frequently the discovery service invalidates its cached model catalog.

The complete directory tree for the verifier package is as follows:

```
internal/verifier/
├─ client.go           # HTTP client for LLMsVerifier
├─ health.go          # Health check integration
├─ pipeline.go         # 8-step verification pipeline
├─ config.go          # Verifier configuration structs
├─ bridge.go          # Interface bridge between abstractions
├─ errors.go          # Verifier-specific error types
├─ scoring/
│   ├─ doc.go          # Package documentation
│   ├─ engine.go       # Scoring engine wrapper
│   ├─ components.go   # 5 component score calculators
│   ├─ composite.go    # Weighted score aggregator
│   └─ history.go      # Score persistence and time-series
├─ discovery/
│   ├─ doc.go          # Package documentation
│   ├─ service.go      # 3-tier discovery coordinator
│   ├─ registry.go     # Model registry (SSOT for available models)
│   ├─ gatekeeper.go   # Model availability gate
│   └─ sync.go         # Background synchronization worker
├─ selection/
│   ├─ doc.go          # Package documentation
│   ├─ engine.go       # Model selection logic
│   └─ fallback.go     # Fallback chain and retry logic
└─ challenges/
    ├─ score_adapter.sh # Challenge: adapter accuracy validation
    ├─ gatekeeping.sh   # Challenge: gatekeeper threshold enforcement
    ├─ discovery.sh     # Challenge: discovery resilience under failure
    └─ selection.sh     # Challenge: selection fairness across providers
```

The `client.go` file at the root of `internal/verifier/` implements an HTTP client that communicates with the LLMsVerifier service over its REST API. It is responsible for connection pooling, request signing (using the `LLMSVERIFIER_API_KEY` environment variable), response deserialization, and circuit-breaker logic. The `health.go` file registers a verifier health probe with the existing `internal/health` package, adding a `/healthz/verifier` endpoint that reports the connectivity status to the LLMsVerifier API and the SQLite database. The `pipeline.go` file defines the 8-step verification pipeline that is fully implemented in Phase 3; in Phase 1 it contains only the pipeline struct definition and a no-op `Run()` method that returns an

unimplemented error. The `bridge.go` file defines the `VerifiedProvider` interface (detailed in Section 2.4) and its adapter implementations.

The `challenges/` subdirectory deserves special mention. Unlike the Go source files in other directories, the four files here are executable Bash scripts that serve as subsystem-level validation scenarios. Each script performs a specific challenge: `score_adapter.sh` validates that the scoring engine correctly maps raw performance metrics to normalized component scores; `gatekeeping.sh` verifies that the gatekeeper correctly rejects unavailable models before they reach the selection engine; `discovery.sh` tests discovery resilience by simulating provider API failures and verifying graceful degradation; `selection.sh` confirms that the selection engine distributes load fairly across providers with equivalent scores. These scripts are executed manually during development and automatically in CI after the full integration is complete. They are not Go tests because they operate at the integration level and require external service dependencies.

2.3 Configuration Schema Design

HelixTranslate uses a unified configuration system in which all application settings are defined as Go structs with struct tags for JSON deserialization, environment variable binding, and default value specification. Phase 1 extends this system by adding a new `LLMsVerifierConfig` struct to `internal/config/config.go` and registering it within the existing `AppConfig` root struct.

The complete configuration struct definition is:

```
type LLMsVerifierConfig struct {
    Enabled    bool           `json:"enabled" env:"LLMSVERIFIER_ENABLED" default:"true"`
    APIURL     string         `json:"api_url" env:"LLMSVERIFIER_API_URL" default:"http://
    ↪ localhost:8080"`
    APIKey     string         `json:"api_key" env:"LLMSVERIFIER_API_KEY" required:"true"`
    DBPath     string         `json:"db_path" env:"LLMSVERIFIER_DB_PATH" default:"./data/verifier.db"`
    CacheTTL   time.Duration `json:"cache_ttl" env:"LLMSVERIFIER_CACHE_TTL" default:"1h"`
    Providers  []ProviderConfig `json:"providers"`
    Scoring    ScoringConfig  `json:"scoring"`
    Discovery  DiscoveryConfig `json:"discovery"`
}
```

This struct is embedded into the root `AppConfig` struct as a field named `Verifier` of type `*LLMsVerifierConfig`. Using a pointer type allows the configuration loader to distinguish between “verifier configuration explicitly disabled” (`Verifier: nil` after parsing when the `verifier` key is absent from YAML) and “verifier configuration present with default values” (a non-nil struct when the key is present). The three substructs—`ProviderConfig`, `ScoringConfig`, and `DiscoveryConfig`—are defined in the same file. `ProviderConfig` captures per-provider settings including API key, model allowlists, rate limits, and custom headers. `ScoringConfig` holds the five weight values and normalization parameters. `DiscoveryConfig` controls the discovery mode (`auto`, `manual`, or `hybrid`), refresh intervals, timeout values, and concurrency limits.

The configuration system supports three override layers in priority order: (1) environment variables, (2) YAML configuration files, (3) struct tag defaults. The `env` tags on the struct fields define the environment variable names that the loader inspects. The `required:"true"` tag on `APIKey` causes the application to exit with a fatal error during startup if the key is not provided via environment variable or configuration

file. This is the only required field; all other fields have sensible defaults that allow the verifier subsystem to start in a local development configuration without any explicit settings.

Table 2.2: Complete Environment Variable Registry for LLMsVerifier Subsystem

#	Environment Variable	Type	Default	Required	Description
1	LLMSVERIFIER_ENABLED	bool	true	No	Master enable/disable switch for the entire verifier subsystem
2	LLMSVERIFIER_API_URL	string	http://localhost:8080	No	Base URL of the LLMsVerifier API service
3	LLMSVERIFIER_API_KEY	string	—	Yes	Authentication key for LLMsVerifier API requests (JWT bearer token)
4	LLMSVERIFIER_DB_PATH	string	./data/verifier.db	No	Filesystem path to the SQLite database for score persistence
5	LLMSVERIFIER_CACHE_TTL	duration	1h	No	Time-to-live for cached model metadata and composite scores
6	LLMSVERIFIER_VERIFICATION_ENABLED	bool	true	No	Enable/disable the 8-step verification pipeline
7	LLMSVERIFIER_SCORING_ENABLED	bool	true	No	Enable/disable the weighted scoring engine
8	LLMSVERIFIER_DISCOVERY_ENABLED	bool	true	No	Enable/disable the 3-tier model discovery service
9	LLMSVERIFIER_MAX_CONCURRENT	int	10	No	Maximum concurrent verification operations per provider
10	LLMSVERIFIER_TIMEOUT	duration	30s	No	Global timeout for single verification requests
11	LLMSVERIFIER_WEIGHT_SPEED	float64	0.20	No	Scoring weight: response speed component
12	LLMSVERIFIER_WEIGHT_COST	float64	0.30	No	Scoring weight: cost-effectiveness component

Table 5 – continued

#	Environment Variable	Type	Default	Required	Description
13	LLMSVERIFIER_WEIGHT_EFFICIENCY	float64	0.25	No	Scoring weight: model efficiency component
14	LLMSVERIFIER_WEIGHT_CAPABILITY	float64	0.20	No	Scoring weight: capability component
15	LLMSVERIFIER_WEIGHT_RECENCY	float64	0.05	No	Scoring weight: recency/staleness component
16	LLMSVERIFIER_DISCOVERY_MODE	string	auto	No	Discovery mode: auto, manual, or hybrid
17	LLMSVERIFIER_DISCOVERY_REFRESH_INTERVAL	duration	1h	No	Interval between automatic discovery refreshes
18	LLMSVERIFIER_DISCOVERY_TIMEOUT	duration	10s	No	Per-provider discovery timeout
19	LLMSVERIFIER_GATEKEEPER_STRICT	bool	false	No	When true, reject models with any failed health check; when false, allow degraded models with reduced scores
20	LLMSVERIFIER_FALLBACK_CHAIN	string	score,tier,cost	No	Comma-ordered fallback strategy for model selection
21	LLMSVERIFIER_LOG_LEVEL	string	info	No	Log level for verifier subsystem: debug, info, warn, error
22	OPENAI_API_KEY	string	—	No*	OpenAI provider API key (required if OpenAI provider is enabled)
23	ANTHROPIC_API_KEY	string	—	No*	Anthropic provider API key
24	GEMINI_API_KEY	string	—	No*	Google Gemini provider API key
25	VERTEX_AI_PROJECT_ID	string	—	No*	Google Cloud project ID for Vertex AI
26	VERTEX_AI_LOCATION	string	us-central1	No*	Vertex AI region
27	AWS_REGION	string	—	No*	AWS region for Bedrock access
28	AWS_ACCESS_KEY_ID	string	—	No*	AWS credentials for Bedrock

Table 5 – continued

#	Environment Variable	Type	Default	Required	Description
29	AWS_SECRET_ACCESS_KEY	string	—	No*	AWS credentials for Bedrock
30	AZURE_OPENAI_ENDPOINT	string	—	No*	Azure OpenAI resource endpoint
31	AZURE_OPENAI_API_KEY	string	—	No*	Azure OpenAI API key
32	DEEPSEEK_API_KEY	string	—	No*	DeepSeek provider API key
33	MISTRAL_API_KEY	string	—	No*	Mistral AI provider API key
34	COHERE_API_KEY	string	—	No*	Cohere provider API key
35	LLMSVERIFIER_METRICS_ENABLED	bool	true	No	Enable Prometheus metrics export for verifier operations
36	LLMSVERIFIER_METRICS_PORT	int	9090	No	Prometheus metrics scrape endpoint port

Table 2.2 documents 36 environment variables (exceeding the 33+ requirement). Variables 1–21 control the verifier subsystem itself; variables 22–34 are the existing provider API keys that were already present in HelixTranslate’s configuration but are now explicitly enumerated because the verifier’s discovery service consumes them during the registry population phase. Variables 35–36 are new additions for observability integration. The five scoring weights (variables 11–15) are validated at startup to ensure they sum to exactly 1.0; if they do not, the application logs a warning and normalizes them proportionally.

The `LLMSVERIFIER_API_KEY` (variable 3) is the only field marked `required:"true"` at the verifier configuration level. This key is a JWT bearer token generated by the LLMsVerifier administrator panel during initial deployment. It is used by the `client.go` HTTP client to sign all outgoing requests with an `Authorization: Bearer <token>` header. The token contains claims for rate-limiting identity and access scope; the verifier API validates the signature against a pre-shared secret. Rotation is supported via a `LLMSVERIFIER_API_KEY_PREVIOUS` environment variable that accepts the previous token for a grace period of 24 hours, preventing authentication failures during key rotation events.

Weight configuration deserves careful treatment because the five scoring components are interdependent. The default weights of `0.20/0.30/0.25/0.20/0.05` prioritize cost-effectiveness (30%) in a production translation workload where API spend is the dominant operational concern, followed by model efficiency (25%) which captures throughput per dollar. Response speed and capability each receive 20%, reflecting equal importance between latency and quality. Recency receives the smallest weight (5%) as a tiebreaker rather than a primary discriminator. Operators can adjust these weights per-environment: a development environment might set `LLMSVERIFIER_WEIGHT_SPEED=0.40` to prioritize fast iteration, while a high-stakes production deployment might set `LLMSVERIFIER_WEIGHT_CAPABILITY=0.40` to prioritize translation quality above all else.

2.4 Interface Alignment

HelixTranslate's existing LLM abstraction is defined in `pkg/translator/llm/provider.go` as the `LLMClient` interface. `LLMsVerifier` defines a similar but not identical abstraction in its own package. Phase 1 must align these two interfaces so that HelixTranslate's existing provider implementations can be wrapped by the verifier's scoring and selection layers without requiring changes to the caller code. This alignment is achieved through a bridge pattern implemented in `internal/verifier/bridge.go`.

The existing `LLMClient` interface in `pkg/translator/llm/provider.go` is:

```
type LLMClient interface {
    Translate(ctx context.Context, text string, sourceLang, targetLang string) (string, error)
    GetModelInfo(ctx context.Context) (ModelInfo, error)
    HealthCheck(ctx context.Context) error
    Close() error
}
```

`LLMsVerifier`'s corresponding provider interface (defined in the `LLMsVerifier` module) is:

```
type Provider interface {
    Completion(ctx context.Context, prompt string, params CompletionParams) (CompletionResponse, error)
    GetModelMetadata(ctx context.Context) (ModelMetadata, error)
    Ping(ctx context.Context) error
    Close() error
}
```

Table 2.3: Method Mapping Between HelixTranslate LLMClient and LLMsVerifier Provider

HelixTranslate Method	LLMsVerifier Method	Semantic Equivalence	Adapter Logic
<code>Translate(ctx, text, srcLang, tgtLang)</code>	<code>Completion(ctx, prompt, params)</code>	Core inference: both execute a prompt against a language model	Convert translation request to prompt template; map source/target languages to <code>params.Languages</code> ; extract text from <code>CompletionResponse.Text</code>
<code>GetModelInfo(ctx)</code>	<code>GetModelMetadata(ctx)</code>	Metadata retrieval: both return model identification and capability information	Map <code>ModelInfo</code> (name, version, maxTokens) to <code>ModelMetadata</code> (id, version, contextWindow, capabilities); capability bitmap translation

Table 6 – continued

HelixTranslate Method	LLMsVerifier Method	Semantic Equivalence	Adapter Logic
HealthCheck(ctx)	Ping(ctx)	Liveness probe: both verify the provider endpoint is reachable and responsive	Direct pass-through with error translation; Ping() returns nil → HealthCheck() returns nil; Ping() returns error → wrap in verifier.ErrProviderUnavailable
Close()	Close()	Resource cleanup: both release connections, close HTTP clients, and free handles	Direct delegation; no transformation needed

Table 2.3 reveals that the two interfaces share identical semantic intent but differ in naming conventions and parameter structures. The `Close()` method is a direct match requiring no adapter logic. The health-check methods differ only in name (`HealthCheck` vs. `Ping`) and can be bridged with a simple wrapper. The metadata methods differ in return types (`ModelInfo` vs. `ModelMetadata`) but both convey the same essential information: model identity, version, and operational parameters. The translation and completion methods differ most significantly: `Translate` takes structured language parameters while `Completion` takes a raw prompt string and parameter bag. The adapter must perform prompt templating—converting the `(text, sourceLang, targetLang)` triple into a translation prompt such as "Translate the following text from {sourceLang} to {targetLang}: \n\n{text}"—and extract the translated text from the completion response.

The bridge interface that unifies both abstractions is defined in `internal/verifier/bridge.go`:

```
type VerifiedProvider interface {
    llm.LLMClient
    GetModelID() string
    GetVerificationStatus() VerificationStatus
    GetCompositeScore() *CompositeScore
    GetTier() ModelTier
    GetCapabilities() CapabilityBitmap
}
```

The `VerifiedProvider` interface embeds `llm.LLMClient`, meaning it inherits all four methods (`Translate`, `GetModelInfo`, `HealthCheck`, `Close`) and is substitutable wherever an `LLMClient` is expected. This is a critical design decision: because `VerifiedProvider` is a subtype of `LLMClient`, the existing `Translator` struct in `internal/translator/engine.go` can accept `VerifiedProvider` instances without modification. The additional methods—`GetModelID`, `GetVerificationStatus`, `GetCompositeScore`, `GetTier`, and `GetCapabilities`—are provided by the verifier subsystem to enable scoring-aware selection while preserving backward compatibility.

The `VerificationStatus` type is an enumerated string with values `StatusUnknown`, `StatusPending`, `StatusVerified`, `StatusDegraded`, and `StatusFailed`. A provider in `StatusVerified` has passed all 8 verifi-

cation steps (Phase 3) within the last cache TTL period. `StatusDegraded` indicates that the provider is functional but has non-critical failures in some verification steps (e.g., elevated latency within acceptable bounds). `StatusFailed` means the provider has critical failures and should not receive traffic. The gatekeeper (Section 2.2, `gatekeeper.go`) uses this status to filter providers before they reach the selection engine.

The `CompositeScore` struct aggregates the five component scores into a single normalized value between 0.0 and 1.0, along with the individual component breakdowns for observability. The `ModelTier` type classifies models into `TierFree`, `TierStandard`, `TierPremium`, and `TierEnterprise` based on their pricing tier and capability profile. The `CapabilityBitmap` is a 64-bit bitmask where each bit represents a specific capability (e.g., multilingual translation, code generation, long-context processing, tool use, vision understanding); this enables fast capability matching during model selection.

The bridge implementation includes two concrete adapter structs: `VerifiedProviderAdapter` wraps an existing `llm.LLMClient` implementation (such as `OpenAIProvider` or `AnthropicProvider`) and adds the verifier-specific methods by delegating to the `LLMsVerifier` API, and `NativeVerifiedProvider` wraps an `LLMsVerifier Provider` directly and implements the `llm.LLMClient` interface. Both adapters implement the full `VerifiedProvider` interface, allowing the selection engine to work with either native `LLMsVerifier` providers or legacy `HelixTranslate` providers that have been enrolled in the verification system.

2.5 Constitution and Governance Documentation

The final deliverable of Phase 1 is a governance document that codifies the rules, invariants, and constraints governing the `LLMsVerifier` integration. This document, `internal/verifier/CONSTITUTION.md`, serves as the authoritative reference for all engineering decisions affecting the verifier subsystem. It is not a design document—it is a normative specification that defines what the system must, must not, and should do. Each rule is assigned a unique identifier for traceability in code comments, commit messages, and issue tracking.

The constitution contains 33 mandatory rules organized into six categories. The document begins with a preamble establishing its authority and scope, followed by the rule definitions, followed by an appendix describing the amendment process.

Table 2.4: Constitution Rule Mapping by Category

Category	Rule IDs	Count	Coverage Area
Scoring	S01-S08	8	Weight normalization, score calculation, component aggregation, score persistence, caching, recency decay, outlier rejection, score transparency
Verification	V01-V08	8	8-step pipeline execution, adapter accuracy, timeout handling, error classification, retry policy, health check frequency, result caching, verification audit trail

Table 7 – continued

Category	Rule IDs	Count	Coverage Area
Discovery	D01–D05	5	Registry consistency, provider enumeration, model metadata validation, discovery refresh scheduling, graceful degradation on provider failure
Selection	L01–L05	5	Score-based ranking, fallback chain execution, tier-based filtering, capability matching, load distribution fairness
Testing	T01–T04	4	Unit test coverage thresholds, integration test requirements, challenge script validation, performance regression detection
Operations	O01–O03	3	Deployment rollback criteria, monitoring and alerting, incident response procedures

Table 2.4 maps the 33 rules across six functional categories. Each rule is written as a normative statement using RFC 2119 keywords (MUST, MUST NOT, SHOULD, MAY). The scoring rules (S01–S08) govern the five-component scoring engine: S01 mandates that weights always sum to 1.0; S02 requires score normalization to the $[0, 1]$ interval; S03 defines the recency decay function (exponential half-life of 24 hours); S04 mandates outlier rejection using the interquartile range method; S05 requires score persistence to SQLite within 5 seconds of calculation; S06 mandates a 1-hour TTL for cached scores; S07 requires score transparency (all component scores must be queryable via the API); S08 mandates that score changes greater than 0.1 trigger an audit log entry.

The verification rules (V01–V08) govern the 8-step pipeline: V01 mandates that all 8 steps execute in sequence and that partial results are discarded if any step fails critically; V02 requires that adapter accuracy (the fidelity of the bridge between `LLMClient` and `Provider`) be validated on every startup with a known-good translation pair; V03 defines a 30-second global timeout per verification request with per-step budget allocation; V04 mandates error classification into `Critical`, `Degraded`, and `Transient` categories; V05 requires exponential backoff with jitter for retry attempts (base 100ms, max 30s); V06 mandates health checks every 60 seconds for active providers; V07 requires verification results to be cached with the same TTL as scores; V08 mandates an audit trail recording every verification event with timestamp, provider ID, step results, and outcome.

The discovery rules (D01–D05) govern the 3-tier discovery service: D01 mandates that the registry be the single source of truth (SSOT) for all model availability data; D02 requires enumeration of all configured providers on startup and at every refresh interval; D03 mandates that discovered model metadata be validated against a schema (ID format, version string, capability bitmap range); D04 requires scheduled refresh with jitter ($\pm 10\%$ of the interval) to prevent thundering herd; D05 mandates graceful degradation—if a provider fails discovery, its previously cached models remain available with a `stale` flag until the next successful discovery or explicit timeout.

The selection rules (L01–L05) govern model selection: L01 mandates score-based ranking as the primary

selection criterion; L02 requires the fallback chain (`score`, `tier`, `cost` by default) to execute in order when the primary criterion yields ties; L03 mandates tier-based filtering when the request specifies a minimum tier; L04 requires capability matching—models without the requested capabilities must be excluded before scoring; L05 mandates load distribution fairness: among providers with equivalent composite scores (difference < 0.01), selection must distribute requests with uniform probability.

The testing rules (T01–T04) establish quality gates: T01 mandates 85% unit test coverage for all `internal/verifier/` packages; T02 requires integration tests for each of the four challenge scripts; T03 mandates that all challenge scripts pass in CI before any pull request affecting the verifier subsystem can be merged; T04 requires performance regression detection—any change that increases the p99 verification latency by more than 10% must be flagged in CI and reviewed.

The operations rules (O01–O03) cover deployment and runtime: O01 mandates automatic rollback if the verifier health check fails for more than 2 minutes post-deployment; O02 requires Prometheus metrics for all verifier operations with defined alert thresholds; O03 mandates an incident response runbook for verifier-related outages, including a decision tree for disabling the verifier via `LLMSVERIFIER_ENABLED=false` as an emergency mitigation.

Two additional documents are updated during Phase 1. `CLAUDE.md` at the repository root receives a new section describing the verifier subsystem architecture and its interaction points with the translation engine. `AGENTS.md` (the agent configuration file for AI-assisted development) receives updated context about the `internal/verifier/` package structure, the location of key interfaces, and the rule identifiers from the constitution that should be referenced when generating code. These updates ensure that AI-assisted development tools have accurate context for the new subsystem.

The constitution is a living document governed by an amendment process: any change to a rule requires (a) a pull request with rationale, (b) review by at least two subsystem owners, (c) update of the rule’s version timestamp, and (d) propagation of the change to all code comments referencing the rule ID. Rules marked with `[IMMUTABLE]` in the constitution—specifically S01 (weight normalization), D01 (registry SSOT), and L05 (fairness)—cannot be amended without a formal architectural review and sign-off from the technical lead.

Phase 1 concludes when all five deliverables are in place: the `go.mod` resolves without errors and all 18 dependency entries are validated; the `internal/verifier/` directory tree is created with stub files and documented package structure; the `LLMsVerifierConfig` struct is defined and the configuration loader recognizes all 36 environment variables; the `VerifiedProvider` bridge interface is defined with both adapter struct types; and the `CONSTITUTION.md` document is created with all 33 rules mapped and traceable. At this point, the codebase compiles successfully with `go build ./...`, all existing tests pass, and no verifier codepaths are active in production. The foundation is ready for Phase 2: Integration and Verification Pipeline Development.

3. Phase 2: Core Verification Engine Integration

Phase 2 forms the operational heart of the `LLMsVerifier` integration by establishing the full verification engine within the `HelixTranslate` service boundary. Where Phase 1 concentrated on configuration scaffolding and service discovery, Phase 2 moves into implementation territory: constructing the Go-based verifier client, defining the eight-step pipeline architecture, implementing each verification stage from existence checks through final validation, and wiring the entire apparatus into the existing `HelixTrans-`

late startup sequence. The deliverables of this phase are concrete, production-hardened Go source files that reside in `internal/verifier/` and are imported by `cmd/helix-translate/main.go`. Every component introduced in this chapter is designed with observability, fault tolerance, and horizontal scalability as first-class concerns. The design assumes that the verifier may reside on a separate host, may experience transient failures under load, and must not block the translation hot path for longer than strictly necessary.

The architectural philosophy guiding this phase is one of defensive composition: each verification step is an isolated unit that exposes a common `Step` interface, participates in a group-based execution strategy (parallel versus sequential), carries its own timeout budget, and reports success or failure through a uniform `StepResult` structure. Steps that are marked as critical can halt the entire pipeline and prevent a model from entering the active pool, while non-critical steps merely contribute to a composite score. This distinction allows HelixTranslate to adopt a graduated gating strategy—models must pass all critical checks but can be admitted with warnings on secondary dimensions such as advanced feature availability or cost competitiveness. The pipeline is orchestrated by a `VerificationPipeline` struct that uses Go's `errgroup` package for concurrent step execution within Group 1, followed by a straightforward sequential loop for Group 2. Retry logic with exponential backoff and circuit-breaker awareness protects against transient network failures and thundering-herd scenarios when multiple verification cycles are triggered simultaneously.

Throughout this chapter, references to file paths assume the standard Go project layout: domain logic lives under `internal/verifier/`, configuration structures are defined in `internal/config/`, shared cache abstractions reside in `internal/cache/`, and the service entry point is `cmd/helix-translate/main.go`. All timeout values, retry parameters, and threshold constants presented here are defaults that can be overridden through the `LLMsVerifierConfig` structure documented in Phase 1. The implementation also respects context cancellation, ensuring that a shutdown signal propagates cleanly through long-running verification steps and causes the pipeline to return immediately rather than hanging on an outbound HTTP call.

3.1 Verifier Client Initialization

The first concrete task in Phase 2 is to create the `VerifierClient` struct and its constructor, which encapsulates all communication between HelixTranslate and the LLMsVerifier backend service. This client is intentionally thin: it is responsible for connection management, request signing via JWT bearer tokens, and response deserialization, but it delegates all business logic to the pipeline layer above it. Keeping the client thin simplifies unit testing—mocking the verifier backend requires only a small HTTP test server—and reduces the blast radius of any API contract changes on the verifier side.

The client is defined in `internal/verifier/client.go`. It holds four fields: an `*http.Client` instance configured with a timeout derived from the global configuration, a `baseURL` string representing the verifier service endpoint, an `apiKey` used for Bearer-token authentication in every outbound request, and a `timeout` field cached locally for diagnostic logging. The constructor `NewVerifierClient` performs strict validation: both `APIURL` and `APIKey` are required fields, and the function returns an explicit error rather than silently substituting defaults for values that represent security-sensitive or connectivity-critical configuration. The timeout is sourced from `cfg.CacheTTL` as a sensible default, but falls back to thirty seconds if that value is zero, ensuring that the client is usable even in minimally configured deployments.

CODE BLOCK 1: `internal/verifier/client.go`

```

type VerifierClient struct {
    httpClient *http.Client
    baseURL    string
    apiKey     string
    timeout    time.Duration
}

func NewVerifierClient(cfg *config.LLMsVerifierConfig) (*VerifierClient, error) {
    if cfg.APIURL == "" { return nil, errors.New("APIURL is required") }
    if cfg.APIKey == "" { return nil, errors.New("APIKey is required") }
    timeout := cfg.CacheTTL
    if timeout == 0 { timeout = 30 * time.Second }
    return &VerifierClient{
        httpClient: &http.Client{Timeout: timeout},
        baseURL:    strings.TrimRight(cfg.APIURL, "/"),
        apiKey:     cfg.APIKey,
        timeout:    timeout,
    }, nil
}

func (c *VerifierClient) Ping(ctx context.Context) error {
    req, err := http.NewRequestWithContext(ctx, "GET", c.baseURL+"/health", nil)
    if err != nil { return err }
    req.Header.Set("Authorization", "Bearer "+c.apiKey)
    resp, err := c.httpClient.Do(req)
    if err != nil { return err }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK { return fmt.Errorf("verifier unhealthy: %d", resp.StatusCode) }
    return nil
}

```

The Ping method is the simplest possible health-check: it sends a GET request to the verifier's /health endpoint, injects the Bearer token into the Authorization header, and validates that the response status is exactly 200 OK. This method is intentionally lightweight so that it can be called frequently during startup and during periodic liveness checks without placing meaningful load on either service. The method respects context cancellation, meaning that if the parent context expires or is cancelled, the outbound HTTP request will abort immediately rather than continuing to occupy a connection slot.

During service startup in `cmd/helix-translate/main.go`, the client initialization must occur after configuration parsing but before the translation service begins accepting traffic. The recommended startup sequence is: parse flags and environment variables into `config.LLMsVerifierConfig`; invoke `NewVerifierClient(cfg)` and check the returned error, logging a `Fatal` if either required field is missing; then call `WaitForHealthy` (defined below) with a context bounded by a configurable maximum wait duration, typically thirty to sixty seconds. Only after the verifier reports a healthy status does the service proceed to initialize its model pool and start the HTTP or gRPC server. This ordering guarantees that no translation request can arrive before the verification subsystem is operational.

Beyond basic connectivity, the client struct serves as the single point of configuration for HTTP connection pooling. The `http.Client` instantiated by the constructor uses Go's default transport, which already supports persistent connections and connection reuse. For high-throughput deployments, operators may

wish to customize the transport with `MaxIdleConns`, `MaxIdleConnsPerHost`, and `IdleConnTimeout` values. Because the transport is created inside the constructor, this customization can be exposed through additional fields in `LLMsVerifierConfig` without changing the call sites of `NewVerifierClient`. The key design invariant is that every outbound request to the verifier service is made through this single client instance, ensuring consistent timeout behavior and authentication across all verification operations.

CODE BLOCK 2: `internal/verifier/health.go`

```
func (c *VerifierClient) WaitForHealthy(ctx context.Context, maxWait time.Duration) error {
    deadline := time.Now().Add(maxWait)
    for time.Now().Before(deadline) {
        if err := c.Ping(ctx); err == nil { return nil }
        time.Sleep(time.Second)
    }
    return fmt.Errorf("verifier not healthy after %v", maxWait)
}
```

The `WaitForHealthy` helper complements `Ping` by providing a blocking poll loop with a hard deadline. It is intended for use exclusively during startup, when `HelixTranslate` must delay its own readiness until the verifier is available. The loop polls once per second, a cadence chosen to be frequent enough to minimize startup latency without generating excessive log noise or network traffic. The deadline is computed from `maxWait`, which should be provided from configuration (e.g., `cfg.StartupMaxWait`) so that operators can tune it based on their deployment topology. If the verifier does not become healthy within the allotted window, the function returns a descriptive error that should be treated as fatal by the caller: attempting to operate a translation service without a functioning verification backend violates the safety guarantees of the integrated system.

3.2 The 8-Step Verification Pipeline

With the client initialized, the next deliverable is the pipeline orchestrator that sequences the eight verification steps. The pipeline is the central abstraction of Phase 2: it receives a model identifier, coordinates the execution of all verification handlers, aggregates their individual results into a composite report, and makes the final admission decision. The design splits the eight steps into two execution groups based on dependency constraints and latency characteristics.

Group 1 contains the first four steps—Existence, Responsiveness, Feature Detection, and Code Visibility. These steps are independent of one another and are relatively fast (timeouts ranging from five to twenty seconds). They are executed in parallel using an `errgroup.Group`, which both bounds concurrency and provides unified error propagation: if any critical step in Group 1 fails, the group context is cancelled and the remaining goroutines receive the cancellation signal, preventing wasted work. Group 2 contains the final four steps—Coding Challenge, Performance, Cost Analysis, and Final Validation. These steps must run sequentially because each step may depend on the results or side effects of its predecessor. For example, the Performance benchmark may use a model configuration discovered during Feature Detection, and Final Validation cannot execute until all prior results are available.

TABLE 1: Pipeline Step Mapping

Step	Name	Handler File	Timeout	Critical	Parallel Group
1	Existence	internal/verifier/ existence.go	5s	Yes	Group 1
2	Responsiveness	internal/verifier/ responsive.go	10s	Yes	Group 1
3	Feature Detection	internal/verifier/ feature_detect.go	15s	No	Group 1
4	Code Visibility	internal/verifier/ code_visibility.go	20s	No	Group 1
5	Coding Challenge	internal/verifier/ coding_challenge.go	60s	No	Group 2
6	Performance	internal/verifier/ perf.go	120s	No	Group 2
7	Cost Analysis	internal/verifier/ cost.go	10s	No	Group 2
8	Final Validation	internal/verifier/ validation.go	5s	Yes	Group 2

Each step in the pipeline implements the `Step` interface, which normalizes interaction between the orchestrator and the individual handlers regardless of their internal complexity.

CODE BLOCK 3: Pipeline with Parallel Groups

```

type Step interface {
    Name() string
    Execute(ctx context.Context, modelID string) (*StepResult, error)
    Timeout() time.Duration
    IsCritical() bool
}

type VerificationPipeline struct {
    group1 []Step // parallel: existence, responsive, feature, code
    group2 []Step // sequential: coding, perf, cost, validation
}

func (p *VerificationPipeline) Run(ctx context.Context, modelID string) (*PipelineResult, error) {
    g, ctx := errgroup.WithContext(ctx)
    results := make([]*StepResult, 0, 8)

    // Group 1: parallel
    for _, step := range p.group1 {
        s := step
        g.Go(func() error { r, err := s.Execute(ctx, modelID); results = append(results, r); return err })
    }
    if err := g.Wait(); err != nil { return nil, err }

    // Group 2: sequential
    for _, step := range p.group2 {

```



```

    r, err := step.Execute(ctx, modelID)
    results = append(results, r)
    if err != nil && step.IsCritical() { return nil, err }
}
return aggregateResults(results), nil
}

```

The `Step` interface enforces four methods. `Name()` returns a human-readable identifier used for logging, metrics tagging, and error messages. `Execute()` performs the actual verification work against a specific model identifier and returns a `*StepResult` containing a boolean `Passed` field, structured output data, and timing metadata. `Timeout()` declares the maximum duration the step is allowed to run before its context is cancelled. `IsCritical()` determines whether a failure in this step should cause the entire pipeline to abort. This last attribute is what enables the graduated admission policy: Existence and Responsiveness are critical because a model that does not exist or cannot respond is fundamentally unusable, while Feature Detection and Code Visibility are non-critical because their absence only limits functionality rather than preventing basic operation.

The `VerificationPipeline.Run` method is the engine's entry point. It begins by creating an `errgroup.WithContext(ctx)`, which derives a child context that is cancelled if any goroutine in the group returns a non-nil error. This is essential for the Group 1 parallel execution: if the Existence check fails critically, the derived context is cancelled, causing in-flight Feature Detection and Code Visibility calls to abort early. After Group 1 completes (either successfully or with a critical failure), Group 2 executes in a simple for loop. Each step's result is appended to the shared `results` slice, and critical failures immediately halt further progress. The final call to `aggregateResults` computes the `PipelineResult`, including the overall pass/fail determination and a summary of per-step outcomes.

Individual steps, particularly those that make network calls to external model APIs, are subject to transient failures. To prevent a single timeout or `503 Service Unavailable` from causing a model to be incorrectly rejected, each step's `Execute` method should be wrapped in a retry decorator. The retry logic uses exponential backoff with jitter to avoid synchronized retry storms, and it respects the circuit-breaker pattern by tracking recent failure rates and temporarily bypassing steps that have exceeded a failure threshold.

CODE BLOCK 4: Retry with Circuit Breaker

```

func executeStepWithRetry(step Step, ctx context.Context, modelID string, cfg RetryConfig)
↳ (*StepResult, error) {
    var lastErr error
    for attempt := 0; attempt < cfg.MaxAttempts; attempt++ {
        if attempt > 0 { time.Sleep(cfg.Backoff * time.Duration(1<<attempt)) }
        result, err := step.Execute(ctx, modelID)
        if err == nil { return result, nil }
        lastErr = err
        if step.IsCritical() && attempt < cfg.MaxAttempts-1 { continue }
    }
    return nil, fmt.Errorf("step %s failed after %d attempts: %w", step.Name(), cfg.MaxAttempts, lastErr)
}

```

The `executeStepWithRetry` function applies a configurable number of retry attempts with exponential backoff. The backoff calculation uses `cfg.Backoff * time.Duration(1<<attempt)`, producing delays that double on each subsequent retry: if the base backoff is one second, the delays are one second, two seconds,

and four seconds for attempts two, three, and four respectively. This aggressive backoff prevents the verifier client from hammering an already-stressed upstream API. The loop continues only for critical steps: if a non-critical step fails on every attempt, the function returns the final error but does not propagate it as a pipeline-halting failure. Instead, the caller records the step as failed but allows subsequent steps to proceed. For critical steps, the loop exhausts all retries before returning the wrapped error, which `VerificationPipeline.Run` then treats as a reason to abort the entire pipeline. The circuit-breaker integration, while not shown inline here, is realized by checking a `CircuitBreaker.Allow()` predicate at the start of `Execute`; if the breaker is open, the method returns immediately with a sentinel error that bypasses the retry loop.

3.3 Existence and Responsiveness Verification

The first two pipeline steps establish the most fundamental prerequisites: that the model identifier corresponds to a real, queryable model in the target provider's catalog, and that the model's inference endpoint responds within acceptable latency. These checks are both critical and fast, making them ideal candidates for Group 1 parallel execution.

Existence verification queries the model provider's API (or the verifier service's cached model registry) to confirm that the model ID is valid and the model is not deprecated or access-restricted. Because model catalogs change infrequently, the existence check benefits heavily from caching. A positive result is cached for five minutes, while a negative result is cached for thirty seconds to prevent rapid re-querying of a model that is legitimately unavailable. The cache key is a composite string formed as `"existence:" + modelID`, namespaced to avoid collisions with other cached verification data.

CODE BLOCK 5: Existence and Responsiveness with Caching

```
func (v *Verifier) VerifyModelExistence(ctx context.Context, modelID string) (bool, error) {
    if cached, ok := v.cache.Get("existence", modelID); ok {
        return cached.(bool), nil
    }
    exists, err := v.client.CheckModelExists(ctx, modelID)
    if err == nil { v.cache.Set("existence", modelID, exists, 5*time.Minute) }
    return exists, err
}
```

The `VerifyModelExistence` method begins by consulting the cache abstraction injected into the `Verifier` struct at construction time. The cache interface is defined in `internal/cache/` and supports pluggable backends (in-memory LRU for single-instance deployments, Redis for replicated deployments). If a cached value is found, it is returned immediately without any network call, reducing the latency of the Existence step to microseconds. On a cache miss, the method delegates to `v.client.CheckModelExists`, which performs an HTTP HEAD or GET request against the verifier service's model registry endpoint. The result is written back to the cache only on success, ensuring that transient errors do not poison the cache with false negatives.

Responsiveness verification complements existence by measuring the round-trip time to the model's inference endpoint. This step does not perform a full translation; instead, it sends a minimal prompt (such as a single-word echo request) and validates that a response is received within the ten-second timeout. The measured latency is recorded in the `StepResult` and contributes to the composite health score, but the step's criticality is based solely on whether any response is received at all. A model that exists but

cannot serve inference within the timeout window is treated as unavailable and is not admitted to the active pool. Like existence, responsiveness results are cached, but with a shorter TTL of one minute to ensure that the system reacts promptly to restored connectivity.

Together, these two steps form the “gatekeeping” layer of the verification pipeline. They are the fastest steps but carry the highest stakes: a failure here means the model is either phantom or unreachable, and no amount of feature richness or performance benchmarking can compensate. The caching strategy is particularly important at scale, where hundreds of models may be verified in a tight loop during a provider catalog refresh. Without caching, the system would generate a thundering herd of existence queries against the verifier service; with caching, only the first verification of each model in a five-minute window incurs network overhead.

3.4 Feature Detection and Code Visibility

Once a model’s existence and basic responsiveness are confirmed, the pipeline proceeds to assess its functional capabilities. Feature Detection enumerates which LLM capabilities are supported by the target model, while Code Visibility evaluates the model’s effectiveness with programming-language content. These two steps are non-critical because HelixTranslate can operate in a degraded mode when advanced features are unavailable, but their results are essential for routing decisions and for presenting accurate capability metadata to downstream consumers.

Feature detection uses a capability bitmap approach, which is both compact and efficient for comparison operations. Each capability is assigned a bit position in a `uint32` mask, allowing the full feature set of a model to be represented as a single integer. This representation enables fast subset queries—for example, checking whether a model supports both streaming and function calling is a single bitwise `AND` operation rather than a map lookup or string comparison.

CODE BLOCK 6: Feature Detection

```
type CapabilityBitmap uint32
const (
    CapStreaming CapabilityBitmap = 1 << iota
    CapFunctionCalling
    CapVision
    CapCodeGeneration
    CapJSONMode
    CapSystemPrompts
    CapLargeContext
    CapMultilingual
)

func DetectFeatures(ctx context.Context, client VerifierClient, modelID string) (CapabilityBitmap,
    error) {
    caps := CapabilityBitmap(0)
    if hasStreaming(ctx, client, modelID) { caps |= CapStreaming }
    if hasFunctions(ctx, client, modelID) { caps |= CapFunctionCalling }
    if hasVision(ctx, client, modelID) { caps |= CapVision }
    return caps, nil
}
```

The `CapabilityBitmap` type defines eight capability flags at present, with the most significant bit positions reserved for future expansion. The `DetectFeatures` function initializes an empty bitmap and then probes each capability independently. Each probe function (`hasStreaming`, `hasFunctions`, `hasVision`) sends a targeted test request to the model: `hasStreaming` checks whether the model returns a `text/event-stream` response for a streaming-enabled request; `hasFunctions` sends a prompt with a tool definition and validates that the model’s response includes a structured tool call; `hasVision` submits a prompt containing a base64-encoded image and checks for coherent visual understanding in the response. Each probe is wrapped in its own timeout context (typically three to five seconds) so that a slow probe for one capability does not block the detection of others.

TABLE 2: Feature Flags to HelixTranslate Capabilities

Feature Flag	Bit Value	HelixTranslate Capability	Fallback Behavior When Absent
CapStreaming	0x01	Real-time translation with incremental output	Buffer complete response before delivering to client
CapFunctionCalling	0x02	Structured glossary/terminology injection via tools	Prepend glossary terms to system prompt
CapVision	0x04	Translation of image-embedded documents (OCR + translate)	Reject image inputs with unsupported-media error
CapCodeGeneration	0x08	Context-aware code translation with syntax preservation	Use generic text translation with code-postprocessing
CapJSONMode	0x10	Structured output schemas (response format enforcement)	Parse free-form response with JSON heuristics
CapSystemPrompts	0x20	Per-request behavior customization via system messages	Embed instructions in the user prompt prefix
CapLargeContext	0x40	Document-level translation (>32K tokens input)	Split document into chunks with overlap stitching
CapMultilingual	0x80	Single-model multi-directional translation	Route through language-pair-specific model selection

Table 2 maps each feature flag to the corresponding HelixTranslate capability and describes the fallback strategy used when the flag is not set. This mapping is the bridge between raw model capability detection and actionable routing logic. For example, when `CapStreaming` is absent, the translation handler switches from a streaming response writer to a full-response buffer, delivering the complete translation only after the model finishes generation. When `CapVision` is absent, the API layer returns a `422 Unprocessable Entity` response with a descriptive error code, rather than attempting translation and producing garbled output. The fallback behaviors are implemented in the translation handlers themselves, which read the model’s cached capability bitmap at request time.

Code Visibility evaluation assesses whether the model can accurately process and translate programming language content. This step is distinct from `CapCodeGeneration` because a model may support generating code (the ability to produce syntactically valid output) without correctly translating existing code comments, variable names, or string literals between natural languages. The Code Visibility step sends a curated set of code snippets containing translatable comments and documentation strings, then evaluates whether the model preserves syntax structure while accurately translating the natural-language content. The evaluation uses a combination of static analysis (AST parsing to verify structural integrity)

and semantic similarity scoring (comparing translated comments against reference translations using embedding-based cosine similarity). Models that score below a configurable threshold on code visibility are still admitted to the pool but are excluded from the code-translation routing tier, ensuring that sensitive programming content is not routed to linguistically capable but code-naïve models.

3.5 Coding Challenge and Performance Benchmarking

Group 2 of the pipeline begins with two resource-intensive steps: the Coding Challenge and the Performance Benchmark. These steps are sequential because both generate load on the target model, and running them concurrently would skew latency measurements and potentially trigger rate limiting from the model provider.

The Coding Challenge step is a specialized variant of Code Visibility that focuses on active problem-solving rather than passive translation. It presents the model with a small set of well-known coding problems (e.g., implementing a specific algorithm, fixing a buggy function, or explaining a code snippet's behavior in another language) and evaluates the correctness of the response. This step serves two purposes: it validates that the model has retained its coding reasoning capabilities under the target inference configuration, and it produces a confidence score that feeds into the model's overall quality rating. The step is non-critical because a model that fails the coding challenge may still be perfectly adequate for general text translation, but the failure is recorded and used to downgrade the model's ranking in the selection algorithm.

The Performance Benchmark step is the longest-running step in the entire pipeline, with a timeout budget of 120 seconds. It measures the model's end-to-end translation latency across a representative sample set, computing percentile distributions and throughput metrics that directly inform routing decisions. The benchmark protocol follows a strict measurement discipline to ensure reproducible and comparable results across models and providers.

CODE BLOCK 7: Performance Benchmark

```
func BenchmarkTranslation(ctx context.Context, modelID string, samples []string) (*PerfResult, error) {
    var latencies []time.Duration
    var tokens int
    for _, text := range samples {
        start := time.Now()
        result, err := client.Translate(ctx, modelID, text)
        if err != nil { return nil, err }
        latencies = append(latencies, time.Since(start))
        tokens += result.TokenCount
    }
    return &PerfResult{
        P50Latency: percentile(latencies, 0.50),
        P95Latency: percentile(latencies, 0.95),
        P99Latency: percentile(latencies, 0.99),
        TokensPerSec: float64(tokens) / totalTime.Seconds(),
    }, nil
}
```

The `BenchmarkTranslation` function takes a context, a model identifier, and a slice of sample texts of varying lengths. It iterates over each sample, records the wall-clock latency of the translation call, and

accumulates the total token count from the response metadata. The function returns a `PerfResult` struct containing the P50, P95, and P99 latency percentiles, along with the average tokens-per-second throughput. These metrics are computed using a standard percentile function that sorts the latency slice and indexes into the sorted array.

TABLE 3: Benchmark Parameters

Parameter	Value	Description
Warm-up iterations	3	Performed before measurement to populate caches and establish connection pools; results are discarded
Sample texts	10	Curated to cover varied lengths from 100 to 5,000 characters, spanning short phrases, paragraphs, and multi-paragraph documents
Measurement duration	60s maximum	Hard cap per model to prevent benchmark runs from monopolizing the verification worker; early termination if all samples complete sooner
P95 TTFT target	< 5s	Time-to-first-token target; models exceeding this threshold receive a latency penalty in the scoring algorithm
P50 TPS target	> 10	Median tokens-per-second target; models below this threshold are flagged as low-throughput and deprioritized in routing

Table 3 enumerates the benchmark’s measurement parameters. The warm-up iterations are critical for fairness: without them, cold-start models (or models accessed over freshly established TLS connections) would report artificially high latencies that do not reflect steady-state performance. The ten sample texts are drawn from a curated corpus stored in `internal/verifier/benchmark_samples.json`, which includes short phrases (100 characters), standard paragraphs (1,000 characters), and long-form documents (5,000 characters) across multiple languages and domains. This diversity ensures that the benchmark score reflects real-world translation workloads rather than a single text length. The 60-second measurement cap prevents a single slow model from blocking the verification pipeline indefinitely; if all samples are not processed within the cap, partial results are extrapolated with a warning annotation. The P95 TTFT and P50 TPS targets are service-level objectives that feed into the admission scoring: a model meeting both targets receives a latency bonus, while a model missing either receives a penalty that affects its position in the routing priority queue.

The `PerfResult` is persisted alongside the model’s record in the verification cache, where it is refreshed on each scheduled verification cycle. Translation request handlers query this cached performance data when selecting a model from the active pool, preferring models with lower P95 latency for latency-sensitive requests and models with higher TPS for throughput-oriented batch jobs.

3.6 Cost Analysis and Final Validation

The final two steps of the pipeline transition from technical measurement to economic assessment and composite decision-making. Cost Analysis evaluates the financial efficiency of using the model for translation workloads, while Final Validation aggregates all preceding step results into a definitive admission verdict.

Cost analysis operates on the pricing data exposed by each model provider, which typically reports costs as separate rates for input tokens and output tokens. Because different providers use different pricing granularities (per 1K tokens, per 1M tokens, per character) and currencies, a normalization step is required before costs can be compared across the model pool.

CODE BLOCK 8: Cost Normalization and Final Validation

```
func NormalizeCost(providerID string, inputCost, outputCost float64) float64 {
    // Normalize to cost per 1M tokens across all providers
    total := inputCost + outputCost*2 // weight output higher
    return 10.0 * math.Exp(-total/10.0) // exponential decay score 0-10
}

func FinalValidation(results []*StepResult) *VerificationReport {
    passed := 0
    for _, r := range results { if r.Passed { passed++ } }
    return &VerificationReport{
        TotalSteps: len(results),
        Passed: passed,
        Failed: len(results) - passed,
        OverallPass: passed >= 6, // 75% threshold
    }
}
```

The `NormalizeCost` function converts raw pricing data into a dimensionless score on a 0-to-10 scale, where 10 represents the most cost-effective model and 0 represents the most expensive. The normalization formula uses an exponential decay: $10.0 * \text{math.Exp}(-\text{total}/10.0)$, where `total` is the weighted sum of input and output costs. Output tokens are weighted at 2x relative to input tokens because translation workloads typically generate output that is comparable in length to the input, and in many language pairs the output is longer due to morphological expansion. The exponential decay ensures that the score remains bounded and that small differences in low-cost models are amplified while large differences among high-cost models are compressed. For example, a model costing \$0.50 per 1M tokens receives a score near 9.5, while a model costing \$20.00 per 1M tokens receives a score near 1.4. This score is stored in the model's metadata and used by the cost-aware router to prefer economical models when quality differences are negligible.

Cost analysis also accounts for provider-specific billing models. Some providers charge for API calls in addition to token consumption; others offer volume discounts or reserved throughput pricing. The cost step queries the verifier service's pricing endpoint, which maintains a normalized pricing database across all supported providers. The step's timeout is intentionally short (10 seconds) because pricing data is cached at the verifier level and should not require live API calls.

The Final Validation step is the pipeline's terminal node. It receives the slice of all `StepResult` values produced by the preceding seven steps and computes the aggregate `VerificationReport`. The report contains the total number of steps, the count of passed and failed steps, and a boolean `OverallPass` field that

applies the admission threshold. The default threshold requires at least six of eight steps to pass, corresponding to a 75% success rate. This threshold is configurable via `LLMsVerifierConfig.MinPassingSteps` and can be tightened or relaxed based on operational requirements.

The threshold logic is intentionally lenient on non-critical steps. Because only three steps are marked as critical (Existence, Responsiveness, and Final Validation itself), a model can fail up to two non-critical steps and still achieve overall passage. For example, a model that passes Existence and Responsiveness but fails Feature Detection (because it lacks streaming support) and Code Visibility (because it performs poorly on code translation) would score six of eight passes, clearing the threshold and being admitted as a general-purpose text translation model with noted limitations. Conversely, a model that passes all non-critical steps but fails either Existence or Responsiveness is automatically rejected regardless of its other scores, because a non-existent or unreachable model cannot perform any translation.

The `VerificationReport` is the return value of `VerificationPipeline.Run` and is consumed by two downstream systems. First, it is written to the model cache, where the `OverallPass` flag determines whether the model is included in the active translation pool. Second, it is emitted as a structured log event and a Prometheus metric, enabling operations dashboards to track verification success rates, per-step failure frequencies, and model pool health over time. When a model transitions from passing to failing (or vice versa) between verification cycles, an alert is triggered so that operators can investigate provider outages, API changes, or model deprecation events.

The wiring of the entire Phase 2 engine into HelixTranslate is completed in `cmd/helix-translate/main.go`. After the configuration is loaded and the verifier client is initialized (as described in Section 3.1), the `VerificationPipeline` is constructed with its eight steps populated from the handler files in `internal/verifier/`. The pipeline is then passed to a background verification worker that runs on a configurable interval (default: five minutes), cycling through all registered models and refreshing their verification reports. The worker uses a bounded goroutine pool to control concurrency, ensuring that the verification load does not overwhelm either the HelixTranslate host or the upstream model providers. On each cycle, models with updated reports are re-evaluated for pool membership, and the translation router is notified of any changes to the active model set.

Phase 2 thus delivers a fully autonomous verification subsystem that continuously assesses model health, capability, performance, and cost, making dynamic admission decisions without human intervention. The combination of parallel fast checks, sequential deep checks, retry resilience, and composite scoring provides a robust foundation for the intelligent routing layer that will be built in Phase 3.

4. Phase 3: Scoring Engine and Score Adapter

The scoring engine constitutes the analytical core of the LLMsVerifier integration, transforming raw benchmark telemetry into actionable, ranked intelligence that HelixTranslate can consume for provider selection. This chapter documents the complete implementation of the `ScoringEngine` type, its five-component weighted scoring model, the composite score aggregation pipeline, the `ScoreAdapter` service that bridges LLMsVerifier's internal score representation to HelixTranslate's provider metadata model, and the persistent history subsystem that enables longitudinal trend analysis. Phase 3 is the longest-running stage of the integration and must be designed for both throughput — processing hundreds of score recalculations per evaluation cycle — and operational longevity, maintaining multi-year score archives without degradation to the translation system's cold-path latency.

4.1 Scoring Engine Initialization

The `ScoringEngine` is instantiated once at application startup and bound to the shared SQLite database handle that Phase 1 established. Its lifecycle is managed by the dependency-injection container defined in `cmd/helix-translate/main.go`, where the engine is registered as a singleton service under the interface `verifier.ScoringProvider`. The constructor receives four dependencies: the database connection (`*sql.DB`), an HTTP client for outbound benchmark requests, a `ScoreWeights` value object, and a structured logger. Each dependency is validated before assignment, and the constructor returns a descriptive error if any invariant is violated.

4.1.1 Configuration Options

The `ScoreWeights` struct and its surrounding configuration are populated from the `[verifier.scoring]` TOML stanza in `config.toml`. The scoring subsystem supports two operational profiles: `"standard"` and `"enhanced"`. In `"standard"` mode, scores are calculated directly from the latest benchmark run with no historical blending. In `"enhanced"` mode, the engine applies a seven-day exponential moving average (EMA) to each component before computing the composite, smoothing transient spikes caused by provider-side rate-limiting or temporary capacity reductions. HelixTranslate deployments default to `"enhanced"` mode in production because translation workloads are cost-sensitive and require stable rankings over multi-day horizons.

The full configuration surface is itemised in **Table 1**.

Table 1 — Scoring Engine Configuration Options

Option	Type	Default	Description
Mode	string	"standard"	"standard" or "enhanced"; enhanced enables 7-day EMA smoothing
WeightSpeed	float64	0.20	Response speed weight in composite calculation
WeightCost	float64	0.30	Cost-effectiveness weight; highest in HelixTranslate profile
WeightEfficiency	float64	0.25	Token-efficiency weight; output-per-input ratio
WeightCapability	float64	0.20	Capability weight; challenge pass-rate normalised
WeightRecency	float64	0.05	Model recency weight; exponential age decay
CacheTTL	duration	1h	In-memory positive-score cache TTL
DBPath	string	./data/verifier.db	Path to the score-history SQLite database

The weight values in **Table 1** reflect the HelixTranslate-optimised profile, not the generic `LLMsVerifier` defaults. The rationale for this redistribution — reducing the emphasis on raw speed and recency while elevating cost and efficiency — is explored in detail in Section 4.2. The `CacheTTL` controls how long a fully resolved `CompositeScore` remains valid in the process-local LRU cache. A one-hour TTL provides a practical balance: short enough to react to daily provider price changes or model deprecations, long enough to avoid redundant database queries during high-volume translation batches that may invoke `Adapt` dozens of times per document.

4.1.2 Engine Constructor

The constructor in **Code Block 1** performs three critical initialisation steps. First, it invokes `validateWeights`, which enforces the mathematical invariant that the five component weights must sum to 1.0 within a tolerance of 0.001. This prevents subtle ranking drift caused by unconstrained weight vectors. Second, it allocates an LRU cache with a capacity of 1,000 entries. Given that the HelixTranslate production fleet currently evaluates approximately 40 active provider-model pairs, this cache comfortably retains the entire working set with a 25× headroom factor for A/B test models and seasonal provider additions. Third, the constructor stores the dependencies in the struct fields, ensuring that the engine holds no nil references after a successful return.

Code Block 1 — Scoring Engine Constructor and Weight Validation

```
type ScoringEngine struct {
    db      *sql.DB
    client  *http.Client
    weights ScoreWeights
    cache   *lru.Cache
    mu      sync.RWMutex
    logger  *log.Logger
}

func NewScoringEngine(db *sql.DB, client *http.Client, weights ScoreWeights, logger *log.Logger)
↳ (*ScoringEngine, error) {
    if err := validateWeights(weights); err != nil {
        return nil, err
    }
    cache, _ := lru.New(1000)
    return &ScoringEngine{db: db, client: client, weights: weights, cache: cache, logger: logger}, nil
}

func validateWeights(w ScoreWeights) error {
    sum := w.Speed + w.Cost + w.Efficiency + w.Capability + w.Recency
    if math.Abs(sum-1.0) > 0.001 {
        return fmt.Errorf("weights must sum to 1.0, got %.3f", sum)
    }
    return nil
}
```

The `sync.RWMutex` (`mu`) serialises concurrent access to the LRU cache. In practice, the cache is read-heavy — once a model's composite score is resolved, subsequent translation requests for the same language pair and provider hit the read-locked fast path. Write locks are acquired only on cache misses and during cache invalidation triggered by the `CacheTTL` expiry. The database handle (`db`) is the same pooled SQLite connection used by the benchmark store and challenge bank, so no additional connection pool tuning is required at the scoring layer.

4.2 Five-Component Score Calculation

The composite score is a weighted sum of five normalised component scores, each mapped to the range [0, 10]. The choice of five components reflects the operational realities of running a translation service:

users demand accurate output (capability), operators must control per-token spend (cost), throughput scales with latency (speed), input-billing models reward token efficiency (efficiency), and newer models generally outperform older ones on multilingual benchmarks (recency). The mathematical formulation of each component is intentionally simple — logarithmic, linear, or ratio-based — to guarantee monotonicity, computational stability, and ease of debugging when a model’s rank shifts unexpectedly.

4.2.1 Weight Comparison: HelixAgent versus HelixTranslate

Table 2 contrasts the default weight vectors shipped with the upstream LLMsVerifier project (“HelixAgent”) against the values calibrated for HelixTranslate. The redistribution is the product of two months of production A/B testing across the six core language pairs supported by the translation service.

Table 2 — Weight Comparison: HelixAgent Defaults versus HelixTranslate Optimised

Component	HelixAgent Default	HelixTranslate Optimised	Rationale
Response Speed	25%	20%	Translation tolerates latency because documents are processed asynchronously; real-time chat demands faster turnaround
Cost	25%	30%	Translation workloads are token-volume heavy; a 1-cent per-million-token price delta compounds to thousands of dollars monthly at scale
Efficiency	20%	25%	Token efficiency is critical for translation because providers bill by input tokens; efficient models produce equivalent translations with fewer prompt tokens

Table 12 – continued

Component	HelixAgent Default	HelixTranslate Optimised	Rationale
Capability	20%	20%	Unchanged — translation quality must remain the floor constraint; no reduction in capability weight is acceptable
Recency	10%	5%	Model age matters less for translation than for general reasoning; established multilingual models retain competitiveness over longer horizons

The most significant change is the elevation of the **Efficiency** component from 20% to 25%. In HelixTranslate’s workload profile, translation prompts are long (hundreds to thousands of source-language tokens), and providers charge by input token count. A model that can achieve the same BLEU-equivalent quality with 30% fewer prompt tokens — for example, by accepting a shorter system instruction or by exhibiting less instruction-following overhead — delivers a direct cost reduction proportional to that efficiency gain. The 5-percentage-point reallocation from Recency to Efficiency reflects this economic reality: a two-year-old multilingual model that produces compact translations outranks a three-month-old generalist that requires verbose prompting.

4.2.2 Response Speed Score — Logarithmic Decay

The response-speed component translates the p50 (median) end-to-end latency, measured in milliseconds from request dispatch to the first response token, into a score on [0, 10]. The mapping uses a negative exponential function with a characteristic time constant of 1,000 milliseconds. At 100 ms, the score is approximately 9.0; at 1,000 ms, it falls to 3.68; at 2,000 ms, it approaches 1.35. This logarithmic compression ensures that ultra-low-latency providers receive diminishing score bonuses beyond the sub-200 ms threshold — a region where human perception of translation turnaround is already saturated — while high-latency providers are penalised rapidly without collapsing the score to zero.

Code Block 2 — Response Speed Score Calculator

```
func (e *ScoringEngine) CalculateResponseSpeedScore(p50LatencyMs float64) float64 {
    if p50LatencyMs <= 0 {
        return 10.0
    }
    score := 10.0 * math.Exp(-p50LatencyMs/1000.0) // 1s -> 3.68, 100ms -> 9.0
}
```

```

    return math.Max(0, math.Min(10, score))
}

```

The guard clause `p50LatencyMs <= 0` returns the maximum score of `10.0`. This handles the edge case where a provider’s benchmark data is stale or temporarily unavailable; the engine conservatively assigns the best possible speed score rather than zeroing out the component, which would unduly penalise the provider’s composite ranking.

4.2.3 Cost Score — Inverse Linear

The cost component maps the normalised cost-per-million-tokens figure onto `[0, 10]` using an inverse linear relationship. At `$0` per million tokens, the score is `10.0`; at `$50` per million, it reaches `0.0`. Providers priced above `$50` per million tokens are clamped to zero, reflecting a hard policy ceiling: any model costing more than this threshold is economically infeasible for production translation at HelixTranslate’s volume tier, regardless of its capability or speed.

Code Block 3 — Cost Score Calculator

```

func (e *ScoringEngine) CalculateCostScore(costPer1M float64) float64 {
    if costPer1M <= 0 {
        return 10.0
    }
    score := 10.0 * (1.0 - costPer1M/50.0) // $0 -> 10, $50 -> 0
    return math.Max(0, math.Min(10, score))
}

```

The cost-per-million value is computed as the weighted average of input and output token pricing, using a 3:1 input-to-output ratio that approximates HelixTranslate’s actual prompt-to-completion token distribution. This ratio is parameterised in the `[verifier.pricing]` configuration stanza and may be adjusted per language pair if empirical telemetry reveals a different distribution. The `costPer1M` parameter passed to `CalculateCostScore` is already normalised by the pricing loader, so the scoring function itself remains provider-agnostic.

4.2.4 Efficiency Score — Output-to-Input Ratio

Token efficiency is measured as the ratio of output tokens (the translated text) to input tokens (the source text plus system prompt, instructions, and any formatting overhead). For a perfectly efficient model, this ratio is `1.0` — every input token is converted to exactly one output token — yielding a score of `10.0`. Ratios below `1.0` indicate that the model “wastes” input tokens on redundant reasoning, chain-of-thought preamble, or verbose formatting, which inflates the provider’s bill without improving translation quality.

Code Block 4 — Efficiency Score Calculator

```

func (e *ScoringEngine) CalculateEfficiencyScore(outputTokens, inputTokens int) float64 {
    if inputTokens <= 0 {
        return 5.0
    }
    ratio := float64(outputTokens) / float64(inputTokens)
    score := ratio * 10.0 // ratio 1.0 -> 10, 0.5 -> 5
    return math.Max(0, math.Min(10, score))
}

```

The guard clause for `inputTokens <= 0` returns a neutral score of `5.0`, preventing division-by-zero panics while neither rewarding nor penalising the provider. In practice, this branch is rarely hit because the benchmark pipeline always records positive token counts. The clamping ensures that pathological cases — such as a model emitting zero output tokens due to a content-policy block — do not produce negative scores.

4.2.5 Capability Score — Challenge Pass Rate

The capability component reflects the model's performance on the curated challenge bank populated in Phase 1. Each challenge in the bank has an expected answer and a grading rubric (exact match, BLEU threshold, or semantic similarity). The pass rate is the ratio of challenges passed to challenges attempted. This figure is already normalised to `[0, 1]` by the challenge runner, and the scoring function simply scales it to `[0, 10]`.

Code Block 5 — Capability Score Calculator

```
func (e *ScoringEngine) CalculateCapabilityScore(passed, total int) float64 {
    if total == 0 {
        return 0.0
    }
    score := float64(passed) / float64(total) * 10.0
    return math.Max(0, math.Min(10, score))
}
```

When `total == 0` — for example, when a new provider has been registered but not yet benchmarked — the function returns `0.0`. This is the only component whose default-on-absence is zero rather than neutral. The rationale is conservative: a provider with no capability data must not be promoted to production routing until at least one full challenge-bank run has completed. The zero score ensures that such a provider's composite ranking falls below the minimum threshold (5.5 for the "budget" tier), keeping it out of the routing pool.

4.2.6 Recency Score — Exponential Age Decay

The recency component penalises model age using an exponential decay function with a half-life of `90` days. A model released today scores `10.0`; after 90 days, its score decays to `5.0`; after 180 days, to `2.5`; after one year, to approximately `1.25`. The half-life of 90 days was selected because it aligns with the typical cadence of major multilingual model releases (e.g., GPT-4 series, Claude family, Gemini updates). Models older than two years receive negligible recency scores but can still rank competitively if their capability, efficiency, and cost scores are sufficiently high.

Code Block 6 — Recency Score Calculator

```
func (e *ScoringEngine) CalculateRecencyScore(releaseDate time.Time) float64 {
    daysOld := time.Since(releaseDate).Hours() / 24
    halfLife := 90.0 // days
    score := 10.0 * math.Exp(-daysOld*math.Ln2/halfLife)
    return math.Max(0, math.Min(10, score))
}
```

The `releaseDate` is sourced from the model metadata table (`model_registry.release_date`), which is populated during Phase 2's provider registration workflow. For providers that do not publicly disclose

release dates, the integration falls back to the first-seen date recorded in the verifier's own telemetry, ensuring that the recency score is always computable even when upstream metadata is incomplete.

4.3 Composite Score and Thresholds

After the five component scores have been independently calculated and clamped to $[0, 10]$, the scoring engine computes the weighted composite using the HelixTranslate-optimised weight vector. The composite score itself is also clamped to $[0, 10]$, though in practice the individual component clamping makes this a defensive measure rather than an active correction.

4.3.1 Composite Score Aggregation

The `CompositeScore` struct serves as the canonical data transfer object (DTO) for downstream consumers. It carries the model identifier, a nested `ComponentScores` struct with the five raw component values, the overall weighted score, a tier classification string, and a UTC timestamp recording when the calculation occurred. This timestamp is critical for cache invalidation: the `ScoreAdapter` uses `CalculatedAt` to determine whether an in-memory cached score has exceeded the `CacheTTL` defined in Table 1.

The tier classification partitions models into three operational buckets:

- **Premium** (`overall` ≥ 8.5): High-trust models eligible for critical-translation routing (legal, medical, financial domains).
- **Standard** (`overall` ≥ 7.0): General-purpose models used for the majority of translation requests.
- **Budget** (`overall` ≥ 5.5): Acceptable models reserved for low-priority or high-volume batch jobs where cost minimisation outweighs quality maximisation.

Models scoring below 5.5 are classified as "unranked" and are excluded from the `ProviderSelector`'s active routing pool. They remain visible in the dashboard and API for diagnostic purposes but receive no production traffic.

Code Block 7 — Composite Score Calculation and Tier Assignment

```
type CompositeScore struct {
    ModelID      string      `json:"model_id"`
    Components    ComponentScores `json:"components"`
    Overall       float64      `json:"overall"`
    Tier          string      `json:"tier"`
    CalculatedAt time.Time    `json:"calculated_at"`
}

func (e *ScoringEngine) CalculateCompositeScore(components ComponentScores) (*CompositeScore, error) {
    overall := e.weights.Speed*components.Speed +
        e.weights.Cost*components.Cost +
        e.weights.Efficiency*components.Efficiency +
        e.weights.Capability*components.Capability +
        e.weights.Recency*components.Recency

    tier := "unranked"
    switch {
    case overall >= 8.5:
        tier = "premium"
```

```

    case overall >= 7.0:
        tier = "standard"
    case overall >= 5.5:
        tier = "budget"
    }

    return &CompositeScore{
        ModelID:    components.ModelID,
        Overall:    overall,
        Tier:       tier,
        Components: components,
        CalculatedAt: time.Now().UTC(),
    }, nil
}

func IsModelQualified(score *CompositeScore, minThreshold float64) bool {
    return score != nil && score.Overall >= minThreshold
}

```

The `IsModelQualified` helper is invoked by the `ProviderSelector` (Chapter 5) before including a model in the ranked candidate list. It performs a nil guard — essential because `CalculateCompositeScore` may return an error if a component calculation panics — and compares the overall score against a caller-defined threshold. In HelixTranslate’s default configuration, the selector uses 7.0 as the minimum threshold, meaning only "premium" and "standard" models are eligible for general routing, while "budget" models are reserved for explicitly marked low-priority jobs.

4.3.2 Enhanced Mode: EMA Smoothing

When the engine is configured in "enhanced" mode, the composite calculation in Code Block 7 is preceded by an exponential moving average (EMA) step applied to each component. The EMA formula is:

```
EMA_today = alpha * raw_score + (1 - alpha) * EMA_yesterday
```

where $\alpha = 2 / (N + 1)$ and $N = 7$ days. This yields an alpha of 0.25, meaning today’s raw score contributes 25% to the smoothed value while the historical EMA contributes 75%. The EMA state is persisted in the `score_ema` table with a composite primary key of (`model_id`, `component_name`), and is updated atomically within the same database transaction that records the raw score history (Section 4.5). The EMA step adds approximately 5 ms per model to the evaluation pipeline — a negligible overhead given that the entire scoring cycle completes in under 200 ms for the full provider fleet.

4.4 Score Adapter Service

The `ScoreAdapter` is the anti-corruption layer between LLMsVerifier’s internal score representation and HelixTranslate’s `ProviderScore` domain model. Its responsibilities are: (1) fetching the latest `CompositeScore` from the verifier’s API or local cache, (2) mapping fields according to the HelixTranslate naming convention, (3) stripping verifier-internal metadata suffixes from model names, and (4) caching the adapted result to avoid redundant transformations. The adapter lives in `internal/verifier/adapter.go` and is consumed by the translation engine’s `ProviderSelector` (Chapter 5) and by the management dashboard’s provider status endpoint.

4.4.1 Field Mapping

Table 3 documents the complete field-level mapping from LLMsVerifier’s CompositeScore and its nested ComponentScores to HelixTranslate’s ProviderScore struct.

Table 3 — Field Mapping: LLMsVerifier to HelixTranslate

LLMsVerifier Field	HelixTranslate Field	Type	Notes
ModelID	ProviderID	string	Direct mapping; used as the routing key
ModelName	ProviderName	string	Strip " (SC:x.x)" suffix added by verifier for display
OverallScore	OverallScore	float64	One-to-one numeric copy
SpeedScore	SpeedScore	float64	One-to-one numeric copy
CostScore	CostScore	float64	One-to-one numeric copy
EfficiencyScore	EfficiencyScore	float64	One-to-one numeric copy
CapabilityScore	CapabilityScore	float64	One-to-one numeric copy
RecencyScore	RecencyScore	float64	One-to-one numeric copy
LastCalculated	LastUpdated	time.Time	Renamed for consistency with HelixTranslate’s audit vocabulary

The mapping is almost entirely mechanical — eight of the nine fields are copied without transformation — which is by design. The scoring engine already performs the complex work of normalising, weighting, and tiering. The adapter’s only semantic responsibility is the **ProviderName** sanitisation: the verifier UI appends a composite-score suffix (e.g., " (SC:7.3)") to model names for dashboard readability, but this suffix must be removed before the name is exposed to end users or logged in translation audit trails.

4.4.2 Adapter Implementation

The adapter maintains its own LRU cache, separate from the scoring engine’s cache, because the two caches have different keys and TTL semantics. The adapter cache is keyed by **modelID** (string) and stores ***ProviderScore** values. The **CacheTTL** from Table 1 controls how long an adapted score remains valid; when a cached entry expires, the adapter re-fetches from the verifier client rather than recomputing the

score. This separation of concerns means that adapter cache misses do not necessarily trigger scoring engine recalculations — they may simply require a field-mapping pass over an already-resolved composite score.

Code Block 8 — ScoreAdapter with Caching and Field Mapping

```
type ScoreAdapter struct {
    client    *VerifierClient
    cache     *lru.Cache
    cacheTTL  time.Duration
    mu        sync.RWMutex
}

func (a *ScoreAdapter) Adapt(ctx context.Context, modelID string) (*ProviderScore, error) {
    a.mu.RLock()
    if cached, ok := a.cache.Get(modelID); ok {
        a.mu.RUnlock()
        return cached.(*ProviderScore), nil
    }
    a.mu.RUnlock()

    score, err := a.client.GetScore(ctx, modelID)
    if err != nil {
        return nil, fmt.Errorf("fetch score for %s: %w", modelID, err)
    }

    adapted := &ProviderScore{
        ProviderID:    score.ModelID,
        ProviderName:  strings.TrimSuffix(score.ModelName, " (SC:"+fmt.Sprintf("%.1f",
            ↪ score.OverallScore)+")"),
        OverallScore:  score.OverallScore,
        SpeedScore:    score.Components.SpeedScore,
        CostScore:     score.Components.CostScore,
        EfficiencyScore: score.Components.EfficiencyScore,
        CapabilityScore: score.Components.CapabilityScore,
        RecencyScore:  score.Components.RecencyScore,
        LastUpdated:  score.LastCalculated,
    }

    a.mu.Lock()
    a.cache.Add(modelID, adapted)
    a.mu.Unlock()
    return adapted, nil
}
```

The `Adapt` method follows a read-through cache pattern. The read lock (`RLock`) is held only for the cache lookup, minimising contention with concurrent translation requests. If the cache misses, the lock is released before the network call to `a.client.GetScore`, preventing the adapter from holding a lock across I/O. After the client returns a `CompositeScore`, the adapter performs the field mapping, acquires a write lock, and inserts the adapted value into the cache. The cache eviction policy is LRU with a capacity of 500 entries, which is half the engine's cache capacity because the adapter typically serves a smaller subset

of models — only those currently enabled for translation routing.

4.4.3 Batch Adaptation

For bulk operations — such as the dashboard’s provider list endpoint or the selector’s initialisation pass — the adapter exposes a `AdaptBatch` method that accepts a slice of `modelID` strings and returns a map of `modelID` to `*ProviderScore`. `AdaptBatch` uses a single read lock for the cache scan, then issues parallel client fetches for the missing entries using a worker pool sized to `min(len(missing), 10)`. This batch path reduces the aggregate latency of adapting 40 models from approximately $40 \times 15\text{ ms} = 600\text{ ms}$ (sequential) to under 80 ms (parallel with 10 workers), a critical improvement for the dashboard’s cold-start render time.

4.5 Score Persistence

Longitudinal score tracking is essential for two operational functions within HelixTranslate: trend-based provider deprecation alerts and retrospective cost-quality analysis. The scoring engine records every composite score calculation to the `score_history` table, which is query-optimised for time-range scans by `model_id` and `calculated_at`. The persistence layer is intentionally simple — raw SQL `INSERT` statements executed against the shared SQLite handle — because score writes are append-only and require no complex transactional logic beyond the atomic EMA update described in Section 4.3.2.

4.5.1 Retention Policy

Score history accumulates rapidly: with 40 models, daily evaluations, and five component scores per model, the system generates approximately 73,000 rows per year. To prevent unbounded storage growth, a three-tier retention policy is applied by a background goroutine launched at engine initialisation.

Table 4 — Score History Retention Policy

Period	Retention Granularity	Action
0–90 days	All individual scores	Full granularity for operational dashboards and debugging
90–365 days	Weekly aggregates	Average composite and component scores per model per ISO week
1+ years	Monthly archives	Compressed JSON blobs moved to cold storage directory (<code>./data/archives/</code>)

The retention policy is enforced by a scheduled job that runs at 02:00 UTC daily, a time chosen to minimise interference with the 03:00 UTC benchmark cycle (Chapter 3). The job executes three SQL statements inside a single transaction:

1. **Roll-up:** For records older than 90 days but newer than 365 days, compute weekly averages grouped by `model_id` and `strftime('%Y-%W', calculated_at)`, insert the aggregates into `score_history_weekly`, and delete the corresponding raw rows from `score_history`.
2. **Archive:** For records older than 365 days, export the rows to a gzip-compressed JSON file named `score_history_YYYY-MM.json.gz` in the cold storage directory, then delete the rows from the hot table.
3. **Vacuum:** Run `VACUUM` on the SQLite database to reclaim the freed pages and reduce the on-disk footprint.

The roll-up step preserves statistical accuracy because the component scores are linear combinations, and the average of a linear combination equals the linear combination of the averages. Therefore, weekly aggregate scores can be fed back into the EMA calculation without loss of mathematical correctness.

4.5.2 History Recording

The `RecordScoreHistory` method is called synchronously at the end of every composite score calculation, ensuring that the database always contains a complete audit trail up to the most recent evaluation. The insertion includes all five component scores, the composite score, and the tier classification, enabling retrospective queries such as “show me all models that transitioned from `standard` to `budget` tier in the last 30 days” — a query that powers the provider-health alert system.

Code Block 9 — Score History Recording

```
func (e *ScoringEngine) RecordScoreHistory(score *CompositeScore) error {
    _, err := e.db.Exec(`
        INSERT INTO score_history
            (model_id, composite_score, component_speed, component_cost,
             component_efficiency, component_capability, component_recency, tier, calculated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
        score.ModelID, score.Overall,
        score.Components.Speed, score.Components.Cost,
        score.Components.Efficiency, score.Components.Capability,
        score.Components.Recency, score.Tier,
        score.CalculatedAt.UTC().Format(time.RFC3339),
    )
    if err != nil {
        e.logger.Printf("ERROR: failed to record score history for %s: %v", score.ModelID, err)
    }
    return err
}
```

The `calculated_at` column is stored as an RFC 3339 string to ensure consistent time-zone handling across SQLite clients. The error is both returned to the caller and logged, because a failed history write is a non-fatal event — the composite score remains valid and usable for routing — but it represents a data-loss incident that operators must be able to detect and investigate.

4.5.3 Query Interface

In addition to raw inserts, the persistence layer exposes three query functions consumed by the dashboard and alerting subsystems:

- `GetScoreHistory(modelID string, since, until time.Time) ([]ScoreHistoryRow, error)` — returns all raw or weekly-aggregated rows for a given model in a time range, transparently routing the query to `score_history` or `score_history_weekly` based on the range endpoints.
- `GetTierTransitions(since time.Time) ([]TierTransition, error)` — returns a list of `(model_id, old_tier, new_tier, transitioned_at)` tuples for all models that changed tier classification within the specified window.
- `GetAverageScore(modelID string, window time.Duration) (ComponentScores, error)` — returns the arithmetic mean of each component score over the specified sliding window, used by the EMA pre-warming routine when a model is re-enabled after a prolonged absence.

All query functions are implemented in `internal/verifier/history_store.go` and share the same database handle as the scoring engine. They use prepared statements cached on the `sql.DB` connection pool to amortise parse-plan overhead across repeated dashboard polls.

4.6 Phase 3 Integration Checklist

Before proceeding to Phase 4 (Provider Selector Integration), the following acceptance criteria must be satisfied:

1. **Weight Validation:** `NewScoringEngine` rejects weight vectors that do not sum to 1.0 ± 0.001 .
2. **Component Monotonicity:** Each of the five component calculators is verified to produce monotonically non-increasing scores as the input metric degrades (higher latency, higher cost, lower efficiency, lower pass rate, older release date).
3. **Tier Assignment:** A model with a synthetic perfect-component profile (`Speed=10`, `Cost=10`, `Efficiency=10`, `Capability=10`, `Recency=10`) receives "premium" tier; a model with all zeros receives "unranked".
4. **Adapter Round-Trip:** The `ScoreAdapter` produces a `ProviderScore` whose fields match the original `CompositeScore` after accounting for the name-suffix stripping.
5. **History Persistence:** After 30 days of continuous operation, the `score_history` table contains exactly one row per model per day, and no score data older than the retention window remains in the hot table.
6. **Cache Correctness:** The combined hit rate of the engine cache and adapter cache exceeds 85% under a simulated production load of 1,000 translation requests per minute across 40 models.

Completion of this checklist confirms that the scoring pipeline is ready to feed ranked provider data into the selector and scheduler layers documented in the following chapters.

5. Phase 4: Model Discovery and Registry Integration

The discovery and registry subsystem forms the informational backbone of the `LLMsVerifier` integration, responsible for identifying available language models across all configured providers, maintaining a durable and queryable catalog of verified models, and enforcing admission policies that prevent unqualified models from entering the translation pipeline. This phase implements a resilient three-tier discovery architecture that balances user control against automation, a local SQLite-backed registry with full-text search capabilities, and a gatekeeping layer that applies composite-score and health-status filters before any model can be selected for inference. All discovery results are persisted to the unified registry, indexed for efficient lookup, and exposed through a stable REST API that serves both the internal scorer and

external administrative tools. The subsystem is designed to operate continuously in the background, refreshing model availability at configurable intervals while broadcasting change events to interested consumers throughout the HelixTranslate system.

5.1 Three-Tier Discovery Service

The `DiscoveryService` orchestrates model enumeration across three independent tiers, each offering a different trade-off between authority and freshness. Tier 1 draws from the user’s explicit configuration in `configs/verifier.yaml`, where administrators can whitelist specific model IDs, set per-model score thresholds, or pin provider preferences. This tier carries the highest priority because user-specified values represent deliberate policy decisions that must override any dynamically discovered alternative. When Tier 1 returns at least one model, the discovery process terminates immediately and returns only the user-configured entries, ensuring that no provider API or community registry can supersede administrator intent.

Tier 2 performs dynamic enumeration by querying each configured provider’s model list endpoint. OpenAI-compatible providers expose `/v1/models`; Anthropic provides `v1/models`; and custom adapters normalize their native APIs to the same response shape. Each provider query is executed with a 30-second timeout and runs in its own goroutine, with results aggregated through a buffered channel. The provider API tier is the primary source of truth for newly released models or pricing changes, but it is only consulted when the user configuration tier returns no entries. This fallback ordering prevents API quota consumption when the administrator has already specified a closed set of acceptable models.

Tier 3 queries the `models.dev` community registry, a publicly maintained database of language model metadata including community-verified scores, pricing benchmarks, and capability tags. This tier serves as the final fallback when both user configuration and live provider APIs are unavailable or return empty results. The registry endpoint responds within 10 seconds and provides models that may not yet be present in any configured provider account, enabling the system to pre-evaluate models before they are commercially available. Because community data can be stale or unverified, Tier 3 results receive the lowest priority and are always marked with `source: "community"` for downstream auditing.

The cascading logic is implemented in `DiscoverAll`, which sequentially attempts each tier, deduplicates results by model ID, and returns the first non-empty set. Deduplication preserves the highest-priority entry when the same model appears in multiple tiers, ensuring that a user-configured whitelist entry always dominates a matching API-discovered entry even if both tiers were eventually queried.

Table 5.1: Discovery Tier Configuration

Tier	Source	Timeout	Priority	Refresh
1	User config (<code>configs/verifier.yaml</code>)	5s	Highest	On config change
2	Provider API dynamic enumeration	30s/provider	Medium	1 hour
3	<code>models.dev</code> community registry	10s	Lowest (fallback)	24 hours

The `DiscoveryService` struct embeds three tier-specific implementations, each satisfying a common `Dis-`

covery interface that abstracts the underlying enumeration mechanism. A `sync.RWMutex` protects the service state during concurrent discovery runs, ensuring that background sync goroutines and on-demand API calls do not race on shared configuration.

```
// pkg/discovery/service.go
type DiscoveryService struct {
    tier1 *ConfigDiscovery // User-configured
    tier2 *APIDiscovery   // Provider APIs
    tier3 *RegistryDiscovery // models.dev
    mu    sync.RWMutex
}

func (s *DiscoveryService) DiscoverAll(ctx context.Context) ([]DiscoveredModel, error) {
    var allModels []DiscoveredModel

    // Tier 1: User config
    if models, err := s.tier1.Discover(ctx); err == nil {
        allModels = append(allModels, models...)
    }
    if len(allModels) > 0 {
        return deduplicate(allModels), nil
    }

    // Tier 2: Provider APIs
    if models, err := s.tier2.Discover(ctx); err == nil {
        allModels = append(allModels, models...)
    }
    if len(allModels) > 0 {
        return deduplicate(allModels), nil
    }

    // Tier 3: models.dev
    if models, err := s.tier3.Discover(ctx); err == nil {
        allModels = append(allModels, models...)
    }
    return deduplicate(allModels), nil
}
```

The `deduplicate` helper selects entries by priority order. Each `DiscoveredModel` carries a `SourceTier` field populated by the originating discovery implementation. When two entries share the same `ModelID`, the one with the numerically lower tier value is retained. After deduplication, the final slice is sorted lexicographically by model ID to provide deterministic output across repeated calls. This deterministic ordering is critical for cache key generation and for test assertions that verify discovery behavior.

Each tier implementation handles its own caching and error semantics. `ConfigDiscovery` watches `configs/verifier.yaml` via `fsnotify` and reloads on `Write` events, providing near-instant refresh without polling. `APIDiscovery` maintains an in-memory cache of provider responses with a 1-hour TTL, serving stale data during transient provider outages while queuing a background refresh. `RegistryDiscovery` implements exponential backoff for failed requests to the `models.dev` endpoint, waiting 1 minute after the first failure, 5 minutes after the second, and capping at 30 minutes to avoid hammering the community service. All three implementations emit structured log entries at `INFO` level for successful discoveries and `WARN` level

for timeouts or non-2xx HTTP responses.

5.2 Unified Model Registry

Discovered models are persisted to a local SQLite database that serves as the single source of truth for all downstream components. The registry schema is designed to capture every dimension needed for model selection: identification, verification status, scoring, capabilities, pricing, availability, and temporal metadata. The `verified_models` table enforces data integrity through `CHECK` constraints on enumerated columns and uses a composite index strategy optimized for the query patterns observed in production.

The `model_id` column stores the canonical model identifier as returned by the provider (e.g., `gpt-4o-2024-08-06`, `claude-sonnet-4-20250514`). The `provider_id` foreign key links to a separate `providers` table defined in Phase 3. The `status` column implements the model lifecycle: `pending` for newly discovered models awaiting first evaluation, `verified` for models that have completed at least one full scoring run, `failed` for models that failed verification due to repeated inference errors or policy violations, and `retired` for models that have been removed from all provider APIs and community registries. A model in `retired` status remains in the database for audit purposes but is excluded from all selection queries.

The `composite_score` column stores the weighted aggregate score produced by Phase 6’s scoring pipeline, ranging from 0.0 to 10.0. The `tier` column classifies models into four bands: `premium` (score ≥ 8.0), `standard` (score ≥ 6.0), `budget` (score ≥ 4.0), and `unranked` (below 4.0 or no score). Both `capabilities` and `pricing` columns store JSON documents, allowing the schema to evolve without migration scripts as providers introduce new features or pricing dimensions. The `is_available` boolean reflects the most recent health check result and is used as a fast filter to exclude down providers without evaluating score predicates.

Table 5.2: SQLite Registry Schema

```
-- pkg/registry/schema.sql
CREATE TABLE verified_models (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  model_id TEXT UNIQUE NOT NULL,
  provider_id TEXT NOT NULL,
  name TEXT NOT NULL,
  status TEXT NOT NULL CHECK(status IN ('pending','verified','failed','retired')),
  composite_score REAL,
  tier TEXT CHECK(tier IN ('premium','standard','budget','unranked')),
  capabilities TEXT, -- JSON
  pricing TEXT, -- JSON
  verified_at TIMESTAMP,
  last_checked TIMESTAMP,
  is_available BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_models_score ON verified_models(composite_score);
CREATE INDEX idx_models_provider ON verified_models(provider_id);
CREATE INDEX idx_models_status ON verified_models(status);
CREATE INDEX idx_models_available ON verified_models(is_available, composite_score);
```

The index design reflects the system’s dominant access patterns. `idx_models_score` accelerates `Filter-`

ByScore queries used by the gatekeeper to find all models above a threshold. `idx_models_provider` speeds up provider-specific administrative views. `idx_models_status` supports batch operations that target models in a particular lifecycle state, such as re-evaluating all `failed` models after a configuration change. The composite index `idx_models_available` is the most critical for runtime performance: nearly every production query includes `is_available = TRUE`, and the secondary ordering by `composite_score` allows the query planner to satisfy `ORDER BY composite_score DESC` without an additional sort step.

The `ModelRegistry` type provides the Go API for all database operations. It uses `database/sql` with `modernc.org/sqlite` as the driver, avoiding CGO dependencies for simpler cross-compilation. Connection pooling is configured with a maximum of 10 open connections and 5 idle connections, sufficient for the single-node deployment model. All methods accept a `context.Context` for timeout and cancellation propagation, and queries exceeding 5 seconds are logged at `WARN` level for operational monitoring.

```
// pkg/registry/registry.go
type ModelRegistry struct {
    db *sql.DB
}

func (r *ModelRegistry) ListModels(ctx context.Context, filter ModelFilter) ([]Model, error)
func (r *ModelRegistry) GetModel(ctx context.Context, modelID string) (*Model, error)
func (r *ModelRegistry) FilterByScore(ctx context.Context, minScore float64) ([]Model, error)
func (r *ModelRegistry) FilterByProvider(ctx context.Context, providerID string) ([]Model, error)
func (r *ModelRegistry) SearchModels(ctx context.Context, query string) ([]Model, error)
```

`ListModels` accepts a `ModelFilter` struct containing optional fields for `ProviderID`, `Status`, `MinScore`, `MaxScore`, `Tier`, and `IsAvailable`. The implementation builds parameterized SQL dynamically, including only non-zero filter predicates in the `WHERE` clause. Results are ordered by `composite_score DESC` when a score filter is present, otherwise by `name ASC`. `GetModel` performs a primary-key lookup by `model_id` and returns `sql.ErrNoRows` as a sentinel when the model is not found, allowing callers to distinguish between missing data and database errors. `FilterByScore` and `FilterByProvider` are convenience methods that delegate to `ListModels` with appropriately populated filters, exposed separately because they represent the two most common query patterns in the selection pipeline.

`SearchModels` implements full-text search over the `name` column using SQLite’s `LIKE` operator with `%` wildcards prepended and appended to the query term. While this is not a true full-text search engine, it satisfies the administrative use case of finding models by partial name match (e.g., searching “gpt-4” to find all GPT-4 variants). For installations requiring more sophisticated search, the method can be replaced with an FTS5-backed implementation without changing the interface. Search results are ordered by `composite_score DESC` to surface the highest-quality matches first.

Table 5.3: Registry Query API Reference

Method	Parameters	Return	SQL Complexity	Cache
ListModels	ModelFilter	[]Model	WHERE + ORDER	5min
GetModel	modelID	*Model	PRIMARY KEY	10min
FilterByScore	minScore	[]Model	WHERE score >=	None
FilterByProvider	providerID	[]Model	WHERE provider	1min
SearchModels	query	[]Model	LIKE on name	None

The registry implements a read-through caching layer using an in-memory LRU cache with a default capacity of 1,000 entries. `GetModel` results are cached for 10 minutes because individual model metadata changes infrequently. `ListModels` with filters is cached for 5 minutes, with cache keys computed from the serialized filter struct. Score and provider filters bypass the cache because they are typically invoked by automated pipelines that benefit from fresh data and operate at frequencies where cache hits would be rare. Search queries are also uncached due to the high cardinality of possible query strings. Cache invalidation is triggered by `ModelUpdatedEvent` broadcasts (see Section 5.4), ensuring that model metadata changes are reflected in subsequent queries without waiting for TTL expiration.

5.3 Verified Model Gatekeeping

The gatekeeping layer enforces admission control, determining whether a specific model is eligible for translation work based on its verification status, composite score, and health state. This component sits at the boundary between model discovery and model selection: it answers the binary question “can this model be used right now?” with a detailed reason when the answer is negative. The `Gatekeeper` type is initialized with a `minScoreThreshold` (default 5.5) and a reference to the `ModelRegistry` for lookups.

The `IsModelAvailable` method performs a synchronous database lookup followed by a sequence of predicate checks. First, it verifies that the model exists in the registry; a missing model returns `false` with reason `"not_found"`. Second, it checks that the model's `status` column equals `"verified"`; models in `pending`, `failed`, or `retired` states are rejected with reason `"verification"`. Third, it compares the model's `CompositeScore` against the configured threshold; models scoring below the minimum are rejected with reason `"score"`. Fourth, it examines the `HealthStatus` field, which must be either `"healthy"` or `"degraded"`; models marked `"down"` or `"unknown"` are rejected with reason `"health"`. Only when all four predicates pass does the method return `true` with an empty reason string.

```
// pkg/gatekeeper/gatekeeper.go
type Gatekeeper struct {
    minScoreThreshold float64
    registry           *ModelRegistry
}

func (g *Gatekeeper) IsModelAvailable(modelID string) (bool, string, error) {
    model, err := g.registry.GetModel(context.Background(), modelID)
    if err != nil {
        return false, "not_found", err
    }
    if model.Status != "verified" {
        return false, "verification", nil
    }
    if model.CompositeScore < g.minScoreThreshold {
        return false, "score", nil
    }
    if model.HealthStatus != "healthy" && model.HealthStatus != "degraded" {
        return false, "health", nil
    }
    return true, "", nil
}
```

The gatekeeper is intentionally conservative: a model must demonstrate sustained positive evidence to

pass. This design prevents transiently available models with unknown quality from entering the translation pipeline. The `minScoreThreshold` is configurable at startup via the `VERIFIER_MIN_SCORE` environment variable and can be adjusted at runtime through a configuration hot-reload endpoint. Lowering the threshold increases model diversity but may reduce average translation quality; raising it improves quality guarantees but decreases redundancy in the model pool.

The reason strings returned by `IsModelAvailable` are used for observability and debugging. Each rejection reason increments a Prometheus counter `gatekeeper_rejections_total{reason="..."}`, allowing operators to monitor whether rejections are dominated by score failures (indicating the threshold may be too high), health failures (indicating provider instability), or verification failures (indicating models stuck in pending state). These metrics feed into the alerting rules defined in Phase 8.

Table 5.4: Gatekeeping Decision Matrix

Verification	Score ≥ 5.5	Health	Available	Reason
Pass	Yes	Healthy	TRUE	OK
Pass	Yes	Degraded	TRUE	DEGRADED
Pass	Yes	Down	FALSE	PROVIDER_DOWN
Pass	No	Any	FALSE	SCORE_LOW
Fail	Any	Any	FALSE	VERIFICATION_FAILED
Pending	Any	Any	FALSE	PENDING

When a model passes gatekeeping with `DEGRADED` health status, it is admitted to the pool but with reduced traffic weight. The selection algorithm (Phase 6) applies a 0.5x multiplier to the degraded model's routing probability, shifting load toward fully healthy alternatives while maintaining the degraded model as a warm standby. If all models in a tier are degraded, the system continues operating at reduced capacity rather than rejecting requests, providing graceful degradation rather than hard failure. A model marked `Down` is completely excluded from the selection set, and if all models across all providers are down, the system returns a 503 Service Unavailable with a descriptive error message listing the affected providers.

5.4 Background Discovery and Event Broadcasting

Discovery is not a one-time operation. Provider catalogs change as new models are released, old models are deprecated, and pricing is adjusted. The `DiscoveryService` runs a background synchronization loop that periodically re-executes the three-tier discovery pipeline, compares results against the current registry state, and publishes change events for any detected differences.

The `StartBackgroundSync` method accepts a context and a refresh interval (default 1 hour). It creates a `time.Ticker` and adds a random jitter of up to 10% of the interval duration before the first tick. This jitter prevents thundering herd behavior when multiple `HelixTranslate` instances start simultaneously and share the same configuration. Each tick triggers a full `DiscoverAll` call; when new models are found, an event is published to the central event bus for each discovered entry. Errors during discovery are logged but do not terminate the goroutine, ensuring that transient network issues do not permanently disable background synchronization.

```
// pkg/discovery/background.go
func (s *DiscoveryService) StartBackgroundSync(ctx context.Context, interval time.Duration) {
    ticker := time.NewTicker(interval)
    jitter := time.Duration(rand.Int63n(int64(interval / 10)))
```

```

go func() {
    time.Sleep(jitter) // Prevent thundering herd
    for {
        select {
        case <-ticker.C:
            models, err := s.DiscoverAll(ctx)
            if err != nil {
                log.Printf("discovery error: %v", err)
                continue
            }
            for _, m := range models {
                events.Publish(ctx, events.ModelDiscovered{
                    ModelID:    m.ID,
                    ProviderID: m.ProviderID,
                })
            }
        case <-ctx.Done():
            ticker.Stop()
            return
        }
    }
}()
}

```

The event bus (`pkg/events`) provides a publish-subscribe mechanism with at-least-once delivery semantics. Subscribers register typed handlers for specific event kinds, and the bus dispatches published events to all matching handlers in parallel. Event delivery is asynchronous, with a 30-second timeout per handler to prevent slow consumers from blocking the publisher. Events that fail delivery are retried up to 3 times with exponential backoff, then written to a dead-letter log for manual inspection.

Four event types capture the full lifecycle of a model in the registry. The `ModelDiscoveredEvent` fires when a background sync or manual discovery finds a model not currently present in the registry. The `ModelUpdatedEvent` fires when an existing model's metadata changes, such as a pricing update or capability addition detected during a subsequent provider API query. The `Changes` map contains only the fields that differ from the previous registry entry, allowing subscribers to perform incremental updates. The `ModelScoreChangedEvent` fires when a model's composite score crosses a tier boundary, enabling subscribers to react to promotions (e.g., budget to standard) or demotions (premium to standard) with appropriate log entries or alerts. The `ModelRemovedEvent` fires when a model that was previously in the registry disappears from all discovery tiers for three consecutive sync cycles, indicating provider deprecation or retirement.

```

// pkg/events/model_events.go
type ModelDiscoveredEvent struct {
    ModelID    string
    ProviderID string
    Name       string
    Source     string
    Timestamp  time.Time
}

type ModelUpdatedEvent struct {

```

```

    ModelID    string
    Changes    map[string]interface{}
    Timestamp  time.Time
}

type ModelScoreChangedEvent struct {
    ModelID    string
    OldScore   float64
    NewScore   float64
    OldTier    string
    NewTier    string
    Timestamp  time.Time
}

type ModelRemovedEvent struct {
    ModelID    string
    Reason     string
    Timestamp  time.Time
}

```

The registry subscribes to all four event types and updates its SQLite database accordingly. On `ModelDiscoveredEvent`, it inserts a new row with `status = 'pending'` and queues an asynchronous scoring job. On `ModelUpdatedEvent`, it applies the changes map using a JSON merge patch for the `capabilities` and `pricing` columns, and bumps the `updated_at` timestamp. On `ModelScoreChangedEvent`, it updates both the `composite_score` and `tier` columns, triggering any downstream cache invalidation. On `ModelRemovedEvent`, it transitions the model to `status = 'retired'` rather than deleting the row, preserving historical audit data. The scoring pipeline (Phase 6) subscribes to `ModelDiscoveredEvent` to trigger initial evaluations and to `ModelScoreChangedEvent` to log tier transitions for analytics.

Administrators can trigger on-demand discovery through the REST API (Section 5.5), which bypasses the background ticker and immediately executes `DiscoverAll`. This is useful when a new model has just been enabled in a provider account and the operator does not want to wait for the next scheduled sync. On-demand discovery publishes the same events as background sync, ensuring consistent handling regardless of trigger source.

5.5 REST API Endpoints

The discovery and registry functionality is exposed through a JSON REST API under the `/api/v1/models` path prefix. All endpoints require authentication via the bearer token middleware established in Phase 3, and all responses include standard `X-Request-ID` headers for traceability. The API follows REST conventions: collection endpoints return arrays, singleton endpoints return objects, and write operations return the affected resource. Error responses conform to RFC 7807 Problem Details format with `application/problem+json` content type.

`GET /api/v1/models` returns a paginated list of models from the registry, accepting query parameters for `provider`, `status`, `min_score`, `tier`, and `available` to filter results. Pagination uses cursor-based navigation with a `limit` parameter (default 50, max 200) and an `after` cursor for consistent ordering across insertions. The response includes a `Link` header with `next` and `prev` relations when additional pages exist. Each model object in the response array contains the full registry record with `capabilities` and `pricing` JSON objects inlined for convenience.

GET `/api/v1/models/{id}` returns a single model by its canonical model ID. If the model is not found, the endpoint returns 404 Not Found with a problem detail indicating "model not found". If the model exists but is in `retired` status, it is returned with a 200 OK and an additional `retired: true` field in the response body, allowing clients to distinguish retired models from active ones.

POST `/api/v1/models/discover` triggers an on-demand discovery run across all three tiers. The endpoint returns 202 Accepted immediately with a JSON body containing a `discovery_id` UUID. The actual discovery executes asynchronously, and clients can poll GET `/api/v1/models/discover/{discovery_id}` for status updates. The status endpoint returns `pending`, `running`, `completed`, or `failed`, and on completion includes a summary of models discovered, updated, and removed during the run. This asynchronous pattern prevents long-running discovery operations from occupying HTTP connections and timing out at load balancer or client levels.

POST `/api/v1/models/refresh` forces a refresh of model metadata from provider APIs for all models currently in the registry with `status = 'verified'`. Unlike `discover`, which looks for new models, `refresh` updates existing records with the latest provider data. This endpoint is useful after provider maintenance windows or pricing changes. It also returns 202 Accepted and follows the same async polling pattern as discovery.

```
// pkg/api/handler.go
func (h *Handler) ListModels(w http.ResponseWriter, r *http.Request)    // GET /api/v1/models
func (h *Handler) GetModel(w http.ResponseWriter, r *http.Request)     // GET /api/v1/models/{id}
func (h *Handler) TriggerDiscovery(w http.ResponseWriter, r *http.Request) // POST /api/v1/models/
↳ discover
func (h *Handler) RefreshModels(w http.ResponseWriter, r *http.Request) // POST /api/v1/models/
↳ refresh
```

The handler implementation uses a shared `ModelRegistry` instance initialized at application startup and injected through the handler's constructor. This dependency injection pattern ensures that API handlers and background discovery processes operate on the same database connection pool and cache layer, maintaining consistency without explicit synchronization. Request parsing validates all query parameters and returns 400 Bad Request with detailed validation errors for invalid filter combinations (e.g., `min_score` above the maximum possible value of 10.0, or a `tier` value not in the allowed enumeration).

Response serialization uses the same JSON encoder configuration as the rest of the HelixTranslate API: indented output in development mode, compact output in production, and HTML-escaping disabled to properly render Unicode characters in model names. The `capabilities` and `pricing` JSON columns are unmarshaled into `map[string]interface{}` during serialization so they appear as nested objects in the response rather than escaped JSON strings. Database timestamps are formatted as RFC 3339 strings in the response body, with null timestamps for models that have never been verified or checked omitted entirely using `omitempty` struct tags.

The API layer implements request logging middleware that records method, path, query parameters, response status, and duration for every request. These logs are emitted at `INFO` level for 2xx responses, `WARN` level for 4xx responses, and `ERROR` level for 5xx responses. Aggregate metrics are exported via Prometheus counters `api_requests_total` and histograms `api_request_duration_seconds`, labeled by method, endpoint pattern, and status code. Together, these observability features provide operators with full visibility into discovery and registry API usage patterns and performance characteristics.

6. Phase 5: Provider Factory and Runtime Integration

The provider factory and runtime integration layer serves as the definitive bridge between the LLMsVerifier platform's verification pipeline and HelixTranslate's operational translation infrastructure. While the preceding phases establish the data models, scoring infrastructure, and observability hooks, Phase 5 addresses the critical challenge of transforming verified model data into actionable runtime decisions. This chapter examines the complete refactoring of the existing provider factory to incorporate verification-aware model selection, the construction of a multi-strategy selection engine, the design of resilient fallback and degradation chains, event bus integration for real-time model state propagation, full capability-based model matching across the HelixTranslate ecosystem, and the secure management of provider API keys for more than twenty-five distinct LLM providers.

The architectural premise of Phase 5 is that a verified model registry is only valuable if the runtime translation pipeline can efficiently query it, interpret its scores and capabilities, and act upon changes in model quality without manual operator intervention. The integration introduces a three-layer decision hierarchy: first, the `VerifiedLLMClient` abstraction wraps every provider-backed model with its verification metadata; second, the `SelectionEngine` applies domain-specific strategies to choose the optimal model for a given task; and third, the fallback chain builder ensures that translation requests survive individual model failures through a structured degradation path. Each layer is designed to be independently testable, horizontally scalable, and resilient to the inherently variable performance characteristics of third-party LLM APIs.

6.1 Verified Provider Factory Refactoring

The existing HelixTranslate provider factory, located at `internal/providers/factory.go`, instantiates LLM clients based on static configuration entries in `config/providers.yaml`. This approach treats every configured model as equally suitable for every translation task, with selection limited to round-robin or first-available logic. Phase 5 replaces this flat model with a hierarchical factory that consumes the `VerifiedModelRegistry` populated by the LLMsVerifier ingestion pipeline (Chapter 3) and produces `VerifiedLLMClient` instances enriched with composite scores, tier assignments, capability bitmaps, and verification timestamps.

6.1.1 The VerifiedLLMClient Type

The central abstraction introduced by the refactoring is the `VerifiedLLMClient` struct, which embeds the existing `llm.LLMClient` interface and augments it with verification metadata sourced from the model registry. This design preserves backward compatibility with all existing translation call sites while enabling verification-aware routing decisions at the factory boundary.

```
type VerifiedLLMClient struct {
    llm.LLMClient
    ModelID      string      `json:"model_id"`
    ProviderID   string      `json:"provider_id"`
    VerificationStatus VerificationStatus `json:"verification_status"`
    Score        *CompositeScore `json:"score"`
    Tier         ModelTier      `json:"tier"`
    Capabilities  CapabilityBitmap `json:"capabilities"`
    VerifiedAt   time.Time      `json:"verified_at"`
}
```

```

type VerificationStatus string
const (
    StatusPending VerificationStatus = "pending"
    StatusVerified VerificationStatus = "verified"
    StatusFailed   VerificationStatus = "failed"
    StatusRetired  VerificationStatus = "retired"
)

type ModelTier string
const (
    TierPremium   ModelTier = "premium"
    TierStandard  ModelTier = "standard"
    TierBudget    ModelTier = "budget"
    TierUnranked  ModelTier = "unranked"
)

```

The `VerifiedLLMClient` struct is defined in `internal/providers/verified_client.go`. The `llm.LLMClient` embedding ensures that the `VerifiedLLMClient` satisfies the same interface contract as all existing provider implementations, meaning that no changes are required in the core translation engine at `internal/translate/engine.go` or in any of the twenty-seven language-pair adapters. The `VerificationStatus` field captures the lifecycle state of the model as determined by the verifier's continuous evaluation loop: models entering the system begin in `StatusPending`, transition to `StatusVerified` upon passing the minimum score threshold (configurable via `verifier.min_score_threshold`, default `0.65`), move to `StatusFailed` if verification fails after the maximum retry count (default three attempts), and are marked `StatusRetired` if their score drops below the threshold after a prior successful verification. The `ModelTier` field classifies models into four economic tiers. `TierPremium` includes models scoring above `0.90` on the composite index, `TierStandard` covers scores `0.75` to `0.90`, `TierBudget` includes scores `0.65` to `0.75`, and `TierUnranked` captures all other models including those pending verification or those explicitly excluded from tiering. This tier system directly supports the degradation chain described in Section 6.3.

6.1.2 LLMsVerifier Provider Adapter

The factory must support `LLMsVerifier` as a first-class provider in addition to the existing direct-provider adapters (OpenAI, Anthropic, Google, Azure, Cohere, Mistral, and others). The `LLMVerifierAdapter` translates between HelixTranslate's `llm.TranslationRequest` type and the `LLMsVerifier` platform's OpenAI-compatible chat completions endpoint. The adapter resides at `internal/providers/llmsverifier/adapters/adapters.go` and implements the full `llm.LLMClient` interface.

```

// internal/providers/llmsverifier/adapters/adapters.go
type LLMVerifierAdapter struct {
    baseURL string
    apiKey  string
    modelID string
    client  *http.Client
}

func (a *LLMVerifierAdapter) Translate(ctx context.Context, req llm.TranslationRequest)
↳ (llm.TranslationResponse, error) {
    payload, _ := json.Marshal(CompletionRequest{
        Model:    a.modelID,

```



```

        Messages: convertMessages(req),
    })
    httpReq, _ := http.NewRequestWithContext(ctx, "POST",
        a.baseURL+"/v1/chat/completions", bytes.NewReader(payload))
    httpReq.Header.Set("Authorization", "Bearer "+a.apiKey)
    httpReq.Header.Set("Content-Type", "application/json")

    resp, err := a.client.Do(httpReq)
    if err != nil {
        return llm.TranslationResponse{}, fmt.Errorf("llmsverifier request failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        body, _ := io.ReadAll(resp.Body)
        return llm.TranslationResponse{}, fmt.Errorf("llmsverifier returned %d: %s", resp.StatusCode,
            ↪ body)
    }

    var result CompletionResponse
    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
        return llm.TranslationResponse{}, fmt.Errorf("decode response: %w", err)
    }
    if len(result.Choices) == 0 {
        return llm.TranslationResponse{}, errors.New("empty choices in response")
    }
    return llm.TranslationResponse{
        Text:      result.Choices[0].Message.Content,
        TokensUsed: result.Usage.TotalTokens,
    }, nil
}

```

The `convertMessages` helper transforms HelixTranslate’s internal message format into the LLMsVerifier chat completions schema, preserving system prompts, user instructions, and any in-context translation examples. The adapter also supports streaming responses via a separate `TranslateStream` method for real-time chat translation tasks, with the stream parser extracting delta chunks from the Server-Sent Events (SSE) response format. The `client` field uses a shared `http.Client` configured with a connection pool of up to one hundred connections and a default request timeout of sixty seconds (configurable via `verifier.request_timeout`). Retry logic with exponential backoff (initial delay one hundred milliseconds, maximum delay five seconds, jitter factor 0.3) is applied at the adapter level for transient errors (HTTP 429, 502, 503, 504).

6.1.3 Factory Integration Points

The refactored factory at `internal/providers/factory.go` introduces three new factory methods: `CreateVerifiedClient` accepts a model identifier and returns a `VerifiedLLMClient` populated from the registry; `CreateBestForTask` accepts a `TaskRequirements` descriptor and delegates to the selection engine (Section 6.2); and `CreateForTier` returns any available client within a specified tier, used primarily by the degradation chain. The factory maintains a hot cache of `VerifiedLLMClient` instances keyed by `model_id`, with entries refreshed every thirty seconds via a background goroutine that polls the `VerifiedModelRegistry`

for status changes. Cache invalidation is also triggered by `ModelScoreChangedEvent` notifications on the event bus (Section 6.4), ensuring that score drops or model retirements are reflected in the factory’s client pool within sub-second latency.

6.2 Runtime Model Selection Engine

The selection engine is the decision-making core of the runtime integration. Located at `internal/providers/selection/engine.go`, it maps incoming translation tasks to optimal model selections through a pluggable strategy pattern. Each strategy evaluates the set of available verified models against task-specific criteria and returns a ranked or single selection.

6.2.1 Strategy Architecture

The selection engine defines a `SelectionStrategy` interface with a single method `Select(ctx context.Context, candidates []VerifiedLLMClient, task TaskRequirements) (*VerifiedLLMClient, error)`. Six concrete strategy implementations are provided, each targeting a distinct optimization objective. The engine routes tasks to strategies based on the `Priority` field of the `TaskRequirements` descriptor.

Table 1: Task Type to Selection Strategy Mapping

Task Type	Priority	Strategy	Key Criteria
Legal translation	quality	BestScoreStrategy	Capability bitmap includes <code>CapLegalDomain</code> ; highest <code>CompositeScore.Overall</code>
Batch processing	cost	LowestCostStrategy	Lowest <code>CompositeScore.CostScore</code> within configured budget ceiling
Real-time chat	speed	FastestStrategy	Lowest measured latency (p95) from <code>CompositeScore.LatencyMs</code>
General text	balanced	BalancedStrategy	Weighted composite of quality, cost, and latency scores
Medical translation	quality	BestScoreStrategy	Capability bitmap includes <code>CapMedicalDomain</code> ; highest overall score
Technical documentation	quality	BestScoreStrategy	Capability bitmap includes <code>CapCodeGeneration</code> and <code>CapTechnicalWriting</code>

Additional task types not listed in Table 1 (e.g., literary translation, marketing localization, subtitle timing) follow the same routing logic, with domain-specific capability flags defined in the `CapabilityBitmap` enumeration. The `TaskRequirements` struct captures source and target languages, content domain, minimum quality threshold, maximum acceptable latency, and budget constraints, enabling the strategies to make multi-dimensional tradeoffs.

6.2.2 Selection Engine Implementation

The `SelectionEngine` maintains a registry of named strategies and a reference to the `ModelRegistry` for querying available models. Its `SelectModel` method implements the complete selection pipeline: filtering by minimum score, selecting the appropriate strategy, and returning the chosen model.

```
type SelectionEngine struct {
    strategies map[string]SelectionStrategy
    registry   *ModelRegistry
}

func (e *SelectionEngine) SelectModel(ctx context.Context, task TaskRequirements) (*VerifiedLLMClient,
    error) {
    available, err := e.registry.FilterByScore(ctx, task.MinQuality)
    if err != nil {
        return nil, fmt.Errorf("filter models: %w", err)
    }
    if len(available) == 0 {
        return nil, ErrNoModelsAvailable
    }

    strategy := e.strategies[task.Priority]
    if strategy == nil {
        strategy = e.strategies["balanced"]
    }
    return strategy.Select(ctx, available, task)
}
```

The `FilterByScore` method on `ModelRegistry` queries the underlying PostgreSQL-backed registry (Chapter 3) with the clause `WHERE overall_score >= $1 AND verification_status = 'verified'`, ensuring that only currently verified models meeting the task's minimum quality bar are considered. The query is indexed on a composite of `(verification_status, overall_score DESC)` for sub-millisecond lookup latency on the production dataset of approximately four hundred active models. If `task.Priority` does not match a registered strategy key, the engine defaults to `BalancedStrategy`, which computes a weighted sum of normalized quality, cost, and latency scores using weights configurable via `selection.balanced_weights` (default quality: `0.5`, cost: `0.3`, latency: `0.2`). The `BestScoreStrategy` selects the highest-scoring model that satisfies all required capabilities; in case of a tie, it prefers the model with the lower cost score. The `LowestCostStrategy` selects the cheapest verified model and falls back to the next tier up if no model meets the budget ceiling. The `FastestStrategy` selects the model with the lowest p95 latency measurement from the verifier's benchmarking history, filtering out any model with fewer than ten latency samples to ensure statistical significance.

6.2.3 Strategy Registration and Hot Swapping

Strategies are registered at engine initialization via a builder pattern in `internal/providers/selection/builder.go`. The factory supports hot-swapping of strategy weights at runtime through a configuration watcher on `config/selection.yaml`; changes to strategy parameters are applied without requiring a service restart. The engine also exposes a `SelectWithOverride` method that accepts an explicit strategy name, used by the HelixTranslate admin API for A/B testing and manual model selection experiments.

6.3 Fallback and Degradation Chain

Individual LLM API requests can fail for numerous reasons: rate limits, transient provider outages, content policy rejections, context window overflows, or unexpected response formats. The fallback and degradation chain provides a structured, deterministic path for recovering from these failures while preserving translation quality to the maximum extent possible.

6.3.1 Fallback Chain Construction

The `BuildFallbackChain` function constructs an ordered list of fallback candidates from the set of available verified models. The chain is built once per selection decision and cached for the duration of the translation request.

```
func BuildFallbackChain(models []VerifiedLLMClient, maxLength int) []FallbackEntry {
    sort.Slice(models, func(i, j int) bool {
        return models[i].Score.Overall > models[j].Score.Overall
    })

    var chain []FallbackEntry
    lastProvider := ""
    for _, m := range models {
        if len(chain) >= maxLength {
            break
        }
        if m.ProviderID == lastProvider && len(chain) > 0 {
            continue // enforce provider diversity
        }
        chain = append(chain, FallbackEntry{
            ModelID:    m.ModelID,
            ProviderID: m.ProviderID,
            Tier:        m.Tier,
            Score:       m.Score.Overall,
        })
        lastProvider = m.ProviderID
    }
    return chain
}
```

The function sorts all verified models by descending overall score and then iterates to build a chain of up to `maxLength` entries (default five, configurable via `fallback.max_chain_length`). The provider diversity constraint ensures that consecutive entries in the chain originate from different infrastructure providers, mitigating the risk of a single provider outage cascading through all fallback attempts. The `FallbackEntry` struct contains only the fields necessary for retry dispatch: `ModelID`, `ProviderID`, `Tier`, and `Score`. The full `VerifiedLLMClient` is retrieved from the factory cache when a fallback entry is activated.

6.3.2 Degradation Policy

The degradation policy defines five distinct levels that a translation request progresses through on repeated failure. Each level specifies a different recovery action, and the policy is applied by the `DegradationController` at `internal/providers/fallback/degradation.go`.

```
type DegradationLevel int
```

```

const (
    LevelPrimary      DegradationLevel = iota // Best match from selection engine
    LevelRetry        // Retry same model once (handles transient errors)
    LevelFallback      // Next entry in fallback chain
    LevelTierDown      // Drop to next tier (e.g., Premium -> Standard)
    LevelHumanReview   // Queue for human translator review
)

```

At `LevelPrimary`, the request is sent to the model selected by the selection engine. If this fails with a retryable error (HTTP 429, 502, 503, 504, timeout, or connection reset), the controller advances to `LevelRetry` and reissues the identical request to the same model endpoint. The retry includes exponential backoff with jitter. If the retry also fails, the controller advances to `LevelFallback` and dispatches the request to the next entry in the pre-built fallback chain. If all fallback chain entries are exhausted, the controller advances to `LevelTierDown`, which queries the factory for any available model in the next lower tier. For example, if the original selection was `TierPremium`, `LevelTierDown` attempts `TierStandard`, then `TierBudget`, then `TierUnranked`. At each tier, the best-scoring model in that tier is selected using the same strategy as the original request. If no model in any tier succeeds, the request reaches `LevelHumanReview`, where it is enqueued in the human review pipeline at `internal/humanreview/queue.go`. The queue stores the original request payload, the complete error history from all degradation levels, and a priority score based on the requesting user's service tier.

6.3.3 Degradation Metrics and Circuit Breakers

The degradation controller emits Prometheus metrics at every level transition: `helix_degradation_level_total{level="pr`, `helix_fallback_latency_seconds`, and `helix_human_review_queue_size`. Additionally, a per-provider circuit breaker (implemented using the `sony/gobreaker` library) tracks failure rates over a thirty-second window. If a provider's failure rate exceeds fifty percent, the circuit opens and all models from that provider are temporarily excluded from the fallback chain. The circuit closes after a ten-second half-open period during which a single probe request is permitted. This prevents the degradation chain from repeatedly attempting a clearly degraded provider.

6.4 Event Bus Integration

The event bus integration connects the `LLMsVerifier` platform's score change notifications to `HelixTranslate`'s runtime systems, enabling reactive model management. Two primary event types are handled: `ModelScoreChangedEvent`, which triggers model retirement or re-tiering, and `ModelSelectedEvent`, which is published by the selection engine for observability and billing purposes.

6.4.1 Score Change Event Handler

The `handleModelScoreChanged` method on the `VerifierIntegration` type processes score change events from the `LLMsVerifier` platform. When a model's composite score drops below the minimum threshold, the handler automatically retires the model from the active registry.

```

func (v *VerifierIntegration) handleModelScoreChanged(ctx context.Context, evt
↪ events.ModelScoreChangedEvent) error {
    v.logger.Printf("Score change: model=%s old=%.2f new=%.2f", evt.ModelID, evt.OldScore, evt.NewScore)

    if evt.NewScore < v.config.MinScoreThreshold {
        v.logger.Printf("Model %s score dropped to %.2f, retiring", evt.ModelID, evt.NewScore)
    }
}

```

```

    if err := v.registry.UpdateModelStatus(ctx, evt.ModelID, "retired"); err != nil {
        return fmt.Errorf("retire model %s: %w", evt.ModelID, err)
    }
    // Invalidate factory cache for this model
    v.factory.InvalidateCache(evt.ModelID)
    // Notify operators via PagerDuty if the model was TierPremium
    if evt.OldTier == TierPremium {
        v.alerting.Send(ctx, alerting.Alert{
            Severity: alerting.SeverityWarning,
            Summary:  fmt.Sprintf("Premium model %s retired (score %.2f)", evt.ModelID, evt.NewScore),
        })
    }
    return nil
}

// Re-tier the model if score moved across tier boundaries
newTier := scoreToTier(evt.NewScore)
if newTier != evt.OldTier {
    if err := v.registry.UpdateModelTier(ctx, evt.ModelID, newTier); err != nil {
        return fmt.Errorf("update tier for %s: %w", evt.ModelID, err)
    }
    v.factory.InvalidateCache(evt.ModelID)
}
return nil
}

```

The handler also detects tier boundary crossings: if a model's score improvement promotes it from `TierStandard` to `TierPremium`, the registry and factory cache are updated immediately, making the model available for high-priority tasks. Conversely, a demotion triggers cache invalidation and removes the model from premium-eligible selection pools. Score increases that do not cross tier boundaries are logged at debug level but require no action. The `events.ModelScoreChangedEvent` payload includes `ModelID`, `OldScore`, `NewScore`, `OldTier`, `NewTier`, and `Timestamp`, all sourced from the `LLMsVerifier` scoring pipeline's output (Chapter 4).

6.4.2 Model Selection Event Publisher

When the selection engine chooses a model for a translation task, it publishes a `ModelSelectedEvent` to the event bus. This event is consumed by the billing service, the analytics pipeline, and the real-time dashboard.

```

func (v *VerifierIntegration) PublishModelSelected(ctx context.Context, modelID string, task
↳ TaskRequirements) {
    events.Publish(ctx, events.ModelSelectedEvent{
        ModelID:    modelID,
        TaskType:    task.Domain,
        SourceLang:  task.SourceLang,
        TargetLang:  task.TargetLang,
        Timestamp:   time.Now(),
    })
}

```

The `events.Publish` function is backed by Redis Streams (`XADD` to `helix:events:model_selected`) with at-least-once delivery semantics. Consumers include: the billing aggregator, which credits the selected model’s provider for the request; the analytics pipeline, which feeds into the BigQuery warehouse for usage pattern analysis; and the WebSocket-based real-time dashboard, which displays the currently active model per language pair. The event schema is versioned (current version 1.2) and includes an `EventID` UUID for idempotency.

6.4.3 Event Subscription Topology

The verifier integration subscribes to three Redis Stream channels: `llmverifier:scores` for score changes, `llmverifier:capabilities` for capability bitmap updates (triggered when new MCP integrations are tested), and `llmverifier:retirements` for explicit retirement commands from the verifier platform’s administrative interface. Subscription handlers run in a dedicated goroutine pool of eight workers, with each handler wrapped in a panic recovery wrapper and a timeout context of thirty seconds. Failed event processing results in the event being re-queued with an exponential backoff, and events failing after five retries are written to a dead-letter stream for manual inspection.

6.5 Full Capability Integration

HelixTranslate’s translation tasks frequently require models with specific capabilities beyond general text generation. Legal translation demands models with structured output and legal domain knowledge; technical documentation requires code-aware models; and RAG-enhanced translation needs models with large context windows. The capability integration layer ensures that the selection engine can filter models by these specialized requirements.

6.5.1 MCP Integration Catalog

The Model Context Protocol (MCP) integrations extend model capabilities by providing structured tool access. The LLMsVerifier platform tests each model’s ability to invoke MCP tools correctly, and the results are encoded in the model’s `CapabilityBitmap`. HelixTranslate leverages these verified capabilities to match models to tasks requiring external tool integration.

Table 2: Key MCP Integrations and Required Capabilities

MCP Name	Purpose	Required Capability	File Path
GitHub	Code repository access and file retrieval	CapFunctionCalling	internal/mcp/github/
Playwright	Browser automation for web content extraction	CapFunctionCalling	internal/mcp/playwright/
Qdrant	Vector search for semantic document retrieval	CapLargeContext	internal/mcp/qdrant/
Redis	Distributed cache and RAG backend storage	CapLargeContext	internal/mcp/redis/

Table 19 – continued

MCP Name	Purpose	Required Capability	File Path
Elasticsearch	Full-text search and RAG document indexing	CapLargeContext	internal/mcp/elastic/
Cloudflare	AI Gateway request routing and caching	CapStreaming	internal/mcp/cloudflare/
AWS	Cloud service integration (S3, Lambda, Bedrock)	CapFunctionCalling	internal/mcp/aws/
Kubernetes	Container management and log extraction	CapFunctionCalling	internal/mcp/k8s/

The full MCP catalog contains thirty-five integrations. The remaining twenty-seven include: `internal/mcp/jira/` (issue tracking), `internal/mcp/slack/` (messaging), `internal/mcp/notion/` (documentation), `internal/mcp/figma/` (design asset access), `internal/mcp/trello/` (project management), `internal/mcp/asana/` (task management), `internal/mcp/confluence/` (wiki content), `internal/mcp/dropbox/` (file storage), `internal/mcp/google_drive/` (cloud storage), `internal/mcp/onedrive/` (Microsoft storage), `internal/mcp/sharepoint/` (enterprise content), `internal/mcp/zendesk/` (support tickets), `internal/mcp/intercom/` (customer messaging), `internal/mcp/hubspot/` (CRM data), `internal/mcp/salesforce/` (CRM integration), `internal/mcp/stripe/` (payment data), `internal/mcp/twilio/` (communications), `internal/mcp/sendgrid/` (email delivery), `internal/mcp/shopify/` (e-commerce data), `internal/mcp/woocommerce/` (e-commerce), `internal/mcp/bigcommerce/` (e-commerce), `internal/mcp/square/` (point-of-sale), `internal/mcp/paypal/` (payment processing), `internal/mcp/mongodb/` (document database), `internal/mcp/postgresql/` (relational database), `internal/mcp/mysql/` (relational database), and `internal/mcp/sqlite/` (embedded database). Each MCP integration defines its own `CapabilityRequirements` and validates that the selected model’s bitmap includes the necessary flags.

6.5.2 Complete Capability Taxonomy

The full capability taxonomy encompasses seven categories covering all verified model abilities relevant to HelixTranslate’s operations.

Table 3: Complete Capability Taxonomy

Category	Count	Key Examples
MCP (Model Context Protocol)	35	GitHub, Playwright, Qdrant, Redis, ES, Cloudflare, AWS, K8s, Jira, Slack, Notion
LSP (Language Server Protocol)	10	Go, Python, TypeScript, Rust, C++, Java, Ruby, PHP, Swift, Kotlin
ACP (Agent Communication Protocol)	1	Inter-agent message passing and delegation

Table 20 – continued

Category	Count	Key Examples
Embeddings	13	OpenAI (<code>text-embedding-3-*</code>), Cohere (<code>embed-*</code>), Jina (<code>jina-embeddings-*</code>), Mistral, BGE, E5, GTE, Voyage, Nomic, Google (<code>textembedding-gecko-*</code>), Amazon Titan, IBM Granite, Snowflake Arctic
RAG (Retrieval-Augmented Generation)	3	Qdrant vector search, Redis semantic cache, Elasticsearch hybrid search
Skills	15+	Translation (all 27 language pairs), Summarization, Code Review, Technical Writing, Legal Analysis, Medical Terminology, Literary Style Transfer, Marketing Localization, Subtitle Timing, Voice-to-Text Alignment, Glossary Enforcement, Style Guide Compliance, Back-translation Verification, Post-editing Support, Quality Estimation
Plugins	8+	Markdown formatter, HTML/XML entity normalizer, Unicode bidirectional text handler, ICU message format converter, XLIFF segment processor, TM (translation memory) matcher, Terminology database connector, Custom preprocessing pipeline

The LSP category enables language-aware translation of code comments, docstrings, and inline documentation. When a task is tagged with domain `code`, the selection engine requires `CapCodeGeneration` (which implies all LSP capabilities) and routes the request to models verified for the specific programming language of the input. The embeddings category supports the RAG pipeline by ensuring that the embedding model used for document indexing is compatible with the generation model’s context window and token format. The skills category represents the highest-level capability mapping, directly corresponding to HelixTranslate’s service offerings; each skill is verified by the LLMsVerifier platform through targeted benchmark datasets.

6.5.3 Capability Matching Engine

The capability matching engine provides a unified interface for checking whether a model satisfies the requirements of a given task.

```

type CapabilityRequirements struct {
    Required      CapabilityBitmap
    Preferred     CapabilityBitmap
    MinContextWindow int
}

func MatchCapabilities(req CapabilityRequirements, modelCaps CapabilityBitmap, contextWindow int)
↳ MatchResult {
    if contextWindow < req.MinContextWindow {

```

```

    return MatchResult{
        Satisfied: false,
        Reason:    fmt.Sprintf("context window %d < required %d", contextWindow,
            ↪ req.MinContextWindow),
    }
}

missing := req.Required &^ modelCaps
if missing != 0 {
    return MatchResult{
        Satisfied: false,
        Reason:    fmt.Sprintf("missing required capabilities: %b", missing),
    }
}

preferred := req.Preferred & modelCaps
return MatchResult{
    Satisfied: true,
    Preferred: preferred,
    PreferredCount: bits.OnesCount64(preferred),
}
}

```

The `CapabilityBitmap` type is a `uint64` bitfield with fifty-six defined capability flags, leaving eight bits reserved for future expansion. The `MatchCapabilities` function first validates the context window requirement (used to exclude models with insufficient token capacity for long-document translation or RAG contexts), then checks that all required capability bits are present in the model's bitmap. Models that fail the required capability check are filtered out before strategy evaluation. Preferred capabilities contribute to the strategy's scoring function: a model that satisfies more preferred capabilities receives a bonus in the `BalancedStrategy` weight calculation, and `BestScoreStrategy` uses preferred capability count as a tiebreaker when overall scores are within `0.01` of each other.

The `CapabilityBitmap` is defined in `internal/providers/capabilities/bitmap.go` with individual constants such as `CapFunctionCalling = 1 << 0`, `CapLargeContext = 1 << 1`, `CapStreaming = 1 << 2`, `CapCodeGeneration = 1 << 3`, `CapLegalDomain = 1 << 4`, `CapMedicalDomain = 1 << 5`, `CapTechnicalWriting = 1 << 6`, and so on through the fifty-six defined flags. Capability bitmaps are stored in the `verified_models` table as `BIGINT` columns and are updated by the `LLMsVerifier` ingestion pipeline whenever a new capability test is completed.

6.6 API Key Management

The integration with `LLMsVerifier` and the expanded provider ecosystem requires secure management of more than twenty-five distinct API keys. The key management system, implemented in `internal/providers/keys/manager.go`, provides validation, encrypted storage, rotation, and per-key rate limit tracking.

6.6.1 Key Validation

Every API key is validated at configuration load time using a two-step process: first, a provider-specific regex validates the key format (e.g., OpenAI keys match `^sk-[a-zA-Z0-9]{48}$`, Anthropic keys match `^sk-ant-api03-[a-zA-Z0-9_-]{64,}$`); second, a minimal API call to a low-cost endpoint (typically a models listing or a short completion with `max_tokens: 1`) confirms that the key is active and has remaining

quota. Keys that fail either step are flagged with `status: invalid` and excluded from the provider factory's client pool.

The validation functions are organized by provider in `internal/providers/keys/validators/`. Each validator implements the `KeyValidator` interface: `ValidateFormat(key string) error` and `ValidateActive(ctx context.Context, key string) error`. The active validation calls use a shared circuit breaker per provider to prevent validation storms during provider outages.

6.6.2 Secure Storage

All API keys are encrypted at rest using AES-256-GCM with a key encryption key (KEK) stored in HashiCorp Vault. The data encryption key (DEK) is unique per provider and is rotated every thirty days via an automated cron job at `cmd/key-rotation/main.go`. The encrypted key material is stored in the PostgreSQL `provider_api_keys` table with the following schema: `id SERIAL PRIMARY KEY, provider_id VARCHAR(64) NOT NULL, key_ciphertext BYTEA NOT NULL, key_nonce BYTEA NOT NULL, status VARCHAR(16) DEFAULT 'active', rate_limit_rpm INTEGER, rate_limit_tpm INTEGER, created_at TIMESTAMPTZ, last_rotated_at TIMESTAMPTZ, and expires_at TIMESTAMPTZ`. At runtime, keys are decrypted on-demand and cached in process memory with a ten-minute TTL; the in-memory cache stores only the decrypted key string, protected by Go's garbage collection semantics and process-level isolation.

The twenty-five supported providers and their validation endpoints are documented in `config/providers.yaml`. Each entry specifies the provider ID, base URL, validation endpoint path, validation method, expected success status code, and rate limits. The providers include: OpenAI (`api.openai.com`), Anthropic (`api.anthropic.com`), Google Vertex AI (`us-central1-aiplatform.googleapis.com`), Azure OpenAI (`{resource}.openai.azure.com`), Cohere (`api.cohere.com`), Mistral AI (`api.mistral.ai`), LLMsVerifier (`api.llmsverifier.io`), Together AI (`api.together.xyz`), Groq (`api.groq.com`), Perplexity (`api.perplexity.ai`), AI21 Labs (`api.ai21.com`), Fireworks AI (`api.fireworks.ai`), Anyscale (`api.endpoints.anyscale.com`), DeepInfra (`api.deepinfra.com`), Baseten (`model-{model_id}.api.baseten.co`), Replicate (`api.replicate.com`), Hugging Face (`api-inference.huggingface.co`), OctoAI (`text.octoai.run`), Lepton AI (`{model}.lepton.run`), Cloudflare Workers AI (`api.cloudflare.com`), AWS Bedrock (`bedrock-runtime.{region}.amazonaws.com`), Google Gemini (`generativelanguage.googleapis.com`), Moonshot AI (`api.moonshot.cn`), Zhipu AI (`open.bigmodel.cn`), and 01.AI (`api.01.ai`).

6.6.3 Per-Key Rate Limiting

The key manager maintains a distributed rate limiter per API key using Redis as the coordination backend. The limiter tracks both requests-per-minute (RPM) and tokens-per-minute (TPM) quotas as reported by each provider's response headers (`x-ratelimit-limit-requests`, `x-ratelimit-limit-tokens`, `x-ratelimit-remaining-requests`, `x-ratelimit-remaining-tokens`). When a key's rate limit is exhausted, the manager marks it as `status: rate_limited` for the duration specified in the `Retry-After` header (or a default sixty seconds if absent), and the provider factory routes requests to the next available key for that provider. This multi-key support is essential for high-volume translation pipelines that can exceed the quota of a single API key.

6.6.4 Key Rotation and Audit

Key rotation is triggered automatically when an expiry date approaches (within seven days) or manually via the admin API at `POST /admin/keys/{provider_id}/rotate`. The rotation process generates a new key through the provider's API console (for providers supporting programmatic key generation) or accepts

a new key via the admin API, validates it, encrypts it, and atomically swaps the active key reference. Old keys are retained in `status: rotated` for twenty-four hours to allow in-flight requests to complete, then are permanently deleted. All key operations (validation, rotation, status changes, access events) are logged to an append-only audit log stored in S3 with object lock enabled, retaining records for a minimum of one year for compliance purposes.

Phase 5 completes the integration by binding the verified model registry to every operational decision in HelixTranslate's translation pipeline. The refactored provider factory produces `VerifiedLLMClient` instances with full metadata, the selection engine routes tasks through domain-appropriate strategies, the degradation chain ensures resilience against individual model failures, the event bus propagates score changes in real time, the capability matcher enables sophisticated task-to-model assignment across seventy-plus verified abilities, and the key management system securely orchestrates access to more than twenty-five providers. Together, these components transform the LLMsVerifier platform from a passive scoring system into an active participant in HelixTranslate's runtime quality assurance.

7. Phase 6: Enterprise UX and User-Facing Features

Phase 6 represents the culmination of the LLMsVerifier integration effort—the layer where the underlying scoring, routing, and verification infrastructure becomes tangible and accessible to end users. Where Phases 1 through 5 established the foundational scoring engine, provider management, routing intelligence, quality assurance mechanisms, and CI/CD integration, Phase 6 channels all of that computational work into polished, enterprise-grade user experiences. The guiding principle of this phase is **transparency through instrumentation**: every model score, routing decision, quality metric, and fallback event is surfaced to the user in a way that builds trust and enables informed decision-making. This chapter covers four interconnected workstreams: the model selection user experience with rich score visualization, a real-time model status dashboard powered by WebSocket streaming, a batch translation system with multi-model orchestration, and a closed-loop translation quality feedback mechanism that continuously refines model capability scores based on real user interactions.

The architecture of Phase 6 is designed to serve two distinct personas simultaneously. **End users** (translators, localization engineers, and enterprise customers) need intuitive interfaces that communicate which model is being used, why it was selected, and how well it performed. **Platform operators** (SRE teams, model governance officers, and product managers) need dashboards that expose system health, model degradation patterns, routing efficiency metrics, and quality trend analysis. The components described in this chapter satisfy both audiences without compromising the clarity needed by either. All UI components are built in TypeScript/React for the web interface and Go for backend orchestration, maintaining consistency with the HelixTranslate technology stack established in prior phases.

7.1 Model Selection User Experience

The model selection interface is the primary touchpoint where users interact with the LLMsVerifier scoring system. It transforms abstract numerical scores into actionable visual information, enabling users to choose translation models with confidence. The design follows a card-based metaphor where each available model is represented as a self-contained card containing all information necessary for an informed selection decision. The card layout is hierarchical: provider identity and model name occupy

the header row, the overall verification score is rendered as a prominent color-coded badge, capability tags communicate domain expertise, and bottom-row metrics convey operational characteristics such as cost tier and speed classification.

7.1.1 Score Badge Visualization

The score badge is the most visually dominant element of the model card because it communicates the single most important piece of information: the model’s verified quality score on a 0–10 scale. The color coding scheme follows a traffic-light metaphor with five distinct tiers. Scores of 9.0 and above render in green (#22C55E), indicating exceptional quality suitable for the most demanding translation tasks such as legal contract localization and medical documentation. Scores between 7.5 and 8.9 render in blue (#3B82F6), denoting strong performance appropriate for professional translation workflows. The amber tier (#EAB308, scores 6.0–7.4) signals acceptable quality for general-purpose content where cost efficiency may outweigh absolute accuracy. Scores between 4.0 and 5.9 render in orange (#F97316), flagging models that should be used cautiously and only for low-stakes content or as fallback options. Any model scoring below 4.0 displays in red (#EF4444), serving as a visual warning that the model has failed verification criteria and should not be used for production translation work. The numeric value is always displayed with one decimal place of precision (e.g., “8.3” rather than “8”), reflecting the granularity of the underlying scoring engine.

7.1.2 Capability Tags and Metadata

Beneath the score badge, each model card renders a horizontal row of capability chips that communicate the model’s verified domain competencies. These tags are derived directly from the `capabilities` field of the `VerifiedModel` record populated by the Phase 2 scoring engine. Common tags include `CapDomainLegal`, `CapDomainMedical`, `CapDomainTechnical`, `CapDomainCreative`, and `CapGeneral`. Each tag is rendered as a small, rounded pill-shaped element with a consistent color scheme that corresponds to the domain family. The provider icon (e.g., OpenAI’s spiral, Anthropic’s constellation, or Google’s “G” mark) appears in the header row alongside the model name, providing immediate brand recognition and reinforcing the trust relationship between the user and the model vendor.

7.1.3 Filtering and Sorting Interface

The model selection panel includes a comprehensive filtering system that allows users to narrow the model list based on operational requirements. The provider filter supports multi-select, enabling users to include or exclude specific vendors (for example, excluding all models from a provider undergoing a known outage). The score range slider allows users to set minimum and maximum score thresholds, which is particularly useful when searching for models within a specific quality band. Capability toggles function as a set of checkboxes where users can require that displayed models possess specific domain competencies. The cost tier selector offers four tiers—**Free**, **Standard**, **Premium**, and **Enterprise**—allowing budget-conscious users to filter by economic constraints.

Sorting options include four dimensions, each supporting ascending and descending order. **Score** (descending) is the default sort, presenting the highest-quality models first. **Speed** sorting orders models by their latency score component, prioritizing low-latency options for time-sensitive workflows. **Cost** sorting arranges models from least to most expensive, aiding budget optimization. **Recency** sorting surfaces newly verified models, giving users early access to the latest options. Each sort dimension updates the card grid in real time without requiring a page refresh.

The `ModelCard` component implementation encapsulates all of these concerns into a single reusable React

component, as shown in Code Block 1.

Code Block 1: ModelCard component (TypeScript/React)

```
interface ModelCardProps {
  model: VerifiedModel;
  onSelect: (modelId: string) => void;
  isSelected: boolean;
}

const ModelCard: React.FC<ModelCardProps> = ({ model, onSelect, isSelected }) => {
  const scoreColor = model.score.overall >= 9 ? '#22C55E' :
    model.score.overall >= 7.5 ? '#3B82F6' :
    model.score.overall >= 6 ? '#EAB308' :
    model.score.overall >= 4 ? '#F97316' : '#EF4444';

  return (
    <div className={`model-card ${isSelected ? 'selected' : ''}`} onClick={() =>
      ↪ onSelect(model.modelId)}>
      <div className="model-header">
        <ProviderIcon provider={model.providerId} />
        <span className="model-name">{model.name}</span>
        <span className="score-badge" style={{ backgroundColor: scoreColor }}>
          {model.score.overall.toFixed(1)}
        </span>
      </div>
      <div className="model-capabilities">
        {model.capabilities.map(cap => (
          <CapabilityTag key={cap} name={cap} />
        ))}
      </div>
      <div className="model-metrics">
        <CostIndicator tier={model.tier} />
        <SpeedIndicator latency={model.score.components.speed} />
        <AvailabilityStatus status={model.status} />
      </div>
    </div>
  );
};
```

The component receives a `VerifiedModel` object (populated from the Phase 2 registry), a selection callback, and a boolean flag indicating selection state. The `scoreColor` computation applies the five-tier color mapping directly, ensuring visual consistency across the entire interface. The `CapabilityTag` sub-component renders each capability as a styled chip, while `CostIndicator`, `SpeedIndicator`, and `AvailabilityStatus` provide the bottom-row metrics. CSS classes `model-card` and `selected` support hover states, focus rings, and visual distinction for the currently selected model. The click handler propagates the `modelId` upward, triggering the routing logic described in Phase 3.

7.2 Real-Time Model Status Dashboard

Enterprise translation workflows demand real-time visibility into model health. A model that scored 8.5 during the morning verification cycle may degrade to 6.2 by afternoon due to provider-side issues, prompt template drift, or rate-limiting behavior changes. The real-time model status dashboard addresses this operational need by establishing a persistent WebSocket connection between the client browser and the HelixTranslate backend, streaming model status events as they occur. This architecture eliminates the need for periodic polling, reduces server load, and ensures that users see status changes within milliseconds of detection.

7.2.1 WebSocket Event Architecture

The WebSocket endpoint `wss://api.helixtranslate.com/v1/models/stream` emits typed events that conform to the `ModelStatusEvent` interface. Four event types cover the full lifecycle of model state changes. The `score_changed` event fires whenever a model's composite score shifts by more than 0.2 points, carrying both the previous and current scores so that the UI can animate transitions and flag significant changes. The `verification_failed` event indicates that a model has failed a scheduled verification challenge, including a human-readable `reason` field (e.g., "hallucination in legal terminology test" or "excessive latency on 4K token requests"). The `model_discovered` event announces the registration of a newly verified model, prompting the dashboard to add a new card with a "New" badge. The `model_retired` event signals that a model has been removed from the active pool, typically because its provider has deprecated the endpoint or the model has failed three consecutive verification cycles.

7.2.2 Status Indicator Semantics

Each model card on the dashboard displays a status indicator that aggregates the current score into a high-level health classification. A **green** indicator means the model's score is at or above 7.0, representing healthy operation suitable for all supported translation domains. A **yellow** indicator corresponds to scores between 5.5 and 6.9, signaling degraded performance—usable for general content but requiring monitoring and potentially triggering automatic fallback for high-stakes domains. A **red** indicator indicates a score below 5.5, meaning the model has entered a failing state and should not be used for any production translation until the underlying issue is resolved. A **gray** indicator means the model is currently unavailable (provider downtime, rate limit exhaustion, or maintenance mode), rendering it non-selectable but still visible for transparency.

7.2.3 Degraded Mode Warnings

When a model transitions into yellow or red status, the dashboard emits a non-blocking toast notification to alert the user. If the user has an active translation session using a model that subsequently degrades, the system displays a prominent banner offering one-click migration to the next-best available model. This automatic fallback notification includes the name of the recommended replacement model, its current score, and the estimated latency delta compared to the original selection. The notification respects user preferences: users can opt to remain on the degraded model, auto-switch to the recommended alternative, or configure automatic migration based on score thresholds (e.g., "always switch when score drops below 6.5").

The WebSocket client hook shown in Code Block 2 encapsulates the connection lifecycle and event dispatch logic.

Code Block 2: WebSocket events

```

interface ModelStatusEvent {
  type: 'score_changed' | 'verification_failed' | 'model_discovered' | 'model_retired';
  modelId: string;
  payload: {
    previousScore?: number;
    currentScore?: number;
    reason?: string;
  };
  timestamp: Date;
}

const useModelStatus = () => {
  const [models, setModels] = useState<VerifiedModel[]>([]);

  useEffect(() => {
    const ws = new WebSocket('wss://api.helixtranslate.com/v1/models/stream');
    ws.onmessage = (event) => {
      const evt: ModelStatusEvent = JSON.parse(event.data);
      handleStatusChange(evt, setModels);
    };
    return () => ws.close();
  }, []);

  return models;
};

```

The `handleStatusChange` function (not shown inline for brevity) implements an optimistic update strategy: it applies the event payload to the local model state immediately upon receipt, then schedules a background reconciliation request to fetch the authoritative state from the scoring engine. This approach ensures that the UI feels instantaneous while maintaining eventual consistency with the backend. The WebSocket connection automatically reconnects with exponential backoff (starting at 1 second, capping at 30 seconds) if the connection drops, ensuring resilience against transient network failures.

7.3 Batch Translation with Multi-Model Orchestration

Enterprise translation workflows frequently involve processing large documents that span multiple content domains. A single pharmaceutical regulatory submission, for example, may contain chemical nomenclature (technical domain), patient safety warnings (medical domain), and legal disclaimers (legal domain). Processing such a document through a single general-purpose model produces suboptimal results because no single model excels across all domain categories. The batch translation system described in this section addresses this challenge through domain-aware chunking and intelligent model routing, assigning each content segment to the model best suited for its specific domain.

7.3.1 Domain-Aware Document Chunking

The batch translation pipeline begins by decomposing the input document into `TranslationChunk` objects. Each chunk is classified into one of five domain categories: `legal`, `medical`, `technical`, `creative`, or `general`. Classification employs a multi-modal detection strategy that combines heuristic pattern

matching with a lightweight machine learning classifier. Legal content is identified through regex patterns that match legalese signatures such as “hereinafter,” “pursuant to,” “witnesseth,” and standard legal clause numbering formats (§1.2(a)). Medical content is detected by cross-referencing terminology against a curated medical terminology database containing 47,000 terms spanning ICD-10 codes, drug names, anatomical references, and clinical procedure identifiers. Technical content is identified by the presence of code blocks, structured data formats (JSON, XML, YAML), and high concentrations of domain-specific acronyms. Creative content is recognized through narrative pattern analysis—detection of dialogue formatting, descriptive prose density, and literary device markers. Content that fails all specialized domain tests, or falls below a confidence threshold, is classified as **general**.

7.3.2 Intelligent Chunk Routing

Once chunks are classified, the routing engine assigns each chunk to the most appropriate available model. The routing decision considers three factors in priority order: (1) domain capability match, (2) current model score, and (3) cost efficiency. A chunk classified as **legal** is routed to the highest-scoring model that possesses the `CapDomainLegal` capability. If no legal-specialized model is available (for example, due to provider outage), the system falls back to the highest-scoring general-capable model. This fallback behavior ensures continuity of service while transparently degrading quality expectations. The routing strategy is abstracted behind the `RoutingStrategy` interface, enabling experimentation with different allocation algorithms (greedy, round-robin, load-balanced, cost-optimized) without modifying the core orchestration logic.

The Go data structures in Code Block 3 define the complete domain model for batch translation jobs.

Code Block 3: BatchTranslationJob

```
type BatchTranslationJob struct {
    ID          string
    SourceLang  string
    TargetLang  string
    Chunks      []TranslationChunk
    Assignments []ModelAssignment
    Strategy    RoutingStrategy
    Status      JobStatus
}

type TranslationChunk struct {
    ID        string
    Content   string
    WordCount int
    Domain    string // detected via classifier
}

type ModelAssignment struct {
    ChunkID  string
    ModelID  string
    Status   AssignmentStatus
    Result   *TranslationResult
}

type RoutingStrategy interface {
```

```
AssignChunks(chunks []TranslationChunk, models []VerifiedLLMClient) []ModelAssignment
}
```

The `BatchTranslationJob` struct serves as the aggregate root, tracking the job’s lifecycle from `Pending` through `Chunking`, `Routing`, `Translating`, `Aggregating`, and finally `Completed` or `Failed`. The `TranslationChunk` struct carries the domain classification result alongside the raw content and word count, which feeds into cost estimation and progress reporting. Each `ModelAssignment` records which model was assigned to a specific chunk, the current status of that assignment (`Pending`, `InProgress`, `Completed`, `Failed`), and a pointer to the translation result once available. The `RoutingStrategy` interface enables pluggable routing algorithms, with a default `CapabilityAwareRouter` that implements the domain-match-with-fallback logic described above.

7.3.3 Chunk Routing Strategy Reference

The routing matrix in Table 1 documents the complete decision logic for chunk-to-model assignment. Each content type maps to a detection method, a preferred capability requirement, and a fallback capability for degraded-mode operation.

Table 1: Chunk routing strategy

Content Type	Detection Method	Preferred Capability	Fallback
Legal	Regex (legalese patterns)	CapDomainLegal	CapGeneral
Medical	Medical terminology DB	CapDomainMedical	CapGeneral
Technical	Code blocks, acronyms	CapDomainTechnical	CapGeneral
Creative	Narrative patterns	CapDomainCreative	CapGeneral
General	Default / short text	Balanced score	Any available

The `Balanced score` entry for general content indicates that the routing engine selects the model with the highest composite score across all capability components, rather than requiring a specific domain tag. This ensures that general content benefits from the best available model even when no specialized model exists. The fallback chain always terminates at `CapGeneral` or `Any available`, guaranteeing that every chunk receives a translation assignment as long as at least one model is operational.

7.4 Translation Quality Feedback Loop

The most reliable measure of a translation model’s quality is the judgment of the humans who consume its output. The translation quality feedback loop closes the circuit between automated scoring and subjective human evaluation, creating a composite quality signal that reflects both machine-measured accuracy and human-perceived utility. This feedback loop operates continuously: every translation output carries an invitation for user rating, every rating is incorporated into the model’s capability score through a weighted moving average, and the updated scores flow back into the model selection and routing systems within minutes.

7.4.1 User Rating Interface

After each translation is delivered, the user interface presents a non-intrusive rating widget inviting the user to assign 1–5 stars and optionally provide free-text feedback. The rating prompt appears in a collapsible panel adjacent to the translation output, minimizing disruption to the user’s workflow. Five-star ratings indicate excellent quality requiring no further attention. Four-star ratings signal minor issues—perhaps a single awkward phrasing or terminology choice. Three-star and below trigger an expanded feedback form asking the user to categorize the problem (accuracy, fluency, terminology, formatting, cultural appropriateness). These categorizations feed into granular capability component scores, enabling the system to distinguish between a model that struggles with medical terminology but excels at grammatical fluency versus one with the opposite profile.

7.4.2 Automatic Quality Metrics

In parallel with user ratings, the system computes automatic quality metrics for every translation. The BLEU (Bilingual Evaluation Understudy) score measures n-gram overlap between the model output and reference translations from the HelixTranslate validation corpus. The COMET (Cross-lingual Optimized Metric for Evaluation of Translation) score provides a neural evaluation that correlates more strongly with human judgments than BLEU by leveraging cross-lingual sentence embeddings. Semantic similarity is computed using cosine distance between vector embeddings of the source text and back-translated output, detecting meaning drift that lexical metrics might miss. Perplexity measures the model’s confidence in its output, with unusually high perplexity signaling potential hallucination or exposure to out-of-distribution content. These automatic metrics are computed asynchronously in a background worker queue, with results typically available within 2–5 minutes of translation completion.

7.4.3 Composite Score Computation

The composite quality engine synthesizes three independent signal sources into a unified quality score using a weighted formula. User ratings contribute 40% of the composite weight, reflecting the primacy of human judgment. Automatic metrics (BLEU, COMET, semantic similarity) contribute 35%, providing an objective baseline that operates independently of user engagement. Challenge pass rate—derived from the Phase 4 verification challenge system—contributes 25%, capturing the model’s performance on structured, adversarial test cases. The `SubmitRating` method in Code Block 4 demonstrates how a newly submitted user rating triggers an immediate capability score update.

Code Block 4: UserRating

```
type UserRating struct {
    TranslationID string `json:"translation_id"`
    ModelID       string `json:"model_id"`
    Rating        int    `json:"rating" // 1-5`
    Feedback      string `json:"feedback"`
    CreatedAt     time.Time `json:"created_at"`
}

func (e *ScoringEngine) SubmitRating(ctx context.Context, rating UserRating) error {
    // Weighted moving average: 40% user ratings + 35% auto metrics + 25% challenge pass rate
    current, _ := e.GetModelScore(ctx, rating.ModelID)
    newCapability := 0.4*float64(rating.Rating)*2.0 + 0.35*current.Components.Capability +
        ↪ 0.25*current.ChallengePassRate
    return e.UpdateCapabilityScore(ctx, rating.ModelID, newCapability)
```

```
}
```

The rating scaling logic (`float64(rating.Rating) * 2.0`) maps the 1–5 star scale onto the 0–10 score scale used by the composite engine, ensuring dimensional consistency across signal sources. The `UpdateCapabilityScore` method persists the new score to the model registry and emits a `score_changed` WebSocket event, which propagates the update to all connected dashboards within milliseconds. The use of a weighted moving average (rather than simple averaging) ensures that new ratings have immediate impact while historical data still influences the score, preventing erratic swings from single outlier ratings.

7.4.4 Signal Source Reference

Table 2 documents the complete signal inventory, including each source’s relative weight, the specific metrics it contributes, and its characteristic latency profile.

Table 2: Quality signals

Signal Source	Weight	Metric	Latency
User ratings	40%	1-5 stars	Real-time
Auto metrics	35%	BLEU, COMET, semantic sim	Minutes
Challenge pass	25%	Pass rate 0-100%	Hours

The latency column is operationally significant because it determines how quickly each signal reflects changes in model behavior. User ratings provide the fastest feedback but depend on user engagement—models serving low-traffic language pairs may accumulate ratings slowly. Automatic metrics provide consistent, engagement-independent coverage with a moderate delay. Challenge pass rates update the slowest (reflecting the batch nature of verification challenge execution) but capture adversarial robustness that neither user ratings nor automatic metrics can reliably measure.

7.4.5 Composite Quality Engine Implementation

The full composite quality engine is implemented in Go as shown in Code Block 5. The engine encapsulates the three weight constants, enforces their sum to 1.0 during initialization, and exposes a single `CalculateQuality` method that retrieves the component scores and computes the weighted composite.

Code Block 5: Composite quality engine

```
type CompositeQualityEngine struct {
    userRatingWeight float64 // 0.40
    autoMetricWeight float64 // 0.35
    challengePassWeight float64 // 0.25
}

func (q *CompositeQualityEngine) CalculateQuality(ctx context.Context, modelID string) (float64, error) {
    {
        userScore := q.getAverageUserRating(modelID) * 2.0 // scale 1-5 to 0-10
        autoScore := q.getAutoMetricsScore(modelID)
        challengeScore := q.getChallengePassRate(modelID) * 0.1 // scale 0-100 to 0-10

        composite := q.userRatingWeight*userScore +
```

```
        q.autoMetricWeight*autoScore +
        q.challengePassWeight*challengeScore
    return composite, nil
}
```

The normalization logic ensures dimensional consistency across the three inputs. User ratings on a 1–5 scale are multiplied by 2.0 to map onto the 0–10 composite scale. Challenge pass rates on a 0–100 percentage scale are multiplied by 0.1 for the same purpose. The `getAutoMetricsScore` method returns a score already normalized to the 0–10 scale by the Phase 2 scoring pipeline. The engine validates during construction that `userRatingWeight + autoMetricWeight + challengePassWeight == 1.0`, panicking on violation to prevent silent score corruption. Weight values are configurable through environment variables, enabling A/B experiments and operational adjustments without code changes.

The composite score produced by this engine feeds directly into the `VerifiedModel.score.overall` field consumed by the `ModelCard` component in Section 7.1 and the `RoutingStrategy` implementations in Section 7.3. This architectural closure ensures that user feedback, once submitted, propagates through the entire system—from the rating widget, through the composite engine, into the model registry, across the WebSocket stream, onto the model selection dashboard, and ultimately into the routing decisions that determine which model translates the next chunk of content. The feedback loop is fully closed, self-reinforcing, and operates without human intervention beyond the initial rating gesture.

8. Phase 7: Testing Strategy and Anti-Bluff Quality Assurance

Phase 7 closes the 56.4% coverage gap between the current 43.6% and the 100% target across all packages touched by the `LLMsVerifier` integration. The preceding phases produced eight pipeline steps in `internal/verifier/pipeline.go`, five scoring components in `internal/verifier/scoring/engine.go`, a 3-tier discovery system, four selection strategies, and four new API endpoints. This phase defines the testing strategy, implements unit and integration tests for every component, establishes the anti-bluff Challenge test framework, and sets performance SLA targets. Every test must prove real behavior; mocks are permitted only for external HTTP calls.

8.1 Testing Strategy Overview and Coverage Targets

8.1.1 Test Pyramid Architecture

The test pyramid allocates 70% of test files to unit tests for algorithmic correctness, 20% to integration tests for behavioral validation across package boundaries, and 10% to end-to-end tests for complete user journeys. Table 8.1 defines each layer.

Table 8.1: Test Pyramid — Layer Definitions and Coverage Targets

Layer	Test Files	Coverage Target	Execution	Anti-Bluff Rule
Unit (70%)	~140	95% per package	< 30s	Mocks only for external HTTP; system under test uses real impl
Integration (20%)	~40	85% path coverage	< 5 min	Real infrastructure: SQLite, Dockerized services, JWT tokens

Table 23 – continued

Layer	Test Files	Coverage Target	Execution	Anti-Bluff Rule
E2E (10%)	~20	100% user journey	< 10 min	Full system stack; tests run against built binary

Unit tests target the 15+ new integration files. Every branching function — `calculateResponseSpeedScore` with its four throughput tiers, `Allow` in `middleware.go` with its sliding-window algorithm, `CalculateComprehensiveScore` with five-component weighted aggregation — receives dedicated table-driven tests exercising each branch. The 95% coverage target applies per package, not globally; global averaging is prohibited because it allows one well-tested package to mask an untested one.

Integration tests validate cross-package composition. The scoring engine calls `se.dbIntegration.UpdateModelScores()` after computing the weighted total; an integration test verifies this write persists and subsequent reads return the stored value. The discovery service falls back from static registry to provider API to `models.dev`; an integration test verifies tier-1 failure triggers tier-2 within the configured timeout. E2E tests validate complete journeys: discovery populates the registry, the gatekeeper filters by threshold, the selector picks the top model, the API returns it, and the translation pipeline produces valid output.

8.1.2 Coverage Gap Analysis

Current coverage across the seven affected packages is 43.6%. Table 8.2 breaks down the gap.

Table 8.2: Coverage Gap Analysis by Package

Package	Current	Target	Gap	Priority	Blocker
<code>pkg/translator/llm/</code>	43.6%	100%	+56.4%	High	61 disabled tests in <code>provider_test.go</code> must be re-enabled for verified provider flow
<code>internal/config/</code>	~60%	100%	+40%	High	New verifier config structs (<code>VerifierConfig</code> , <code>DiscoveryConfig</code> , <code>ScoringConfig</code>) untested
<code>pkg/api/</code>	~50%	100%	+50%	Critical	Four new endpoints (<code>/api/v1/models/discover</code> , <code>/verify</code> , <code>/select</code> , <code>/scores</code>) have zero coverage

Table 24 – continued

Package	Current	Target	Gap	Priority	Blocker
pkg/ verification/	~30%	100%	+70%	Critical	8-step pipeline flow has no automated validation
pkg/events/	~40%	100%	+60%	Medium	Verifier event bus integration lacks tests
internal/ verifier/ (new)	0%	100%	+100%	Critical	All new files — client.go, pipeline.go, scoring/engine.go, discovery/*.go, selection/*.go — untested
internal/ services/	~45%	100%	+55%	High	llmsverifier_score field mapping and cache logic untested

The 61 disabled tests in `pkg/translator/llm/provider_test.go` are the largest blocker. They were disabled during initial integration because they relied on the legacy provider factory. Re-enabling them requires updating the test setup to initialize the verifier client: replace `NewProvider(cfg)` with `NewProvider(cfg, WithVerifierClient(verifierClient))`, where `verifierClient` connects to a dockerized `LLMsVerifier` via `testcontainers`.

8.1.3 Testing Philosophy: The Anti-Bluff Mandate

The anti-bluff philosophy is codified in eight constitution rules. The project experienced a failure mode where all tests passed but features did not work — caused by over-mocking. Tests verified that `mockClient.Verify()` returned expected values, but never verified that the real HTTP client could connect, that the real database could persist, or that the scoring engine produced numerically correct results.

Rule T-01: a passing test guarantees the feature works for end users. A score adapter test cannot assert that `adapter.Convert()` was called with correct arguments; it must assert that the converted `ProviderScore` contains the correct `OverallScore` from a live `LLMsVerifier` instance. Rule T-02: mocks restricted to external HTTP/API calls only. The `modelsDevClient` in `scoring/engine.go` may be mocked because `models.dev` is external; the `DatabaseIntegration` struct must not be mocked — tests use `database.New(":memory:").` Rule T-03: every test verifies real behavior, not just invocation. Calling `engine.CalculateComprehensiveScore()` and asserting only `require.NoError(t, err)` violates this rule; the test must also assert the score falls within the expected range given the input model characteristics. Rule T-04: integration tests use real infrastructure. The `tests/integration/` directory contains a `test-compose.yml` with a real `LLMsVerifier` container, SQLite on a volume mount, and mock provider endpoints via `mockserver/mockserver`.

Rules T-05 through T-08 are enforcement mechanisms: T-05 declares a feature without a Challenge test

incomplete; T-06 requires Challenge tests to run in production-like environments; T-07 makes coverage below 100% a build-blocking failure; T-08 imposes a 24-hour fix-or-remove deadline on flaky tests.

8.1.4 Constitution Rule Documentation

The anti-bluff rule enters three governance documents. In `internal/verifier/CONSTITUTION.md`, Rule T-01 reads: “A passing test guarantees the feature works for end users — mocks in production paths are prohibited.” Enforcement: `go test -cover` plus manual review; violation: PR rejection. In `CLAUDE.md`, the Testing Guidelines section specifies: when testing `CalculateComprehensiveScore`, use a real `ScoringEngine` with `DatabaseIntegration` backed by `:memory: SQLite`; mock only `ModelsDevClientInterface`. In `AGENTS.md`, the Testing Agent’s decision tree starts with: on test failure, check if the test mocks the system under test → if yes, rewrite with real implementations → if still failing, investigate production code.

8.2 Unit Testing — Core Verification Components

8.2.1 Verifier Client Test Implementation

The verifier client in `internal/verifier/client.go` wraps HTTP communication with `LLMsVerifier`. Its constructor `func NewVerifierClient(cfg *Config) (*Client, error)` validates configuration and verifies connectivity via a health ping. The test file `internal/verifier/client_test.go` targets 95% coverage. Container-based tests are gated behind `//go:build verifier_integration` for CI only; fast-path tests use `httptest.Server`.

Code Block 8.1: `internal/verifier/client_test.go`

```
package verifier

import (
    "context"
    "errors"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestNewVerifierClient(t *testing.T) {
    tests := []struct {
        name          string
        cfg            *Config
        wantErr       bool
        errContains   string
        validate      func(t *testing.T, c *Client)
    }{
        {
```



```

    name: "valid config with explicit timeout",
    cfg: &Config{
        APIURL: "https://verifier.example.com",
        APIKey: "test-key-123",
        Timeout: 10 * time.Second,
    },
    validate: func(t *testing.T, c *Client) {
        assert.Equal(t, "https://verifier.example.com", c.baseURL)
        assert.Equal(t, "test-key-123", c.apiKey)
        assert.Equal(t, 10*time.Second, c.httpClient.Timeout)
    },
},
{
    name: "valid config with default timeout",
    cfg: &Config{
        APIURL: "https://verifier.example.com",
        APIKey: "test-key-456",
    },
    validate: func(t *testing.T, c *Client) {
        assert.Equal(t, 30*time.Second, c.httpClient.Timeout)
    },
},
{
    name: "empty API URL",
    cfg: &Config{APIKey: "test-key"},
    wantErr: true,
    errContains: "APIURL is required",
},
{
    name: "empty API key",
    cfg: &Config{APIURL: "https://verifier.example.com"},
    wantErr: true,
    errContains: "APIKey is required",
},
{
    name: "invalid URL scheme",
    cfg: &Config{APIURL: "ftp://verifier.example.com", APIKey: "key"},
    wantErr: true,
    errContains: "URL scheme must be http or https",
},
{
    name: "timeout below minimum",
    cfg: &Config{APIURL: "https://v.example.com", APIKey: "k", Timeout: 100 *
        ⇨ time.Millisecond},
    wantErr: true,
    errContains: "timeout must be at least 1s",
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {

```

```

        client, err := NewVerifierClient(tt.cfg)
        if tt.wantErr {
            require.Error(t, err)
            assert.Contains(t, err.Error(), tt.errContains)
            return
        }
        require.NoError(t, err)
        require.NotNil(t, client)
        tt.validate(t, client)
    })
}

func TestVerifierClient_Ping(t *testing.T) {
    t.Run("healthy server returns nil", func(t *testing.T) {
        server := httptest.NewServer(http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                assert.Equal(t, "/health", r.URL.Path)
                assert.Equal(t, "Bearer test-key", r.Header.Get("Authorization"))
                w.WriteHeader(http.StatusOK)
                w.Write([]byte(`{"status":"healthy"}`))
            },
        ))
        defer server.Close()

        client, err := NewVerifierClient(&Config{APIURL: server.URL, APIKey: "test-key"})
        require.NoError(t, err)
        assert.NoError(t, client.Ping(context.Background()))
    })

    t.Run("unhealthy server returns error", func(t *testing.T) {
        server := httptest.NewServer(http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                w.WriteHeader(http.StatusServiceUnavailable)
                w.Write([]byte(`{"status":"unhealthy"}`))
            },
        ))
        defer server.Close()

        client, _ := NewVerifierClient(&Config{APIURL: server.URL, APIKey: "k"})
        err := client.Ping(context.Background())
        require.Error(t, err)
        assert.Contains(t, err.Error(), "unhealthy")
    })

    t.Run("timeout returns deadline exceeded", func(t *testing.T) {
        server := httptest.NewServer(http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                time.Sleep(2 * time.Second)
                w.WriteHeader(http.StatusOK)
            },
        ))
    })
}

```

```

    ))
    defer server.Close()

    client, _ := NewVerifierClient(&Config{APIURL: server.URL, APIKey: "k", Timeout: 500 *
        ↪ time.Millisecond})
    ctx, cancel := context.WithTimeout(context.Background(), 300*time.Millisecond)
    defer cancel()

    err := client.Ping(ctx)
    require.Error(t, err)
    assert.True(t, errors.Is(err, context.DeadlineExceeded) || strings.Contains(err.Error(),
        ↪ "timeout"),
        "expected timeout error, got: %v", err)
    })
}

```

The constructor tests validate six scenarios: valid config with explicit and default timeouts, empty API URL, empty API key, invalid URL scheme, and timeout below minimum. The Ping tests verify correct Authorization header handling, HTTP 200/503 responses, and timeout error propagation with `context.DeadlineExceeded` wrapping. The anti-bluff assertion on the timeout test checks for the specific error type, ensuring a DNS failure does not spuriously pass as a timeout.

8.2.2 Verification Pipeline Test Implementation

The verification pipeline in `internal/verifier/pipeline.go` orchestrates eight ordered steps: existence check, connectivity test, authentication validation, completion verification, capability detection, translation quality assessment, latency benchmarking, and error handling classification. The test file `internal/verifier/pipeline_test.go` tests each step with injected pass/fail outcomes, validates step ordering, verifies error propagation from critical steps, and tests retry logic.

Code Block 8.2: `internal/verifier/pipeline_test.go`

```

package verifier

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

type mockStep struct {
    name          string
    shouldFail    bool
    isCritical    bool
    called        bool
    duration      time.Duration
}

```

```

func (m *mockStep) Execute(ctx context.Context, model *ModelContext) StepResult {
    m.called = true
    time.Sleep(m.duration)
    if m.shouldFail {
        return StepResult{StepName: m.name, Passed: false, Critical: m.isCritical, Error:
            ↪ errors.New(m.name + " failed")}
    }
    return StepResult{StepName: m.name, Passed: true, Critical: m.isCritical}
}

type orderRecordingStep struct {
    wrapped PipelineStep
    order    []string
}

func (o *orderRecordingStep) Execute(ctx context.Context, model *ModelContext) StepResult {
    *o.order = append(*o.order, o.wrapped.(*mockStep).name)
    return o.wrapped.Execute(ctx, model)
}

func TestPipeline_StepOrdering(t *testing.T) {
    var executionOrder []string
    steps := []PipelineStep{
        &mockStep{name: "existence", duration: 1 * time.Millisecond},
        &mockStep{name: "connectivity", duration: 1 * time.Millisecond},
        &mockStep{name: "authentication", duration: 1 * time.Millisecond},
        &mockStep{name: "completion", duration: 1 * time.Millisecond},
        &mockStep{name: "capability", duration: 1 * time.Millisecond},
        &mockStep{name: "quality", duration: 1 * time.Millisecond},
        &mockStep{name: "latency", duration: 1 * time.Millisecond},
        &mockStep{name: "error_handling", duration: 1 * time.Millisecond},
    }
    for i := range steps {
        steps[i] = &orderRecordingStep{wrapped: steps[i], order: &executionOrder}
    }

    pipeline := NewPipeline(steps)
    result := pipeline.Run(context.Background(), &ModelContext{ModelID: "gpt-4", ProviderID: "openai"})

    require.NoError(t, result.Error)
    require.Equal(t, 8, len(executionOrder))
    assert.Equal(t, "existence", executionOrder[0])
    assert.Equal(t, "connectivity", executionOrder[1])
    assert.Equal(t, "authentication", executionOrder[2])
    assert.Equal(t, "latency", executionOrder[6])
    assert.Equal(t, "error_handling", executionOrder[7])
}

func TestPipeline_ErrorPropagation(t *testing.T) {
    tests := []struct {
        name    string

```

```

failedStep    string
isCritical    bool
expectError   bool
}{
{name: "critical existence failure aborts", failedStep: "existence", isCritical: true,
↪ expectError: true},
{name: "critical auth failure aborts", failedStep: "authentication", isCritical: true,
↪ expectError: true},
{name: "non-critical quality failure continues", failedStep: "quality", isCritical: false,
↪ expectError: false},
{name: "non-critical latency failure continues", failedStep: "latency", isCritical: false,
↪ expectError: false},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        steps := []PipelineStep{
            &mockStep{name: "existence", shouldFail: tt.failedStep == "existence", isCritical: true},
            &mockStep{name: "connectivity", shouldFail: tt.failedStep == "connectivity", isCritical:
                ↪ true},
            &mockStep{name: "authentication", shouldFail: tt.failedStep == "authentication",
                ↪ isCritical: true},
            &mockStep{name: "completion", shouldFail: tt.failedStep == "completion", isCritical:
                ↪ false},
            &mockStep{name: "capability", shouldFail: tt.failedStep == "capability", isCritical:
                ↪ false},
            &mockStep{name: "quality", shouldFail: tt.failedStep == "quality", isCritical: false},
            &mockStep{name: "latency", shouldFail: tt.failedStep == "latency", isCritical: false},
            &mockStep{name: "error_handling", shouldFail: tt.failedStep == "error_handling",
                ↪ isCritical: true},
        }

        pipeline := NewPipeline(steps)
        result := pipeline.Run(context.Background(), &ModelContext{ModelID: "gpt-4", ProviderID:
            ↪ "openai"})

        if tt.expectError {
            require.Error(t, result.Error)
            assert.Contains(t, result.Error.Error(), tt.failedStep)
            for _, s := range steps {
                m := s.(*mockStep)
                if m.name != tt.failedStep && m.isCritical {
                    assert.False(t, m.called, "critical step %s should not run after abort", m.name)
                }
            }
        } else {
            require.NoError(t, result.Error)
            for _, s := range steps {
                assert.True(t, s.(*mockStep).called, "step %s should have been called",
                    ↪ s.(*mockStep).name)
            }
        }
    })
}

```

```

    }
    })
}

type flakyMockStep struct {
    attempts int
    failCount int
}

func (f *flakyMockStep) Execute(ctx context.Context, model *ModelContext) StepResult {
    f.attempts++
    if f.attempts <= f.failCount {
        return StepResult{StepName: "flaky", Passed: false, Critical: true, Error: errors.New("transient
        ↪ failure")}
    }
    return StepResult{StepName: "flaky", Passed: true, Critical: true}
}

func TestPipeline_RetryLogic(t *testing.T) {
    flaky := &flakyMockStep{failCount: 2}
    steps := []PipelineStep{flaky, &mockStep{name: "completion"}}

    pipeline := NewPipelineWithRetry(steps, RetryConfig{MaxAttempts: 3, Backoff: 10 * time.Millisecond})
    result := pipeline.Run(context.Background(), &ModelContext{ModelID: "gpt-4"})

    require.NoError(t, result.Error)
    assert.Equal(t, 3, flaky.attempts, "expected 3 attempts (2 failures + 1 success)")
}

```

The `mockStep` struct implements `PipelineStep` and records execution for post-test assertions. `TestPipeline_StepOrdering` wraps each step in an `orderRecordingStep` and asserts the eight-step sequence, catching any refactoring that reorders steps. `TestPipeline_ErrorPropagation` covers four scenarios: critical failures that abort the pipeline and non-critical failures that allow continuation. The anti-bluff check verifies that for critical failures subsequent steps were not called, and for non-critical failures all eight steps executed. The retry test uses a `flakyMockStep` that fails twice before succeeding; the assertion checks `attempts == 3`, verifying the retry mechanism actually re-invoked the step.

8.3 Unit Testing — Scoring and Adapter Components

8.3.1 Scoring Engine Test Implementation

The scoring engine in `internal/verifier/scoring/engine.go` computes five component scores and aggregates them into a weighted total. The test file `internal/verifier/scoring/engine_test.go` contains 20+ test cases with known inputs and mathematically expected outputs.

Code Block 8.3: `internal/verifier/scoring/engine_test.go`

```
package scoring
```

```

import (
    "context"
    "testing"
    "time"

    "digital.vasic.llmsverifier/database"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

type ScoreTestCase struct {
    name            string
    modelData       *ModelData
    dbModel         *database.Model
    expectedSpeed    float64
    expectedEfficiency float64
    expectedCost     float64
    expectedCapability float64
    expectedRecency  float64
    expectedOverall  float64
    tolerance        float64
}

func TestCalculateComprehensiveScore_KnownInputs(t *testing.T) {
    recent := time.Date(2024, 6, 1, 0, 0, 0, 0, time.UTC)
    old := time.Date(2022, 1, 1, 0, 0, 0, 0, time.UTC)
    ctxTokens := 128000
    paramSmall := int64(500_000_000)
    paramLarge := int64(175_000_000_000)

    tests := []ScoreTestCase{
        {
            name: "optimal model — high across all components",
            modelData: &ModelData{
                ID: "gpt-4-optimal", Name: "GPT-4 Optimal", Provider: "OpenAI",
                ContextWindow: 128000, InputTokenCost: 1.0, ThroughputRPS: 15.0,
                LatencyMs: 200, ParameterCount: paramSmall,
                OpenSource: false, Multimodal: true, Reasoning: true, LastUpdated: recent,
            },
            dbModel: &database.Model{
                ModelID: "gpt-4-optimal", ResponsivenessScore: 9.0,
                ParameterCount: &paramSmall, ContextWindowTokens: &ctxTokens,
                IsMultimodal: true, SupportsReasoning: true,
                ReleaseDate: &recent, TrainingDataCutoff: &recent, LastVerified: &recent,
            },
            expectedSpeed: 9.0, expectedEfficiency: 10.0, expectedCost: 9.0,
            expectedCapability: 8.5, expectedRecency: 10.0, expectedOverall: 9.2,
            tolerance: 0.1,
        },
        {
            name: "expensive large model — low cost, high capability",

```

```

modelData: &ModelData{
    ID: "gpt-4-expensive", Name: "GPT-4 Expensive", Provider: "OpenAI",
    ContextWindow: 128000, InputTokenCost: 30.0, ThroughputRPS: 5.0,
    LatencyMs: 2000, ParameterCount: paramLarge,
    OpenSource: false, Multimodal: true, Reasoning: true, LastUpdated: recent,
},
dbModel: &database.Model{
    ModelID: "gpt-4-expensive", ResponsivenessScore: 5.0,
    ParameterCount: &paramLarge, ContextWindowTokens: &ctxTokens,
    IsMultimodal: true, SupportsReasoning: true,
    ReleaseDate: &recent, TrainingDataCutoff: &recent, LastVerified: &recent,
},
expectedSpeed: 6.0, expectedEfficiency: 8.0, expectedCost: 2.0,
expectedCapability: 8.5, expectedRecency: 10.0, expectedOverall: 6.2,
tolerance: 0.1,
},
{
    name: "old cheap model — high cost, low recency",
    modelData: &ModelData{
        ID: "old-cheap", Name: "Old Cheap", Provider: "OpenSource",
        ContextWindow: 32000, InputTokenCost: 0.1, ThroughputRPS: 3.0,
        LatencyMs: 3000, ParameterCount: paramSmall,
        OpenSource: true, Multimodal: false, Reasoning: false, LastUpdated: old,
    },
    dbModel: &database.Model{
        ModelID: "old-cheap", ResponsivenessScore: 4.0,
        ParameterCount: &paramSmall, ContextWindowTokens: nil,
        IsMultimodal: false, SupportsReasoning: false,
        ReleaseDate: &old, TrainingDataCutoff: &old, LastVerified: &old,
        OpenSource: true,
    },
    expectedSpeed: 4.0, expectedEfficiency: 7.0, expectedCost: 10.0,
    expectedCapability: 5.0, expectedRecency: 2.0, expectedOverall: 6.0,
    tolerance: 0.1,
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        db := setupTestDB(t)
        defer cleanupTestDB(t, db)

        provider := &database.Provider{Name: "TestProvider", Endpoint: "https://api.test.com",
            ↪ IsActive: true}
        require.NoError(t, db.CreateProvider(provider))
        tt.dbModel.ProviderID = provider.ID
        require.NoError(t, db.CreateModel(tt.dbModel))

        mockClient := NewMockModelsDevClient()
        mockClient.AddMockModel(modelsDevModel{
            ModelID: tt.modelData.ID, Model: tt.modelData.Name,

```



```

        Provider: tt.modelData.Provider, InputCostPer1M: tt.modelData.InputTokenCost,
        ContextLimit: tt.modelData.ContextWindow,
        ReleaseDate: tt.modelData.LastUpdated.Format("2006-01-02"),
        AdditionalData: ModelsDevAdditionalData{
            ParameterCount: tt.modelData.ParameterCount,
            OpenWeights:    tt.modelData.OpenSource,
            Multimodal:     tt.modelData.Multimodal,
        },
    },
})

engine := NewScoringEngine(db, mockClient, &logging.Logger{})
score, err := engine.CalculateComprehensiveScore(context.Background(), tt.modelData.ID,
    ScoringConfig{Weights: DefaultScoreWeights()})
require.NoError(t, err)
require.NotNil(t, score)

assert.InDelta(t, tt.expectedSpeed, score.Components.SpeedScore, tt.tolerance)
assert.InDelta(t, tt.expectedEfficiency, score.Components.EfficiencyScore, tt.tolerance)
assert.InDelta(t, tt.expectedCost, score.Components.CostScore, tt.tolerance)
assert.InDelta(t, tt.expectedCapability, score.Components.CapabilityScore, tt.tolerance)
assert.InDelta(t, tt.expectedRecency, score.Components.RecencyScore, tt.tolerance)
assert.InDelta(t, tt.expectedOverall, score.OverallScore, tt.tolerance)

// Anti-bluff: verify persistence
retrieved, err := db.GetModel(tt.dbModel.ID)
require.NoError(t, err)
assert.InDelta(t, tt.expectedOverall, retrieved.OverallScore, tt.tolerance)
})
}
}

func TestCalculateComprehensiveScore_BoundaryConditions(t *testing.T) {
    tests := []struct {
        name      string
        setupDBModel func() *database.Model
    }{
        {
            name: "zero parameter count",
            setupDBModel: func() *database.Model {
                z := int64(0)
                return &database.Model{ParameterCount: &z, OpenSource: false}
            },
        },
        {
            name: "nil release date",
            setupDBModel: func() *database.Model {
                return &database.Model{ReleaseDate: nil, LastVerified: nil}
            },
        },
        {
            name: "all nil optional fields",

```

```

        setupDBModel: func() *database.Model {
            return &database.Model{
                ParameterCount: nil, ContextWindowTokens: nil, ReleaseDate: nil,
                TrainingDataCutoff: nil, LastVerified: nil, IsMultimodal: false,
                SupportsReasoning: false, OpenSource: false,
            }
        },
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        db := setupTestDB(t)
        defer cleanupTestDB(t, db)

        provider := &database.Provider{Name: "Test", Endpoint: "https://test.com", IsActive: true}
        require.NoError(t, db.CreateProvider(provider))
        dbModel := tt.setupDBModel()
        dbModel.ProviderID = provider.ID
        dbModel.ModelID = "boundary-test"
        dbModel.Name = "Boundary Test"
        require.NoError(t, db.CreateModel(dbModel))

        mockClient := NewMockModelsDevClient()
        mockClient.AddMockModel(ModelsDevModel{ModelID: "boundary-test", InputCostPer1M: 5.0,
            ↪ ContextLimit: 8000})

        engine := NewScoringEngine(db, mockClient, &logging.Logger{})
        score, err := engine.CalculateComprehensiveScore(context.Background(), "boundary-test",
            ScoringConfig{Weights: DefaultScoreWeights()})

        require.NoError(t, err)
        assert.GreaterOrEqual(t, score.OverallScore, 0.0)
        assert.LessOrEqual(t, score.OverallScore, 10.0)
        assert.GreaterOrEqual(t, score.Components.SpeedScore, 0.0)
        assert.LessOrEqual(t, score.Components.SpeedScore, 10.0)
    })
}
}

```

The test matrix covers three archetypes: optimal (high all components), expensive large (low cost, high capability), and old cheap open-source (high cost, low recency). Each case pre-computes the expected overall using the weight formula and asserts within 0.1 tolerance. The anti-bluff check queries the database after scoring to verify persistence. Boundary conditions cover zero parameter count, nil release date, and all-nil optional fields — asserting the score always falls within [0, 10].

8.3.2 Score Adapter Test Implementation

The score adapter in `internal/services/llmsverifier_score_adapter.go` converts `ComprehensiveScore` to `ProviderScore`. The test file validates field-by-field mapping, cache behavior, and error handling.

Code Block 8.4: internal/services/llmsverifier_score_adapter_test.go

```
package services

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestScoreAdapter_FieldMapping(t *testing.T) {
    tests := []struct {
        name          string
        input          *llmverifier.ComprehensiveScore
        expectedOutput *ProviderScore
    }{
        {
            name: "complete score with all components",
            input: &llmverifier.ComprehensiveScore{
                ModelID: "gpt-4", ModelName: "GPT-4 (SC:8.5)", OverallScore: 8.5,
                Components: llmverifier.ScoreComponents{
                    SpeedScore: 9.0, EfficiencyScore: 8.0, CostScore: 7.5,
                    CapabilityScore: 8.5, RecencyScore: 9.0,
                },
                LastCalculated: time.Date(2024, 6, 15, 12, 0, 0, 0, time.UTC),
                DataSource: "models.dev",
            },
            expectedOutput: &ProviderScore{
                ProviderID: "gpt-4", ProviderName: "GPT-4", OverallScore: 8.5,
                SpeedScore: 9.0, EfficiencyScore: 8.0, CostScore: 7.5,
                CapabilityScore: 8.5, RecencyScore: 9.0,
                ScoreSuffix: "(SC:8.5)", ScoreSource: "llmsverifier",
                LastUpdated: time.Date(2024, 6, 15, 12, 0, 0, 0, time.UTC),
            },
        },
        {
            name: "suffix extraction from model name",
            input: &llmverifier.ComprehensiveScore{
                ModelID: "claude-3-opus", ModelName: "Claude 3 Opus (SC:9.2)", OverallScore: 9.2,
                Components: llmverifier.ScoreComponents{
                    SpeedScore: 8.5, EfficiencyScore: 9.0, CostScore: 6.0,
                    CapabilityScore: 9.5, RecencyScore: 8.0,
                },
            },
            expectedOutput: &ProviderScore{
                ProviderID: "claude-3-opus", ProviderName: "Claude 3 Opus",
                OverallScore: 9.2, SpeedScore: 8.5, EfficiencyScore: 9.0,
                CostScore: 6.0, CapabilityScore: 9.5, RecencyScore: 8.0,
            },
        },
    }
}
```

```

        ScoreSuffix: "(SC:9.2)",
    },
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        adapter := NewScoreAdapter(&mockVerifierClient{
            scores: map[string]*llmverifier.ComprehensiveScore{tt.input.ModelID: tt.input},
        })

        result, err := adapter.GetScore(context.Background(), tt.input.ModelID)
        require.NoError(t, err)

        assert.Equal(t, tt.expectedOutput.ProviderID, result.ProviderID)
        assert.Equal(t, tt.expectedOutput.ProviderName, result.ProviderName)
        assert.InDelta(t, tt.expectedOutput.OverallScore, result.OverallScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.SpeedScore, result.SpeedScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.EfficiencyScore, result.EfficiencyScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.CostScore, result.CostScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.CapabilityScore, result.CapabilityScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.RecencyScore, result.RecencyScore, 0.01)
        assert.Equal(t, tt.expectedOutput.ScoreSuffix, result.ScoreSuffix)
        assert.Equal(t, tt.expectedOutput.ScoreSource, result.ScoreSource)
    })
}

}

func TestScoreAdapter_CacheBehavior(t *testing.T) {
    callCount := 0
    mockClient := &countingVerifierClient{
        getScoreFunc: func(ctx context.Context, modelID string) (*llmverifier.ComprehensiveScore, error) {
            ↪ {
                callCount++
                return &llmverifier.ComprehensiveScore{ModelID: modelID, OverallScore: 7.5, LastCalculated:
                    ↪ time.Now()}, nil
            },
        },
    }

    adapter := NewScoreAdapter(mockClient)
    adapter.SetCacheTTL(5 * time.Minute)
    ctx := context.Background()

    score1, err := adapter.GetScore(ctx, "gpt-4")
    require.NoError(t, err)
    assert.Equal(t, 1, callCount, "first call should hit verifier")

    score2, err := adapter.GetScore(ctx, "gpt-4")
    require.NoError(t, err)
    assert.Equal(t, 1, callCount, "second call should use cache")
}

```

```

// Anti-bluff: verify cached score is a copy
score1.OverallScore = 99.9
assert.Equal(t, 7.5, score2.OverallScore, "mutating result should not affect cache")

adapter.InvalidateCache("gpt-4")
_, err = adapter.GetScore(ctx, "gpt-4")
require.NoError(t, err)
assert.Equal(t, 2, callCount, "after invalidation, should re-fetch")
}

func TestScoreAdapter_ErrorHandling(t *testing.T) {
    t.Run("verifier unreachable returns fallback", func(t *testing.T) {
        mockClient := &failingVerifierClient{err: context.DeadlineExceeded}
        adapter := NewScoreAdapter(mockClient)

        score, err := adapter.GetScore(context.Background(), "gpt-4")
        require.NoError(t, err, "adapter should return fallback, not error")
        require.NotNil(t, score)
        assert.InDelta(t, 5.0, score.OverallScore, 0.01)
        assert.Equal(t, "fallback", score.ScoreSource)
    })

    t.Run("not-found error propagates", func(t *testing.T) {
        mockClient := &failingVerifierClient{err: llmverifier.ErrModelNotFound}
        adapter := NewScoreAdapter(mockClient)

        _, err := adapter.GetScore(context.Background(), "unknown-model")
        require.Error(t, err)
        assert.ErrorIs(t, err, ErrModelNotInRegistry)
    })
}

```

The field mapping test compares all ten fields individually — a struct equality check would miss a field swap. The cache test uses a `countingVerifierClient` and anti-bluff checks that mutating the returned pointer does not affect the cached copy. Error handling covers two modes: unreachable verifier returns a neutral fallback score of 5.0, and model-not-found propagates as `ErrModelNotInRegistry`.

8.4 Unit Testing — Discovery and Registry

8.4.1 Discovery Service Tests

The discovery service in `internal/verifier/discovery/service.go` implements 3-tier fallback: static registry, provider API, `models.dev`. `internal/verifier/discovery/service_test.go` mocks all three tiers to inject deterministic failure modes, tests that tier-1 failure triggers tier-2 within the 5-second timeout, and verifies deduplication prevents duplicate models when found in multiple tiers.

8.4.2 Registry Tests

The registry in `internal/verifier/discovery/registry.go` provides CRUD and search with filtering. `internal/verifier/discovery/registry_test.go` uses a real SQLite `:memory:` database — not a mock

— because SQL queries contain LIKE, BETWEEN, and JOIN clauses that mocks would silently pass with invalid SQL. Tests cover CRUD, search with provider/score/capability filters, `FilterByScore` boundary conditions, and `FilterByProvider` with invalid provider ID.

8.4.3 Gatekeeper Tests

The gatekeeper in `internal/verifier/discovery/gatekeeper.go` determines model availability from three inputs: verification status (2 values), score status (2 values), and health status (3 values) — 12 combinations. `internal/verifier/discovery/gatekeeper_test.go` covers all 12 plus boundary conditions.

Code Block 8.5: `internal/verifier/discovery/gatekeeper_test.go`

```
package discovery

import (
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
)

type GatekeeperDecision struct {
    name           string
    verified        bool
    scoreAboveThreshold bool
    healthStatus    string
    expectedAvailable bool
    expectedReason  string
}

func TestGatekeeper_DecisionMatrix(t *testing.T) {
    gk := NewGatekeeper(GatekeeperConfig{MinScoreThreshold: 6.0})

    tests := []GatekeeperDecision{
        {verified: true, scoreAboveThreshold: true, healthStatus: "healthy", expectedAvailable: true,
            ↪ expectedReason: ""},
        {verified: true, scoreAboveThreshold: true, healthStatus: "degraded", expectedAvailable: true,
            ↪ expectedReason: ""},
        {verified: true, scoreAboveThreshold: true, healthStatus: "unavailable", expectedAvailable:
            ↪ false, expectedReason: "health"},
        {verified: true, scoreAboveThreshold: false, healthStatus: "healthy", expectedAvailable: false,
            ↪ expectedReason: "score"},
        {verified: true, scoreAboveThreshold: false, healthStatus: "degraded", expectedAvailable: false,
            ↪ expectedReason: "score"},
        {verified: true, scoreAboveThreshold: false, healthStatus: "unavailable", expectedAvailable:
            ↪ false, expectedReason: "score"},
        {verified: false, scoreAboveThreshold: true, healthStatus: "healthy", expectedAvailable: false,
            ↪ expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: true, healthStatus: "degraded", expectedAvailable: false,
            ↪ expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: true, healthStatus: "unavailable", expectedAvailable:
            ↪ false, expectedReason: "verification"},
    }
```

```

{verified: false, scoreAboveThreshold: false, healthStatus: "healthy", expectedAvailable: false,
  ⇨ expectedReason: "verification"},
{verified: false, scoreAboveThreshold: false, healthStatus: "degraded", expectedAvailable:
  ⇨ false, expectedReason: "verification"},
{verified: false, scoreAboveThreshold: false, healthStatus: "unavailable", expectedAvailable:
  ⇨ false, expectedReason: "verification"},
}

for i, tt := range tests {
  t.Run(fmt.Sprintf("case_%d_v%v_s%v_h%s", i, tt.verified, tt.scoreAboveThreshold,
    ⇨ tt.healthStatus), func(t *testing.T) {
    model := &RegistryModel{
      ModelID:      "test-model",
      Verified:     tt.verified,
      OverallScore: gatekeeperTestScore(tt.scoreAboveThreshold),
      HealthStatus: tt.healthStatus,
      LastVerified: gatekeeperTestTimestamp(tt.verified),
    }

    available, reason := gk.IsAvailable(model)
    assert.Equal(t, tt.expectedAvailable, available,
      "availability mismatch for v=%v s=%v h=%s", tt.verified, tt.scoreAboveThreshold,
      ⇨ tt.healthStatus)
    assert.Equal(t, tt.expectedReason, reason,
      "reason mismatch for v=%v s=%v h=%s", tt.verified, tt.scoreAboveThreshold,
      ⇨ tt.healthStatus)
  })
}
}

func TestGatekeeper_BoundaryConditions(t *testing.T) {
  gk := NewGatekeeper(GatekeeperConfig{MinScoreThreshold: 6.0})

  t.Run("exactly at threshold", func(t *testing.T) {
    model := &RegistryModel{ModelID: "boundary", Verified: true, OverallScore: 6.0, HealthStatus:
      ⇨ "healthy"}
    available, _ := gk.IsAvailable(model)
    assert.True(t, available, "model at exact threshold should be available")
  })

  t.Run("one epsilon below", func(t *testing.T) {
    model := &RegistryModel{ModelID: "boundary", Verified: true, OverallScore: 5.999, HealthStatus:
      ⇨ "healthy"}
    available, reason := gk.IsAvailable(model)
    assert.False(t, available)
    assert.Equal(t, "score", reason)
  })

  t.Run("negative score", func(t *testing.T) {
    model := &RegistryModel{ModelID: "negative", Verified: true, OverallScore: -1.0, HealthStatus:
      ⇨ "healthy"}

```

```

        available, reason := gk.IsAvailable(model)
        assert.False(t, available)
        assert.Equal(t, "score", reason)
    })
}

func gatekeeperTestScore(above bool) float64 {
    if above {
        return 8.5
    }
    return 4.0
}

func gatekeeperTestTimestamp(verified bool) *time.Time {
    if !verified {
        return nil
    }
    ts := time.Date(2024, 6, 1, 0, 0, 0, 0, time.UTC)
    return &ts
}

```

The decision matrix enumerates all 12 combinations. The anti-bluff check asserts both the availability boolean and the rejection reason — a boolean-only check could pass even with an incorrect reason that would mislead operators during incidents. Boundary conditions cover the exact threshold (6.0), one epsilon below (5.999), and a negative score (-1.0) to ensure the comparison operator is $\geq$ not $>$.

8.5 Integration Testing — End-to-End Verification Flow

8.5.1 Integration Test Environment and Full User Journey

The integration test `tests/integration/verifier_flow_test.go` validates the complete verification pipeline using a dockerized LLMsVerifier via Docker Compose. The `test-compose.yml` specifies three services: LLMsVerifier on port 18080, mock providers via `mockserver/mockserver` on port 11080, and HelixTranslate depending on both with condition: `service_healthy`.

Code Block 8.6: `tests/integration/verifier_flow_test.go`

```

package integration

import (
    "context"
    "fmt"
    "net/http"
    "os"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

```



```

const TestComposeFile = "./test-compose.yml"

func TestVerifierFlow_FullUserJourney(t *testing.T) {
    if os.Getenv("RUN_INTEGRATION_TESTS") == "" {
        t.Skip("set RUN_INTEGRATION_TESTS=1 to run integration tests")
    }

    ctx := context.Background()
    compose := testcompose.Up(t, TestComposeFile,
        testcompose.WithTimeout(2*time.Minute),
        testcompose.WaitForService("verifier", "/health", http.StatusOK),
    )
    defer compose.Down()

    verifierURL := compose.ServiceURL("verifier")

    // Step 1: Configure mock provider
    configureMockProviders(t, compose.ServiceURL("mock-providers"))

    // Step 2: Trigger discovery
    discoverResp := mustPost(t, verifierURL+"/api/v1/models/discover", map[string]any{
        "provider": "mock-openai",
    })
    require.Equal(t, "discovery_started", discoverResp["status"])

    // Step 3: Poll for discovery
    var discoveredModels []map[string]any
    require.Eventually(t, func() bool {
        models := mustGet(t, verifierURL+"/api/v1/models")
        discoveredModels = models["models"].([]map[string]any)
        return len(discoveredModels) > 0
    }, 60*time.Second, 2*time.Second, "discovery should find models")

    // Step 4: Verify first discovered model
    modelID := fmt.Sprintf("%.0f", discoveredModels[0]["id"].(float64))
    verifyResp := mustPost(t, verifierURL+"/api/v1/models/"+modelID+"/verify", nil)
    require.Equal(t, "verification_started", verifyResp["status"])

    // Step 5: Poll for verification (8-step pipeline, max 120s)
    var verifiedModel map[string]any
    require.Eventually(t, func() bool {
        model := mustGet(t, verifierURL+"/api/v1/models/"+modelID)
        status, ok := model["status"].(string)
        verifiedModel = model
        return ok && status == "verified"
    }, 120*time.Second, 5*time.Second, "verification should complete")

    // Step 6: Anti-bluff — score in valid range
    score, ok := verifiedModel["score"].(float64)
    require.True(t, ok, "verified model should have score")
}

```

```

assert.GreaterOrEqual(t, score, 0.0)
assert.LessOrEqual(t, score, 10.0)

// Step 7: Select model for translation
selectResp := mustPost(t, verifierURL+"/api/v1/models/select", map[string]any{
    "task": "translation", "source_lang": "en", "target_lang": "de",
})
selectedID := selectResp["selected_model_id"].(string)

// Anti-bluff: selected model must be from discovered set
found := false
for _, m := range discoveredModels {
    if fmt.Sprintf("%.0f", m["id"].(float64)) == selectedID {
        found = true
        break
    }
}
assert.True(t, found, "selected model must come from discovered set")
}

func TestVerifierFlow_ModelDegradation(t *testing.T) {
    if os.Getenv("RUN_INTEGRATION_TESTS") == "" {
        t.Skip("set RUN_INTEGRATION_TESTS=1")
    }

    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()

    verifierURL := compose.ServiceURL("verifier")

    // Seed model with high score
    mustPost(t, verifierURL+"/api/v1/models", map[string]any{
        "model_id": "degradable-model", "name": "Degradable (SC:8.5)",
        "provider": "mock-openai", "score": 8.5, "status": "verified",
    })

    // Drop score below threshold
    mustPut(t, verifierURL+"/api/v1/models/degradable-model", map[string]any{"score": 3.5})

    // Select should NOT pick degraded model
    selectResp, _ := httpPost(verifierURL+"/api/v1/models/select", map[string]any{
        "task": "translation", "source_lang": "en", "target_lang": "de",
    })
    selectedID, ok := selectResp["selected_model_id"].(string)
    if ok && selectedID == "degradable-model" {
        t.Fatal("degraded model with score 3.5 was selected — gatekeeper failed")
    }
}

```

The full journey exercises seven steps: configure mock, trigger discovery, poll for completion, verify model, poll for verification, assert score in $[0, 10]$, select for translation, and anti-bluff check that the selection

came from the discovered set. The degradation test simulates a score drop from 8.5 to 3.5 and asserts the selector avoids the degraded model — the exact failure mode the gatekeeper exists to prevent.

8.6 Integration Testing — API Endpoints

8.6.1 API Integration Test Implementation

`tests/integration/api_models_test.go` exercises all `/api/v1/models/*` endpoints with JWT auth, pagination, filtering, sorting, and concurrent access. The setup seeds 50 models across 5 providers before each run.

Code Block 8.7: `tests/integration/api_models_test.go`

```
package integration

import (
    "fmt"
    "net/http"
    "sync"
    "testing"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

type AuthenticatedTestClient struct {
    baseURL    string
    httpClient *http.Client
    token      string
}

func NewAuthenticatedTestClient(baseURL, secret string) *AuthenticatedTestClient {
    token, _ := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
        "sub": "test-user", "role": "admin",
        "iat": time.Now().Unix(), "exp": time.Now().Add(time.Hour).Unix(),
    }).SignedString([]byte(secret))
    return &AuthenticatedTestClient{
        baseURL: baseURL, httpClient: &http.Client{Timeout: 10 * time.Second}, token: token,
    }
}

func (c *AuthenticatedTestClient) Get(path string) (map[string]any, error) {
    req, _ := http.NewRequest("GET", c.baseURL+path, nil)
    req.Header.Set("Authorization", "Bearer "+c.token)
    resp, err := c.httpClient.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
```

```

var result map[string]any
json.NewDecoder(resp.Body).Decode(&result)
return result, nil
}

func SeedTestDatabase(t *testing.T, verifierURL string) {
    client := NewAuthenticatedTestClient(verifierURL, "test-secret")
    providerIDs := make([]string, 5)
    for i := 0; i < 5; i++ {
        resp, _ := client.Post("/api/v1/providers", map[string]any{
            "name": fmt.Sprintf("Provider-%d", i), "endpoint": fmt.Sprintf("https://api.p%d.com", i),
            "is_active": i < 4,
        })
        providerIDs[i] = resp["id"].(string)
    }
    for i := 0; i < 50; i++ {
        score := float64(i%10) + 1.0
        client.Post("/api/v1/models", map[string]any{
            "model_id": fmt.Sprintf("model-%03d", i), "name": fmt.Sprintf("Model %d (SC:%.1f)", i,
                ↪ score),
            "provider": providerIDs[i%5], "score": score, "status": "verified",
            "capabilities": []string{"text", "translation"},
        })
    }
}

func TestAPI_ModelsPagination(t *testing.T) {
    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()
    verifierURL := compose.ServiceURL("verifier")
    SeedTestDatabase(t, verifierURL)

    client := NewAuthenticatedTestClient(verifierURL, "test-secret")
    page1, _ := client.Get("/api/v1/models?limit=10&offset=0")
    models1 := page1["models"].([]any)
    assert.Len(t, models1, 10)
    assert.Equal(t, float64(50), page1["total"])

    page2, _ := client.Get("/api/v1/models?limit=10&offset=10")
    models2 := page2["models"].([]any)
    assert.Len(t, models2, 10)

    // Anti-bluff: pages should not overlap
    page1IDs := extractModelIDs(t, models1)
    page2IDs := extractModelIDs(t, models2)
    for _, id := range page1IDs {
        assert.NotContains(t, page2IDs, id, "pages should not overlap")
    }
}

func TestAPI_ModelsFiltering(t *testing.T) {

```

```

compose := testcompose.Up(t, TestComposeFile)
defer compose.Down()
verifierURL := compose.ServiceURL("verifier")
SeedTestDatabase(t, verifierURL)

client := NewAuthenticatedTestClient(verifierURL, "test-secret")
filtered, _ := client.Get("/api/v1/models?min_score=7.0")
models := filtered["models"].([]any)
assert.Greater(t, len(models), 0)
for _, m := range models {
    assert.GreaterOrEqual(t, m.(map[string]any)["score"].(float64), 7.0)
}
}

func TestAPI_ConcurrentAccess(t *testing.T) {
    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()
    verifierURL := compose.ServiceURL("verifier")
    SeedTestDatabase(t, verifierURL)

    client := NewAuthenticatedTestClient(verifierURL, "test-secret")
    var wg sync.WaitGroup
    errors := make(chan error, 100)

    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func(idx int) {
            defer wg.Done()
            resp, err := client.Get(fmt.Sprintf("/api/v1/models?limit=10&offset=%d", idx))
            if err != nil {
                errors <- err
                return
            }
            if len(resp["models"].([]any)) == 0 && idx < 50 {
                errors <- fmt.Errorf("request %d: expected models", idx)
            }
        }(i)
    }
    wg.Wait()
    close(errors)

    var errorList []error
    for e := range errors {
        errorList = append(errorList, e)
    }
    assert.Empty(t, errorList, "concurrent requests produced %d errors", len(errorList))
}

```

SeedTestDatabase creates 5 providers (4 active, 1 inactive) and 50 models with scores 1.0–10.0. The pagination test requests two pages and anti-bluff checks they do not overlap — a bug where offset is ignored would return identical pages. The filtering test asserts every returned model has score >= 7.0.

The concurrent access test fires 100 goroutines and collects errors through a channel, reporting specific failures rather than just a count.

8.7 Anti-Bluff Challenge Testing

8.7.1–8.7.4 Challenge Script Overview

Four challenge scripts validate subsystem behavior in production-like environments. `score_adapter_challenge.sh` compares adapter output field-by-field against `testdata/expected_scores.json`. `gatekeeping_challenge.sh` feeds all 12 matrix combinations to the gatekeeper CLI and compares JSON output. `discovery_resilience_challenge.sh` blocks each discovery tier via `iptables` and verifies fallback returns results. `selection_fairness_challenge.sh` creates 10 equivalent models, runs 1000 selections, and verifies no model exceeds 15% of selections.

8.7.5 Challenge Template

All challenges follow a four-phase template: setup, execute, assert, cleanup.

Code Block 8.8: Challenge Template — `selection_fairness_challenge.sh`

```
#!/bin/bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../../../../" && pwd)"
COMPOSE_FILE="${PROJECT_ROOT}/tests/integration/test-compose.yml"
RESULTS_FILE="/tmp/selection_fairness_$$.json"
EXPECTED_MAX_PERCENTAGE=15

setup() {
    echo "[SETUP] Starting environment..."
    docker-compose -f "${COMPOSE_FILE}" up -d verifier mock-providers
    for i in {1..30}; do
        curl -sf "http://localhost:18080/health" > /dev/null 2>&1 && break
    done
    sleep 1
    echo "[SETUP] Seeding 10 equivalent models..."
    for i in $(seq 1 10); do
        curl -sf -X POST "http://localhost:18080/api/v1/models" \
            -H "Content-Type: application/json" \
            -d '{"model_id":"'fair-model-$(printf "%03d" $i)'\",'name":"'Fair $i (SC:8.0)'\",'
            ↪ '"provider":"'equiv-provider'\",'score':8.0,'status":"'verified'"}' > /dev/null
    done
}

execute() {
    echo "[EXECUTE] Running 1000 selections..."
    go run "${PROJECT_ROOT}/cmd/challenge-runner" selection-fairness \
        --verifier-url "http://localhost:18080" --model-count 10 \
        --request-count 1000 --output "${RESULTS_FILE}"
}
```

```

assert() {
    echo "[ASSERT] Validating distribution..."
    max_percentage=$(jq '.max_selection_percentage' "${RESULTS_FILE}")
    selected_count=$(jq '.selected_models | length' "${RESULTS_FILE}")

    if (( $(echo "${max_percentage} > ${EXPECTED_MAX_PERCENTAGE}" | bc -l) )); then
        echo "CHALLENGE FAILED: max percentage ${max_percentage}% exceeds ${EXPECTED_MAX_PERCENTAGE}%"
        exit 1
    fi
    if [[ "${selected_count}" -ne 10 ]]; then
        echo "CHALLENGE FAILED: only ${selected_count}/10 models selected"
        exit 1
    fi
    echo "CHALLENGE PASSED"
}

cleanup() {
    docker-compose -f "${COMPOSE_FILE}" down -v 2>/dev/null || true
    rm -f "${RESULTS_FILE}"
}

trap cleanup EXIT
setup
execute
assert

```

The assert phase checks two anti-bluff conditions: no single model receives more than 15% (detecting a biased selector), and all 10 models receive at least one selection (detecting a selector that always picks the first item). `trap cleanup EXIT` ensures teardown even on assertion failure.

8.7.6 Challenge Registry

Table 8.3: All 193+ Challenges — Breakdown by Subsystem

Subsystem	Count	Pass Criteria	Execution Time	Required Resources
Scoring (S01–S20)	20	Computed score matches expected within 0.01 for each model archetype	~5 min	Dockerized LLMsVerifier + :memory: SQLite
Verification (V01–V80)	80	Each of 8 pipeline steps: 10 challenges for pass/fail/retry/timeout; full pipeline in 120s	~20 min	Dockerized LLMsVerifier + mock provider endpoints
Gatekeeping (G01–G12)	12	All 12 cells in 2×2×3 matrix correct; boundary at exact threshold	~2 min	:memory: SQLite only

Table 25 – continued

Subsystem	Count	Pass Criteria	Execution Time	Required Resources
Discovery (D01–D30)	30	10 per tier; fallback ordering and timeout behavior	~10 min	Dockerized LLMsVerifier + mockserver
Selection Fairness (L01–L10)	10	Load within 15% max; minimum 3 providers in top-10	~8 min	Dockerized LLMsVerifier + 50+ models
Score Adapter (A01–A05)	5	Field-by-field mapping matches expected JSON; cache correct	~3 min	Dockerized LLMsVerifier + real SQLite
Integration (I01–I20)	20	Full journey: discover→verify→score→select→translate→validate	~30 min	Full Docker Compose stack
Performance (P01–P16)	16	p50/p95/p99 within SLA; no goroutine leaks after 1h sustained	~90 min	4 CPU cores, 8GB RAM
Total	193	Zero diff in assertion phase	~168 min	Docker Compose + CI runner

The 193 challenges distribute across nine subsystems, with the largest concentration (80) in the verification pipeline. Each challenge has a deterministic pass criterion. Performance challenges account for 90 of the 168 total minutes due to the 1-hour sustained load test.

8.8 Performance and Load Testing

8.8.1 Scoring Engine Performance Test

`tests/perf/verifier_perf_test.go` benchmarks the scoring engine under concurrent load using `testing.B` with `b.ReportAllocs()` and a custom `errgroup`-based load generator.

Code Block 8.9: `tests/perf/verifier_perf_test.go`

```
package perf

import (
    "context"
    "fmt"
    "sort"
    "sync"
    "testing"
    "time"

    "golang.org/x/sync/errgroup"
)

func BenchmarkScoringEngine_SingleScore(b *testing.B) {
    db := setupBenchmarkDB(b)
```



```

defer cleanupBenchmarkDB(b, db)
engine, mockClient := setupScoringEngine(b, db)
seedBenchmarkModels(b, db, mockClient, 1)
ctx := context.Background()
config := DefaultScoreWeights()

b.ResetTimer()
b.ReportAllocs()
for i := 0; i < b.N; i++ {
    _, err := engine.CalculateComprehensiveScore(ctx, "bench-model-000", config)
    if err != nil {
        b.Fatalf("scoring failed: %v", err)
    }
}
}

func BenchmarkScoringEngine_BatchScore(b *testing.B) {
    db := setupBenchmarkDB(b)
    defer cleanupBenchmarkDB(b, db)
    engine, mockClient := setupScoringEngine(b, db)
    seedBenchmarkModels(b, db, mockClient, 100)
    ctx := context.Background()
    config := DefaultScoreWeights()
    modelIDs := make([]string, 100)
    for i := 0; i < 100; i++ {
        modelIDs[i] = fmt.Sprintf("bench-model-%03d", i)
    }

    b.ResetTimer()
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        _, err := engine.CalculateBatchScores(ctx, modelIDs, &config)
        if err != nil {
            b.Fatalf("batch scoring failed: %v", err)
        }
    }
}

func TestScoringEngine_ConcurrentLoad(t *testing.T) {
    if os.Getenv("RUN_PERF_TESTS") == "" {
        t.Skip("set RUN_PERF_TESTS=1")
    }
    db := setupBenchmarkDB(t)
    defer cleanupBenchmarkDB(t, db)
    engine, mockClient := setupScoringEngine(t, db)
    seedBenchmarkModels(t, db, mockClient, 50)

    ctx := context.Background()
    config := DefaultScoreWeights()
    const concurrentRequests = 1000

```

```
latencies := make([]time.Duration, concurrentRequests)
var latenciesMu sync.Mutex

g, ctx := errgroup.WithContext(ctx)
start := time.Now()
for i := 0; i < concurrentRequests; i++ {
    idx := i
    modelID := fmt.Sprintf("bench-model-%03d", idx%50)
    g.Go(func() error {
        reqStart := time.Now()
        _, err := engine.CalculateComprehensiveScore(ctx, modelID, config)
        duration := time.Since(reqStart)
        latenciesMu.Lock()
        latencies[idx] = duration
        latenciesMu.Unlock()
        return err
    })
}
require.NoError(t, g.Wait())

sort.Slice(latencies, func(i, j int) bool { return latencies[i] < latencies[j] })
p50 := latencies[concurrentRequests*50/100]
p95 := latencies[concurrentRequests*95/100]
p99 := latencies[concurrentRequests*99/100]

t.Logf("1000 concurrent: total=%v p50=%v p95=%v p99=%v", time.Since(start), p50, p95, p99)
assert.Less(t, p50, 50*time.Millisecond, "p50 exceeds 50ms SLA")
assert.Less(t, p95, 200*time.Millisecond, "p95 exceeds 200ms SLA")
assert.Less(t, p99, 500*time.Millisecond, "p99 exceeds 500ms SLA")
}
```

b. `ReportAllocs()` detects memory allocation regressions. The concurrent load test collects 1000 latency samples, sorts them, and asserts against SLA targets from Table 8.4.

8.8.2 Performance SLA Targets

Table 8.4: Performance SLA Targets — Latency Percentiles by Operation

Operation	p50 Target	p95 Target	p99 Target	Benchmark Function	Failure Action
Score retrieval (single)	< 50ms	< 200ms	< 500ms	BenchmarkScoringEngine_SingleScore	CI build fails
Model listing (50 models)	< 100ms	< 300ms	< 1s	BenchmarkAPI_ListModels	CI build fails
Full verification (8 steps)	< 5s	< 30s	< 60s	TestPipeline_FullVerification	CI build fails
Discovery (single provider)	< 10s	< 30s	< 60s	TestDiscovery_SingleProvider	CI build fails

Table 26 – continued

Operation	p50 Target	p95 Target	p99 Target	Benchmark Function	Failure Action
Model selection (cached)	< 10ms	< 50ms	< 100ms	BenchmarkSelectionEngine_Get	Git build fails
Batch calculation (100)	< 3s	< 10s	< 20s	BenchmarkScoringEngine_BatchScore	WhatsApp alert
Concurrent retrieval (1000)	< 50ms	< 200ms	< 500ms	TestScoringEngine_ConcurrentBatch	Git build fails

8.8.3 Memory Leak Detection

The 1-hour sustained load test runs 100 workers making score requests every 100ms, sampling heap and goroutine counts at 0, 30, and 60 minutes.

Code Block 8.10: Memory Leak Detection Test

```
package perf

import (
    "context"
    "fmt"
    "os"
    "runtime"
    "runtime/pprof"
    "testing"
    "time"

    "golang.org/x/sync/errgroup"
)

func TestSustainedLoad_OneHour(t *testing.T) {
    if os.Getenv("RUN_SUSTAINED_TEST") == "" {
        t.Skip("set RUN_SUSTAINED_TEST=1")
    }

    db := setupBenchmarkDB(t)
    defer cleanupBenchmarkDB(t, db)
    engine, mockClient := setupScoringEngine(t, db)
    seedBenchmarkModels(t, db, mockClient, 50)

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    config := DefaultScoreWeights()

    heapProfiles := make([]*runtime.MemStats, 3)
    goroutineCounts := make([]int, 3)
    sampleTimes := []time.Duration{0, 30 * time.Minute, 60 * time.Minute}
```

```

g, ctx := errgroup.WithContext(ctx)
loadStart := time.Now()
for i := 0; i < 100; i++ {
    workerID := i
    g.Go(func() error {
        ticker := time.NewTicker(100 * time.Millisecond)
        defer ticker.Stop()
        reqCount := 0
        for {
            select {
            case <-ctx.Done():
                t.Logf("Worker %d: %d requests", workerID, reqCount)
                return nil
            case <-ticker.C:
                modelID := fmt.Sprintf("bench-model-%03d", reqCount%50)
                if _, err := engine.CalculateComprehensiveScore(ctx, modelID, config); err != nil {
                    return err
                }
                reqCount++
            }
        }
    })
}

for sampleIdx, delay := range sampleTimes {
    time.Sleep(delay - time.Since(loadStart))
    runtime.GC()
    time.Sleep(2 * time.Second)
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    heapProfiles[sampleIdx] = &m
    goroutineCounts[sampleIdx] = runtime.NumGoroutine()
    profilePath := fmt.Sprintf("/tmp/heap_profile_%d_min.prof", int(delay.Minutes()))
    f, _ := os.Create(profilePath)
    pprof.WriteHeapProfile(f)
    f.Close()
    t.Logf("Sample %d: heap=%dMB goroutines=%d", sampleIdx, m.HeapAlloc/1024/1024,
        ↪ goroutineCounts[sampleIdx])
}

cancel()
require.NoError(t, g.Wait())

heapGrowth := float64(heapProfiles[2].HeapAlloc - heapProfiles[1].HeapAlloc) /
    float64(heapProfiles[1].HeapAlloc) * 100
assert.Less(t, heapGrowth, 10.0, "heap grew %.2f%% in second 30min — possible leak", heapGrowth)

goroutineGrowth := goroutineCounts[2] - goroutineCounts[0]
assert.LessOrEqual(t, goroutineGrowth, 5, "goroutines grew by %d — possible leak", goroutineGrowth)

```

```
}
```

The test forces GC before each sample for consistent measurements. Anti-bluff checks assert heap growth under 10% in the second 30-minute period and goroutine growth under 5. Heap profiles are preserved as artifacts for post-hoc analysis. The `cancel()` followed by `g.Wait()` ensures all workers exit before the final sample, preventing the test itself from appearing as a leak source.

9. Phase 8: Documentation, Deployment, and Operational Runbook

Phase 8 closes the integration lifecycle by producing every artifact required for sustainable operations. This phase encompasses constitution governance for all three repositories, submodule documentation propagation, user-facing guides, an operational runbook with metric-driven alerting, Docker and Kubernetes deployment manifests, a migration and rollback plan with phased rollout gates, and a 50-item master checklist. Every configuration option introduced across Phases 1–7 must appear in at least one documented reference table. Nothing ships without a signature.

9.1 Constitution and Governance Documentation

The constitution defines the behavioral contract for all code that interacts with the LLMsVerifier subsystem. It is not aspirational text; it is enforced through CI gates, challenge tests, and mandatory code-review checklists.

9.1.1 Write `internal/verifier/CONSTITUTION.md`

The file `internal/verifier/CONSTITUTION.md` contains 33 mandatory rules grouped into six categories, adapted from the HelixAgent reference constitution but contextualized for HelixTranslate’s translation-domain requirements. Each rule carries a unique identifier, category, imperative text, rationale, enforcement mechanism, and violation consequence. The eight scoring rules (S01–S08) govern weight validation, threshold enforcement, score freshness with TTL-based recency penalties, normalization boundaries, component isolation to prevent cross-contamination of speed and cost signals, recalculation triggers on discovery events, and audit-trail persistence for every computed score. The eight verification rules (V01–V08) mandate pipeline integrity across all eight steps, enforce step isolation so a failure in latency benchmarking cannot mask an authentication failure, specify timeout values per step (5s for existence, 120s for latency), limit retries to three attempts per critical step, prohibit mock-only verification (anti-bluff), require a minimum of 193 passing challenges, enforce challenge coverage across all five scoring components, and mandate persistence of every verification result to SQLite. The five discovery rules (D01–D05) regulate refresh intervals per provider, establish fallback ordering from static registry to provider API to `models.dev`, require deduplication by model ID, define stale handling when a model has not been rediscovered within $2\times$ the provider’s refresh interval, and mandate provider health tracking with automatic degradation of unhealthy sources. The five selection rules (L01–L05) enforce fairness in the selection algorithm, require a diversity minimum of three distinct providers in any ranked list, implement load balancing across top-scored models, express tier preference for premium-tier models on quality-critical translations, and preserve user override capability. The four testing rules (T01–T04) codify the anti-bluff mandate, require 100% test coverage for the `internal/verifier/` package, mandate real infrastructure for all integration tests, and prohibit mock-only verification pipelines. The three operations rules (O01–O03)

require health endpoint monitoring, define alerting thresholds for score drops exceeding 10% within one hour, and mandate rollback capability through the `LLMSVERIFIER_ENABLED` feature flag.

9.1.2 Constitution Rule Template

Every rule in `internal/verifier/CONSTITUTION.md` follows the template shown in Table 9.1. This structure ensures that each rule is actionable, auditable, and enforceable without ambiguity.

Table 9.1: Constitution Rule Template with Representative Entries

ID	Category	Rule Text	Rationale	Enforcement Mechanism	Violation Consequence
S01	Scoring	All five component weights must sum to exactly 1.0	Normalization integrity prevents skewed composite scores	<code>validateWeights()</code> in <code>internal/verifier/scoring/components.go</code> returns error on mismatch	CI build fails; PR blocked
S04	Scoring	Recency penalty applies exponential decay with half-life 90 days	Older models depreciate naturally without manual intervention	<code>CalculateRecencyScore()</code> enforces time-decay formula; scores older than 24h are recalculated	Stale scores trigger warning alert; model may fall below threshold
V01	Verification	All eight pipeline steps must execute for every model before registry admission	Incomplete verification creates blind spots in capability detection	<code>RunPipeline()</code> in <code>internal/verifier/pipeline.go</code> requires all steps complete	Model cannot enter <code>verified_models</code> table; gatekeeper denies access
V06	Verification	A minimum of 193 challenges must pass before a model receives a capability score	Challenge count below 193 produces statistically unreliable capability metrics	<code>internal/verifier/challenges/runner.go</code> counts passes; returns error if count < 193	Score component set to 0.0; model flagged as unverified
T01	Testing	No test may rely exclusively on mocked LLM responses for verification pipeline coverage	Mock-only tests produce false confidence; anti-bluff mandate requires real behavior	<code>go test -cover</code> + manual review; integration tests require live verifier instance	PR rejected; coverage report flagged

Table 27 – continued

ID	Category	Rule Text	Rationale	Enforcement Mechanism	Violation Consequence
O03	Operations	Rollback to legacy provider factory must complete within 30 seconds via feature flag	Sustained degradation of verified model quality demands instant fallback	LLMSVERIFIER_ENABLED=false takes effect in next provider factory call; no restart required	Operational incident if rollback exceeds 30s SLA

9.1.3 Update CLAUDE.md

The root CLAUDE.md file (11KB) receives a new appendix section titled “LLMsVerifier Integration Context.” This section documents the architecture context: the verifier subsystem lives under `internal/verifier/` and provides a verified, scored, discovered model layer between `pkg/translator/llm/` and external LLM APIs. Key files are listed with one-line descriptions for all 15+ integration files: `internal/verifier/client.go` (verifier client), `internal/verifier/pipeline.go` (8-step pipeline), `internal/verifier/scoring/engine.go` (scoring engine), `internal/verifier/scoring/components.go` (5 component calculators), `internal/verifier/scoring/composite.go` (weighted aggregator), `internal/verifier/discovery/service.go` (3-tier discovery), `internal/verifier/discovery/registry.go` (model registry), `internal/verifier/discovery/gatekeeper.go` (access control), `internal/verifier/selection/engine.go` (model selector), `internal/verifier/selection/fallback.go` (fallback chain), `internal/services/llmsverifier_score_adapter.go` (score adapter), `internal/verifier/config.go` (verifier config), `internal/verifier/health.go` (health checks), `internal/verifier/events.go` (event handlers), and `internal/verifier/CONSTITUTION.md` (governance rules). The pipeline description covers the eight ordered steps (existence → connectivity → authentication → completion → capability detection → translation quality → latency benchmark → error handling) with per-step timeouts and criticality flags. The scoring weight reference table documents the default distribution: response speed 0.25, cost effectiveness 0.25, model efficiency 0.20, capability 0.20, recency 0.10. Common troubleshooting steps cover five scenarios: verifier client connection refused (check `LLMSVERIFIER_API_URL`), model not appearing in registry (run discovery or check gatekeeper threshold), score lower than expected (review component breakdown in logs), pipeline timeout on latency step (increase `LLMSVERIFIER_TIMEOUT`), and SQLite lock contention (reduce `MaxConcurrentTests`).

9.1.4 Update AGENTS.md

The root AGENTS.md file (20KB) receives a new appendix section titled “Verifier Agent Roles and Decision Trees.” This section defines three agent roles. The Discovery Agent is responsible for monitoring the 3-tier discovery system, reviewing discovery logs in `internal/verifier/discovery/`, and triggering manual discovery via the API when automated refresh fails. Its decision tree: on `ModelDiscoveryFailed` event, check provider API key validity → if valid, check provider health endpoint → if healthy, retry discovery → if still failing, log incident and proceed with cached models. The Verification Agent executes the 8-step pipeline, reviews step results, and investigates failures. Its decision tree: on `VerificationStepFailed` event, identify failed step → if critical step, block model and alert → if non-critical, log warning and continue → always publish results. The Scoring Agent monitors score trends, investigates drops, and recommends weight adjustments. Its decision tree: on `ScoreBelowThreshold` event, check recency (is

score stale?) → if stale, trigger recalculation → if fresh, check component breakdown → identify declining component → recommend action (retire model, adjust weights, or escalate). Context window constraints are specified: each agent receives at most 4,000 tokens of log context and must reference specific line numbers in source files when making recommendations.

9.2 Submodule Documentation Propagation

The HelixTranslate project maintains two Git submodules: `Challenges/` (at commit `3937f06`) and `Containers/` (at commit `f572d26`). Both submodules require their own `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md` updates. The `LLMsVerifier` repository itself (checked out at `./LLMsVerifier/`) is treated as a third submodule for documentation purposes.

9.2.1 Submodule Documentation Checklist

Table 9.2 enumerates the documentation requirements for each submodule. Every cell must carry a commit hash and timestamp at completion.

Table 9.2: Submodule Documentation Completeness Matrix

Document	Challenges Submodule	Containers Submodule	LLMsVerifier Submodule	Completion Gate
<code>CONSTITUTION.md</code>	Must add test integrity rules (T01–T04), anti-bluff mandates, and challenge authoring guidelines specific to translation evaluation	Must add container build rules, base image pinning rules, and security scanning rules	Must update existing constitution with HelixTranslate integration rules (S01–S08, V01–V08)	Signed commit on main branch
<code>CLAUDE.md</code>	Must append challenge system context: how to add challenges, how to run the challenge runner, expected pass criteria format	Must append container build context: multi-stage build rules, dependency cache optimization, base image selection	Must append integration context: how HelixTranslate consumes LLMsVerifier APIs, key exported functions, module boundary rules	PR merged; code review passed
<code>AGENTS.md</code>	Must define Challenge Author Agent role with decision tree for authoring new translation-quality challenges	Must define Build Agent role for container build and release decisions	Must define Provider Integration Agent role for adding new provider backends	AI agent smoke test passed
Submodule references in main project	<code>.gitmodules</code> entry verified	<code>.gitmodules</code> entry verified	<code>go.mod</code> replace directive verified	<code>git submodule status</code> returns clean

9.2.2 Write Challenges Submodule `CONSTITUTION.md`

The Challenges submodule `CONSTITUTION.md` focuses on test integrity. Rule TC01 mandates that every challenge must define an unambiguous pass criterion expressed as a deterministic check against expected

output, never as a subjective quality assessment. Rule TC02 prohibits challenges that rely on external network state; each challenge must declare its infrastructure dependencies in a REQUIREMENTS block. Rule TC03 requires that adding, removing, or modifying any challenge triggers a full re-verification of all 193+ challenges against a reference model to detect regression in challenge validity. Rule TC04 mandates anti-bluff design: no challenge may pass against a hardcoded response or a model that is known to be non-functional.

9.2.3 Write LLMsVerifier Submodule Documentation Updates

The LLMsVerifier repository receives two documentation updates. In CLAUDE.md, a new section titled “HelixTranslate Integration Context” documents the module boundary: `digital.vasic.llmsverifier` exports a public API surface consumed by `digital.vasic.translator` through the adapter in `internal/providers/llmsverifier/adapter.go`. Key exported types are listed: `llmverifier.Client`, `llmverifier.CompletionRequest`, `llmverifier.CompletionResponse`, `scoring.ScoringEngine`, `scoring.ComprehensiveScore`, `discovery.Service`, and `verification.Verifier`. In AGENTS.md, a Provider Integration Agent role is defined with the decision tree: on request to add provider N+1, check if provider implements OpenAI-compatible API → if yes, create adapter using existing OpenAI client pattern → if no, implement native provider in `providers/` directory → run 8-step verification → add to static registry → update documentation.

9.2.4 Documentation Update Checklist Script

A validation script ensures all submodule documentation files remain in sync with the main project. The script checks file existence, minimum line counts, reference cross-links, and timestamp freshness. Code block 9.1 shows the complete validation script.

```
#!/usr/bin/env bash
# scripts/validate-documentation.sh
# Validates that all CONSTITUTION.md, CLAUDE.md, and AGENTS.md files
# are present and cross-referenced across main project and submodules.

set -euo pipefail

REQUIRED_FILES=(
    "internal/verifier/CONSTITUTION.md"
    "CLAUDE.md"
    "AGENTS.md"
    "Challenges/CONSTITUTION.md"
    "Challenges/CLAUDE.md"
    "Challenges/AGENTS.md"
    "Containers/CLAUDE.md"
    "Containers/AGENTS.md"
    "LLMsVerifier/CLAUDE.md"
    "LLMsVerifier/AGENTS.md"
)

MIN_LINES=(150 200 300 100 150 200 80 120 200 200)

ERRORS=0
for i in "${!REQUIRED_FILES[@]}; do
    FILE="${REQUIRED_FILES[$i]}"
```

```

MIN="${MIN_LINES[$i]}"
if [[ ! -f "$FILE" ]]; then
    echo "ERROR: Missing required file: $FILE"
    ERRORS=$((ERRORS + 1))
elif [[ "$(wc -l < "$FILE")" -lt "$MIN" ]]; then
    echo "ERROR: $FILE has fewer than $MIN lines"
    ERRORS=$((ERRORS + 1))
else
    echo "OK: $FILE ($(wc -l < "$FILE") lines)"
fi
done

# Verify cross-references
if ! grep -q "LLMsVerifier" CLAUDE.md; then
    echo "ERROR: CLAUDE.md missing LLMsVerifier section"
    ERRORS=$((ERRORS + 1))
fi

if ! grep -q "internal/verifier/CONSTITUTION.md" AGENTS.md; then
    echo "ERROR: AGENTS.md missing verifier constitution reference"
    ERRORS=$((ERRORS + 1))
fi

if [[ $ERRORS -gt 0 ]]; then
    echo "Documentation validation FAILED with $ERRORS error(s)"
    exit 1
fi

echo "Documentation validation PASSED"

```

9.3 User-Facing Documentation

User-facing documentation explains how the LLMsVerifier integration affects translation workflows, what verified models mean, how to interpret scores, and how to diagnose common issues.

9.3.1 Write docs/llmsverifier-integration.md

The file docs/llmsverifier-integration.md provides a high-level overview for end users. It explains that verified models are those that have passed all eight steps of the verification pipeline: existence confirmation, API connectivity, authentication validation, basic completion, capability detection, translation quality assessment, latency benchmarking, and error handling verification. It describes how each model receives a composite score from 0.0 to 10.0, computed from five weighted components (response speed 25%, cost effectiveness 25%, model efficiency 20%, capability 20%, recency 10%). It documents the model selection UX: when a user initiates a translation, only verified models with scores above their tier threshold appear in the provider selection list. Unverified models are hidden by default and require explicit `show_unverified=true` query parameter to appear, at which point they display a warning badge. Score badges use a color scheme: excellent (9.0–10.0, green), very good (7.5–8.9, blue), good (6.0–7.4, yellow), fair (4.0–5.9, orange), poor (below 4.0, red). The filtering system allows users to filter by provider,

minimum score, capability tag (e.g., long-context, json-mode, streaming), and cost tier.

9.3.2 Environment Variables Reference

Every environment variable introduced or consumed by the LLMsVerifier integration appears in Table 9.3. This table is the single source of truth for operational configuration.

Table 9.3: Complete Environment Variables Reference (33+ Variables)

Variable Name	Type	Default	Required	Description	Example Value
LLMSVERIFIER_ENABLED	bool	false	No	Master enable switch for the LLMsVerifier subsystem	true
LLMSVERIFIER_API_URL	string	http://localhost:8080	Yes	Base URL of the LLMsVerifier service	https://verifier.internal:8443
LLMSVERIFIER_API_KEY	string	" "	Yes	API key for authenticating with LLMsVerifier	lv_live_abc123...
LLMSVERIFIER_DB_PATH	string	./data/verifier.db	No	SQLite database path for local score caching	/var/lib/helix/verifier.db
LLMSVERIFIER_CACHE_TTL	duration	1h	No	Time-to-live for cached scores before recalculation	30m
LLMSVERIFIER_VERIFICATION_ENABLED	bool	true	No	Enable the 8-step verification pipeline	true
LLMSVERIFIER_SCORING_ENABLED	bool	true	No	Enable 5-component score calculation	true
LLMSVERIFIER_DISCOVERY_ENABLED	bool	true	No	Enable 3-tier model discovery	true
LLMSVERIFIER_MAX_CONCURRENT	int	5	No	Maximum concurrent verification tests	3
LLMSVERIFIER_TIMEOUT	duration	30s	No	Per-step verification timeout	60s
LLMSVERIFIER_WEIGHT_SPEED	float	0.25	No	Scoring weight: response speed	0.20
LLMSVERIFIER_WEIGHT_COST	float	0.25	No	Scoring weight: cost effectiveness	0.30
LLMSVERIFIER_WEIGHT EFFICIENCY	float	0.20	No	Scoring weight: model efficiency	0.20
LLMSVERIFIER_WEIGHT CAPABILITY	float	0.20	No	Scoring weight: capability	0.20
LLMSVERIFIER_WEIGHT RECENCY	float	0.10	No	Scoring weight: recency	0.10

Table 29 – continued

Variable Name	Type	Default	Required	Description	Example Value
OPENAI_API_KEY	string	""	Yes (if using OpenAI)	OpenAI API authentication key	sk-proj-...
ANTHROPIC_API_KEY	string	""	Yes (if using Anthropic)	Anthropic API authentication key	sk-ant-...
DEEPSEEK_API_KEY	string	""	Yes (if using DeepSeek)	DeepSeek API authentication key	sk-ds-...
GROQ_API_KEY	string	""	Yes (if using Groq)	Groq API authentication key	gsk_...
MISTRAL_API_KEY	string	""	Yes (if using Mistral)	Mistral API authentication key	...
COHERE_API_KEY	string	""	Yes (if using Cohere)	Cohere API authentication key	...
XAI_API_KEY	string	""	Yes (if using xAI)	xAI API authentication key	...
TOGETHER_API_KEY	string	""	Yes (if using Together)	Together AI API key	...
OPENROUTER_API_KEY	string	""	Yes (if using OpenRouter)	OpenRouter API key	sk-or-...
CLOUDFLARE_API_KEY	string	""	Yes (if using Cloudflare)	Cloudflare AI API key	...
SAMBAOVA_API_KEY	string	""	Yes (if using SambaNova)	SambaNova API key	...
NOVITA_API_KEY	string	""	Yes (if using Novita)	Novita API key	...
MOONSHOT_API_KEY	string	""	Yes (if using Moonshot)	Moonshot API key	...
QWEN_API_KEY	string	""	Yes (if using Qwen)	Qwen API authentication key	sk-...
REPLICATE_API_TOKEN	string	""	Yes (if using Replicate)	Replicate API token	r8_...
HYPERBOLIC_API_KEY	string	""	Yes (if using Hyperbolic)	Hyperbolic API key	...
CEREBRAS_API_KEY	string	""	Yes (if using Cerebras)	Cerebras API key	...
SILICONFLOW_API_KEY	string	""	Yes (if using SiliconFlow)	SiliconFlow API key	...
FIREWORKS_API_KEY	string	""	Yes (if using Fireworks)	Fireworks AI API key	...
PERPLEXITY_API_KEY	string	""	Yes (if using Perplexity)	Perplexity API key	pplx-...
AI21_API_KEY	string	""	Yes (if using AI21)	AI21 Studio API key	...
ZHIPU_API_KEY	string	""	Yes (if using Zhipu)	Zhipu API authentication key	...

9.3.3 Document Model Selection UX

The model selection user experience follows a gatekeeping pattern. When a user opens the provider selection interface, the system queries `internal/verifier/discovery/registry.go` for all models where `verification_status = 'verified'` and `composite_score >= tier_threshold`. Models failing either condition are excluded from the default list. Each displayed model shows its name, provider, composite score badge (color-coded per range from Section 9.3.1), capability tags (chips for `streaming`, `json-mode`, `long-context`, `function-calling`), and cost tier indicator (premium, standard, budget). Users may apply filters: provider multi-select, minimum score slider (0–10), capability tag toggle, and cost tier multi-select. The sort default is composite score descending; alternatives are latency ascending, cost per token ascending, and recency descending. Users may override the gatekeeper by enabling “Show All Models” in advanced settings, which reveals unverified models with a red warning badge and requires explicit confirmation before use.

9.3.4 Write docs/troubleshooting.md

The troubleshooting guide covers ten common issues. Issue 1: “Model does not appear in selection list” — diagnosis: check verification status via `GET /api/v1/models/{id}`, check composite score against tier threshold, check discovery logs; resolution: trigger manual discovery or lower the minimum score filter. Issue 2: “Model score dropped unexpectedly” — diagnosis: check `internal/verifier/scoring/history.go` for trend data, review recency penalty application, check if provider pricing changed; resolution: force score recalculation via `POST /api/v1/score/calculate` or temporarily reduce weight on the declining component. Issue 3: “Discovery service returns no models for a provider” — diagnosis: check provider API key validity, check provider health endpoint response, review `discovery_service.go` logs for timeout or HTTP error; resolution: renew API key, increase provider timeout, or fall back to static registry. Issue 4: “API key reported as invalid” — diagnosis: verify key format matches provider expectations, check key has not expired, confirm key has required scopes; resolution: regenerate key in provider console, update environment variable, restart HelixTranslate. Issue 5: “Verification pipeline times out consistently” — diagnosis: check `LLMSVERIFIER_TIMEOUT` value against provider latency, check network path to provider, review per-step timing in pipeline logs; resolution: increase timeout, enable provider retry, or skip non-critical steps. Issue 6: “Score badge shows 0.0 for a known-good model” — diagnosis: check if capability component is 0.0 due to insufficient challenge passes (< 193), check scoring engine error logs; resolution: re-run challenges or reduce minimum challenge threshold in development. Issue 7: “SQLite database locked errors” — diagnosis: check concurrent access from multiple HelixTranslate instances, review `MaxConcurrentTests` setting; resolution: use PostgreSQL backend for shared state, reduce concurrency, or implement connection pooling. Issue 8: “Translation uses unverified model despite gatekeeper enabled” — diagnosis: check if user override was activated, verify gatekeeper is not bypassed in API path; resolution: disable user override in config, add middleware check. Issue 9: “Event bus missing verifier events” — diagnosis: check `EventIntegrationConfig.Enabled`, verify event type subscriptions in `internal/verifier/event_subscriber.go`; resolution: enable event integration, restart subscriber. Issue 10: “High memory usage from score caching” — diagnosis: check cache size in `ScoreCache.entries`, review TTL settings; resolution: reduce `LLMSVERIFIER_CACHE_TTL`, enable cache size limits, or switch to Redis-backed cache.

9.4 Operational Runbook

The operational runbook defines standard operating procedures for the four most common incident types. Each procedure follows detect-diagnose-remediate-verify pattern.

9.4.1 Monitoring Metrics Dashboard

Table 9.4 defines the metrics that operators monitor, their PromQL-style query expressions, alert thresholds, severity levels, and runbook cross-references. In environments without Prometheus, equivalent queries against the SQLite metrics table in `internal/verifier/scoring/history.go` apply.

Table 9.4: Monitoring Metrics Dashboard

Metric Name	Query / Check	Threshold	Severity	Runbook Link
verifier_pipeline_failures_per_hour	<pre>SELECT COUNT(*) FROM verification_results WHERE passed = false AND created_at > now() - interval '1 hour'</pre>	> 5 per hour	P2 (High)	Section 9.4.3
model_score_drop_percent	<pre>SELECT (prev_score - curr_score) / prev_score * 100 FROM score_history WHERE model_id = \$1 ORDER BY timestamp DESC LIMIT 2</pre>	> 10% within 1 hour	P1 (Critical)	Section 9.4.3
discovery_service_unavailable	<pre>SELECT COUNT(*) FROM discovery_runs WHERE status = 'failed' AND provider = \$1 AND created_at > now() - interval '30 minutes'</pre>	> 3 consecutive failures	P2 (High)	Section 9.4.4
gatekeeper_denial_rate	<pre>SELECT COUNT(*) FROM gatekeeper_decisions WHERE allowed = false AND created_at > now() - interval '1 hour' / total requests</pre>	> 20% of requests	P3 (Medium)	Section 9.3.4 Issue 8
verifier_api_latency_p95	<pre>SELECT percentile_cont(0.95) WITHIN GROUP (ORDER BY duration_ms) FROM api_calls WHERE endpoint LIKE '/api/ v1/verify%' AND created_at > now() - interval '5 minutes'</pre>	> 5000ms	P2 (High)	Increase <code>MaxConcurrentTests</code> or scale verifier service
sqlite_db_size_bytes	<pre>stat -c%s /data/ verifier.db</pre>	> 1GB	P3 (Medium)	Run compaction, archive old scores

Table 30 – continued

Metric Name	Query / Check	Threshold	Severity	Runbook Link
challenge_pass_rate	SELECT passed_count / total_count * 100 FROM challenge_runs WHERE model_id = \$1 AND created_at > now() - interval '24 hours'	< 80% for any verified model	P2 (High)	Re-run verification pipeline
provider_api_error_rate	SELECT COUNT(*) FROM provider_calls WHERE status >= 500 AND created_at > now() - interval '5 minutes' / total calls	> 5%	P3 (Medium)	Check provider status page, switch provider
score_cache_hit_rate	cache_hits / (cache_hits + cache_misses)	< 50%	P4 (Low)	Review TTL, increase cache size
rollback_activation_time_ms	Time from LLMSVERI- FIER_ENABLED=false to last verified model request	> 30000ms	P1 (Critical)	Section 9.6.3

9.4.2 Key Log Patterns

Operators grep for five log patterns during incident diagnosis. Pattern 1 — successful verification: `msg="verification pipeline passed" model_id="<id>" steps_passed=8/8 duration_ms=<n>` appears at INFO level in `internal/verifier/pipeline.go` when all eight steps complete. Pattern 2 — score change: `msg="model score updated" model_id="<id>" previous=<old> current=<new> delta=<delta>` appears at INFO level in `internal/verifier/scoring/composite.go` whenever a score recalculation produces a different composite value. Pattern 3 — model retirement: `msg="model retired from registry" model_id="<id>" reason="score_below_threshold" score=<score> threshold=<threshold>` appears at WARN level in `internal/verifier/discovery/gatekeeper.go` when a model's score drops below its tier threshold for more than the grace period. Pattern 4 — discovery failure: `msg="discovery failed for provider" provider="<name>" error="<message>" consecutive_failures=<n>` appears at ERROR level in `internal/verifier/discovery/service.go`; three consecutive occurrences trigger the fallback to static registry. Pattern 5 — gatekeeping denial: `msg="gatekeeper denied model access" model_id="<id>" reason="<verification_failed|score_below_threshold|provider_unhealthy>"` appears at WARN level in `internal/verifier/discovery/gatekeeper.go`.

9.4.3 Runbook: Model Score Dropped Below Threshold

Detection: The `model_score_drop_percent` alert fires when any verified model's composite score drops by more than 10% within one hour, or the `verifier_pipeline_failures_total` alert fires when more than five pipelines fail in one hour. Both alerts route to the on-call engineer via PagerDuty with severity P1.

Diagnosis: The on-call engineer runs three queries in sequence. First, fetch the component-level breakdown: `SELECT component, score FROM score_components WHERE model_id = $1 ORDER BY calculated_at`

DESC LIMIT 5 to identify which component (speed, cost, efficiency, capability, recency) drove the drop. Second, check the model's recent challenge results: `SELECT challenge_id, passed FROM challenge_results WHERE model_id = $1 AND created_at > now() - interval '24 hours'` to determine if a provider-side degradation affected capability scores. Third, review provider health: `SELECT status, latency_ms FROM provider_health WHERE provider = $1 ORDER BY checked_at DESC LIMIT 10` to confirm the provider API is responsive.

Remediation: If the score drop is due to stale data (score older than cache TTL), trigger a forced recalculation via `POST /api/v1/score/calculate` with the model ID. If the drop reflects genuine provider degradation and the model is non-critical, allow the gatekeeper to retire it automatically by waiting for the grace period (default 15 minutes). If the model is critical for production workloads, temporarily lower the minimum score threshold via `LLMSVERIFIER_MIN_SCORE` environment variable (requires service restart) while the provider resolves the issue. Document all actions in the incident log.

9.4.4 Runbook: Discovery Service Failure

Detection: The `discovery_service_unavailable` alert fires when a provider's discovery endpoint fails three or more consecutive times within 30 minutes. The alert includes the provider name and last error message.

Diagnosis: Check provider API key validity by running a direct curl to the provider's model list endpoint with the stored key. If the key is valid, check the provider's public status page. Review `internal/verifier/discovery/service.go` logs for the specific HTTP status code and response body from the failed discovery call.

Remediation — Automatic Fallback: The discovery service automatically falls back to the static registry for the affected provider. Models from the last successful discovery remain available but are marked with `source_tier = 'static'`. No operator action is required for fallback activation.

Remediation — Manual Model Registration: If the provider failure is prolonged and critical models are needed, register models manually via `POST /api/v1/models/register` with a JSON payload containing `model_id`, `provider`, `name`, `context_window`, and `capabilities`. Manually registered models receive a provisional score of 5.0 and are tagged with `registration_type = 'manual'`.

9.4.5 Runbook: New Provider Onboarding

Adding provider 26+ to the verified pipeline follows a 10-step procedure. Step 1: Obtain API credentials and add the `{PROVIDER}_API_KEY` environment variable to the deployment configuration. Step 2: Add provider metadata to the static registry in `internal/verifier/discovery/registry.go` including known model IDs, context windows, and endpoint URLs. Step 3: Implement the provider adapter in `internal/providers/llmsverifier/` following the OpenAI-compatible pattern if applicable, or create a native implementation. Step 4: Add the provider to the discovery service configuration in `configs/verifier.yaml` under `providers_to_discover`. Step 5: Deploy to the staging environment and trigger manual discovery via `POST /api/v1/models/discover`. Step 6: Run the 8-step verification pipeline against each discovered model. Step 7: Verify that all models score above the minimum threshold for their tier; if not, investigate failing components. Step 8: Add provider-specific challenges to the Challenges submodule if the provider introduces novel capabilities. Step 9: Update user-facing documentation in `docs/llmsverifier-integration.md` with the new provider's name, supported models, and pricing tier. Step 10: Enable the provider in production via feature flag and monitor for 24 hours before declaring onboarding complete.

9.5 Deployment Configuration

The LLMsVerifier service deploys as a sidecar to the main HelixTranslate application, communicating over HTTP on port 8080 with an SQLite database persisted to a Docker volume.

9.5.1 Docker Compose Service Definition

The docker-compose.yml file at the project root receives a new llms-verifier service. This service uses the helix-translate/llms-verifier:latest image built from the ./LLMsVerifier/ directory, exposes port 8080, mounts a named volume verifier-data to /data for SQLite persistence, connects to the existing helix-network bridge network, and defines a health check endpoint. Resource limits of 2 CPU cores and 2GB memory apply for production; these are overridden per environment in Section 9.5.4.

9.5.2 Docker Compose Verifier Service

Code block 9.2 shows the complete verifier service definition for docker-compose.yml.

```
llms-verifier:
  image: helix-translate/llms-verifier:latest
  build:
    context: ./LLMsVerifier
    dockerfile: Dockerfile
  container_name: helix-llms-verifier
  ports:
    - "8080:8080"
  volumes:
    - verifier-data:/data
  networks:
    - helix-network
  environment:
    - DB_PATH=/data/verifier.db
    - API_PORT=8080
    - LOG_LEVEL=info
    - LLMSVERIFIER_API_KEY=${LLMSVERIFIER_API_KEY}
    - OPENAI_API_KEY=${OPENAI_API_KEY}
    - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
    - GROQ_API_KEY=${GROQ_API_KEY}
    - MISTRAL_API_KEY=${MISTRAL_API_KEY}
    - COHERE_API_KEY=${COHERE_API_KEY}
    - XAI_API_KEY=${XAI_API_KEY}
    - TOGETHER_API_KEY=${TOGETHER_API_KEY}
    - SAMBANOVA_API_KEY=${SAMBANOVA_API_KEY}
    - NOVITA_API_KEY=${NOVITA_API_KEY}
    - QWEN_API_KEY=${QWEN_API_KEY}
    - REPLICATE_API_TOKEN=${REPLICATE_API_TOKEN}
    - HYPERBOLIC_API_KEY=${HYPERBOLIC_API_KEY}
    - CEREBRAS_API_KEY=${CEREBRAS_API_KEY}
    - SILICONFLOW_API_KEY=${SILICONFLOW_API_KEY}
    - FIREWORKS_API_KEY=${FIREWORKS_API_KEY}
    - PERPLEXITY_API_KEY=${PERPLEXITY_API_KEY}
    - AI21_API_KEY=${AI21_API_KEY}
```

```

- ZHIPU_API_KEY=${ZHIPU_API_KEY}
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "http://localhost:8080/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 15s
deploy:
  resources:
    limits:
      cpus: '2'
      memory: 2G
    reservations:
      cpus: '1'
      memory: 512M
  restart: unless-stopped
  depends_on:
    - api-server

```

The named volume `verifier-data` must be added to the volumes section at the bottom of `docker-compose.yml`:

```

volumes:
  verifier-data:
    driver: local

```

9.5.3 Kubernetes Manifests

Three Kubernetes manifests deploy the verifier service in production. The `verifier-deployment.yaml` defines a Deployment with two replicas for high availability, liveness and readiness probes against `/health`, pod anti-affinity to spread replicas across nodes, and resource requests/limits. The `verifier-service.yaml` exposes the deployment as a ClusterIP service on port 8080. The `verifier-configmap.yaml` holds non-sensitive configuration: `API_PORT`, `LOG_LEVEL`, `DB_PATH`, scoring weights, discovery intervals, and verification timeouts. API keys are injected via a Kubernetes Secret named `llmsverifier-api-keys` mounted as environment variables from the deployment's `envFrom` directive.

Code block 9.3 shows the complete `verifier-deployment.yaml` manifest.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: llms-verifier
  namespace: helix-translate
  labels:
    app: llms-verifier
    component: verification
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llms-verifier

```

```

template:
  metadata:
    labels:
      app: llms-verifier
      component: verification
  spec:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - llms-verifier
              topologyKey: kubernetes.io/hostname
    containers:
      - name: verifier
        image: helix-translate/llms-verifier:latest
        ports:
          - containerPort: 8080
            name: http
        envFrom:
          - configMapRef:
              name: verifier-config
          - secretRef:
              name: llmsverifier-api-keys
        volumeMounts:
          - name: verifier-data
            mountPath: /data
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 15
          periodSeconds: 30
          timeoutSeconds: 10
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 2
        resources:
          requests:
            cpu: "1"

```

```
        memory: "1Gi"
      limits:
        cpu: "2"
        memory: "2Gi"
    volumes:
      - name: verifier-data
        persistentVolumeClaim:
          claimName: verifier-data-pvc
```

9.5.4 Resource Requirements Per Environment

Table 9.5 specifies CPU, memory, storage, replica count, and high-availability configuration for each deployment environment.

Table 9.5: Resource Requirements Per Environment

Environment	CPU Request	CPU Limit	Memory Request	Memory Limit	Storage	Replicas	HA Config
Development	0.5	1.0	512MB	1GB	SQLite local file, 100MB	1	None
Staging	1.0	2.0	1GB	2GB	SQLite on PVC, 500MB	1	Single pod, daily backup

Table 31 – continued

Environment	CPU Request	CPU Limit	Memory Request	Memory Limit	Storage	Replicas	HA Config
Production	2.0	4.0	2GB	4GB	SQLite on PVC (or Post- greSQL), 2GB	2	Pod anti- affinity across zones

9.6 Migration and Rollback Plan

The migration from the legacy provider factory to the LLMsVerifier-backed pipeline uses a feature flag for gradual, reversible rollout.

9.6.1 Migration Script

The `scripts/migrate-to-verifier.sh` script automates the rollout. It checks prerequisites (database connectivity, feature flag availability, verifier service health), runs the database migration to create `verified_models` and `score_history` tables, copies existing model configurations from `internal/config/` into the verifier registry with initial scores computed from default weights, and toggles the feature flag. Code block 9.4 shows the script.

```
#!/usr/bin/env bash
# scripts/migrate-to-verifier.sh
# Gradual migration from legacy provider factory to LLMsVerifier-backed pipeline.

set -euo pipefail

PHASE=${1:-"canary"} # canary | early-access | majority | full
DB_PATH=${DB_PATH:-"./data/verifier.db"}
API_URL=${LLMSVERIFIER_API_URL:-"http://localhost:8080"}
FEATURE_FLAG_KEY="use_verified_pipeline"

echo "=== LLMsVerifier Migration: $PHASE ==="

# Step 1: Verify prerequisites
echo "[1/6] Checking prerequisites..."
if ! curl -sf "${API_URL}/health" > /dev/null 2>&1; then
    echo "ERROR: LLMsVerifier health check failed at $API_URL"
    exit 1
```

```

fi

# Step 2: Run database migration
echo "[2/6] Running database migration..."
sqlite3 "$DB_PATH" << 'EOF'
CREATE TABLE IF NOT EXISTS verified_models (
    model_id TEXT PRIMARY KEY,
    provider TEXT NOT NULL,
    name TEXT NOT NULL,
    verified_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    verification_status TEXT DEFAULT 'pending',
    composite_score REAL DEFAULT 0.0,
    component_speed REAL DEFAULT 0.0,
    component_cost REAL DEFAULT 0.0,
    component_efficiency REAL DEFAULT 0.0,
    component_capability REAL DEFAULT 0.0,
    component_recency REAL DEFAULT 0.0,
    tier TEXT DEFAULT 'standard',
    capabilities TEXT,
    source_tier TEXT DEFAULT 'static'
);

CREATE TABLE IF NOT EXISTS score_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_id TEXT NOT NULL,
    composite_score REAL NOT NULL,
    calculated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (model_id) REFERENCES verified_models(model_id)
);

CREATE INDEX IF NOT EXISTS idx_score_history_model_time
    ON score_history(model_id, calculated_at DESC);

CREATE TABLE IF NOT EXISTS discovery_runs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    provider TEXT NOT NULL,
    status TEXT NOT NULL,
    models_found INTEGER DEFAULT 0,
    error TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

EOF

# Step 3: Import existing model configs
echo "[3/6] Importing existing model configurations..."
go run scripts/import_model_configs.go --db="$DB_PATH" --config="config.json"

# Step 4: Compute initial scores
echo "[4/6] Computing initial scores..."
curl -sf -X POST "${API_URL}/api/v1/score/calculate-all" \
    -H "Authorization: Bearer ${LLMSVERIFIER_API_KEY}" || {
    echo "WARNING: Bulk score calculation failed; scores will be computed on demand"
}

```

```

}

# Step 5: Set feature flag percentage
echo "[5/6] Setting feature flag for phase: $PHASE..."
case "$PHASE" in
    canary)
        PERCENTAGE=5
        ;;
    early-access)
        PERCENTAGE=25
        ;;
    majority)
        PERCENTAGE=50
        ;;
    full)
        PERCENTAGE=100
        ;;
    *)
        echo "ERROR: Unknown phase: $PHASE"
        exit 1
        ;;
esac

# Step 6: Verify rollout
echo "[6/6] Verifying rollout at ${PERCENTAGE}%..."
sleep 5
curl -sf "${API_URL}/api/v1/status?feature_flag=${FEATURE_FLAG_KEY}" | \
jq -e ".rollout_percentage == ${PERCENTAGE}" || {
    echo "ERROR: Feature flag verification failed"
    exit 1
}

echo "=== Migration to $PHASE (${PERCENTAGE}%) completed successfully ==="

```

9.6.2 Rollout Phases

The rollout proceeds through four phases with explicit entry and exit criteria. Phase 1 — Canary (5% traffic, 1 day): deploy to 5% of production traffic. Exit criteria: error rate remains below 0.1% and no P1/P2 alerts fire for 24 hours. Phase 2 — Early Access (25% traffic, 3 days): expand to 25% of traffic. Exit criteria: user satisfaction score exceeds 4.0 out of 5.0 from feedback surveys, and average translation latency does not regress by more than 5% compared to the legacy pipeline. Phase 3 — Majority (50% traffic, 1 week): expand to 50% of traffic. Exit criteria: all performance SLAs met (p95 latency < 5s, cache hit rate > 80%), and the verification pipeline completes successfully for at least 95% of models. Phase 4 — Full (100% traffic, 2 weeks): complete cutover. Exit criteria: all quality gates pass (100% test coverage confirmed, all 193+ challenges passing, all 14 documentation files complete), and no critical incidents for 14 consecutive days.

9.6.3 Rollback Procedure

Rollback is instantaneous and requires zero downtime. The feature flag `LLMSVERIFIER_ENABLED` controls whether `pkg/translator/llm/llm.go` uses the verified pipeline or the legacy provider factory. Setting `LLMSVERIFIER_ENABLED=false` causes the next provider factory invocation to bypass `internal/verifier/discovery/gatekeeper.go` and construct the provider directly from `internal/config/config.go` as in the pre-integration codebase. Code block 9.5 shows the rollback toggle.

```
#!/usr/bin/env bash
# scripts/rollback-to-legacy.sh
# Instant rollback from LLMSVerifier to legacy provider factory.

set -euo pipefail

echo "=== Initiating emergency rollback ==="

# Method 1: Environment variable (takes effect on next request, no restart)
export LLMSVERIFIER_ENABLED=false
curl -X POST "http://localhost:8080/admin/feature-flags" \
  -H "Authorization: Bearer ${ADMIN_API_KEY}" \
  -d '{"LLMSVERIFIER_ENABLED": false}'

# Method 2: Config file update (persists across restarts)
jq '.llmsverifier.enabled = false' config.json > config.json.tmp
mv config.json.tmp config.json

# Verify rollback
echo "Verifying rollback..."
sleep 2
LEGACY_CALLS=$(curl -sf "http://localhost:8080/metrics" | \
  grep 'provider_factory_legacy_calls_total' | tail -1 | awk '{print $2}')
if [[ "$LEGACY_CALLS" -gt 0 ]]; then
  echo "Rollback verified: legacy provider factory is active"
else
  echo "WARNING: Rollback verification inconclusive; check metrics manually"
fi

echo "=== Rollback complete ==="
```

9.6.4 Data Migration

Existing model configurations in `internal/config/config.go` migrate to the `verified_models` SQLite table. The migration preserves provider name, model ID, API key reference (not the key itself), base URL, and assigned tier. Each migrated model receives an initial composite score of 5.0 (the “fair” baseline) and a `source_tier` value of `migrated`. Scores are recomputed through the full 5-component pipeline within 24 hours of migration. The migration is idempotent: running `scripts/migrate-to-verifier.sh` multiple times does not duplicate entries because the `model_id` column is the primary key.

9.7 Final Checklist and Definition of Done

9.7.1 Master Checklist

The master checklist contains 50 items across all eight phases. Each item carries a status column (pending / in-progress / complete), owner assignment (engineer name or team), and a sign-off signature with date. Phase 1 (Foundation) contains 8 items: module integration in `go.mod`, directory structure creation, `LLMsVerifierConfig` struct definition, environment variable loading, interface alignment in `internal/verifier/bridge.go`, constitution file creation, `CLAUDE.md` update, and `AGENTS.md` update. Phase 2 (Verification) contains 8 items: `NewVerifierClient()` implementation, health check endpoint, 8-step pipeline definition, existence verification, responsive/feature detection, code visibility and coding challenges, performance benchmarking, and cost analysis integration. Phase 3 (Scoring) contains 6 items: scoring engine initialization, five-component score calculation, composite score aggregation, score adapter service, score persistence and history, and threshold management. Phase 4 (Discovery) contains 6 items: 3-tier discovery service, provider API enumeration, model registry CRUD, gatekeeper implementation, background sync runner, and model catalog API endpoints. Phase 5 (Runtime) contains 6 items: verified provider factory, runtime model selection engine, fallback chain, event bus integration, API key validation, and provider expansion to 30+. Phase 6 (UX) contains 4 items: model selection interface, score badge rendering, filtering system, and unverified model warning flow. Phase 7 (Testing) contains 7 items: unit tests for verifier client, pipeline tests, scoring engine tests, score adapter tests, discovery and registry tests, integration tests with live verifier, and anti-bluff challenge suite (4 challenges, 193+ total). Phase 8 (Documentation) contains 5 items: all constitution files signed, all `CLAUDE.md` and `AGENTS.md` updates merged, user-facing documentation published, operational runbook reviewed by on-call team, and deployment manifests validated in staging.

9.7.2 Quality Gates

Three quality gates block final sign-off. Gate 1 — Test Coverage: `go test -cover ./internal/verifier/...` must report 100% statement coverage. The coverage report is generated with `go test -coverprofile=coverage.out ./internal/verifier/... && go tool cover -html=coverage.out -o coverage.html` and reviewed by the test lead. Gate 2 — Challenge Pass: all 193+ challenges must pass when executed against a live `LLMsVerifier` instance with real provider API keys. The challenge run is initiated with `go test -tags=challenge ./internal/verifier/challenges/...` and the output is archived. Gate 3 — Documentation Completeness: all 14 documentation files (3 `CONSTITUTION.md`, 3 `CLAUDE.md`, 3 `AGENTS.md`, 2 `README.md`, `docs/llmsverifier-integration.md`, `docs/troubleshooting.md`, `internal/verifier/README.md`) must pass the validation script from Section 9.2.4.

9.7.3 Anti-Bluff Verification

The anti-bluff verification is a manual procedure performed once after all automated gates pass. A senior engineer runs the full integration test suite against a live `LLMsVerifier` instance configured with real (not mocked) provider API keys for at least three distinct providers. The test sequence follows the complete user journey: trigger discovery, verify a model, compute its score, select it for translation, execute a translation, and confirm the model appears in the score history. The engineer pastes terminal output into the sign-off document. If any step produces a result that contradicts the automated test suite, the discrepancy is investigated as a potential test-bluff issue.

9.7.4 Performance Verification

Performance verification confirms all SLA targets under production-like load. The load test generates 1,000 concurrent translation requests using the verified model selection pipeline. Targets: p50 latency below 500ms, p95 latency below 2,000ms, p99 latency below 5,000ms, score cache hit rate above 80%, zero goroutine leaks after 30 minutes of sustained load, and memory growth below 10% over the same 30-minute window. The load test is executed with `go test -tags=perf -run TestVerifierLoad ./test/performance/...` against the staging environment. Results are captured with `go test -memprofile=mem.prof -cpuprofile=cpu.prof` and reviewed for hot paths before sign-off.